Coding Intros

Prof. Dr. Dominik Liebl

Last Updated on November 06, 2026

Table of contents

Preface **3**
Downloads . . . 3
On Using LLMs . . . 3
Why Do We Use R and Python? . . . 3
Some Additional References and Tutorials for R . . . 4
Some Additional References and Tutorials for Python . . . 4

I Introduction to R **5**

1 The First Steps in R **6**
1.1 Positron for the First Time . . . 6
1.2 Simple Calculations . . . 6
1.3 Assigments and Matrix Algebra . . . 7
1.4 Operators . . . 13
1.5 Further Data Objects . . . 15
1.6 R Help . . . 15
1.7 R Packages . . . 16
1.8 Exercises I . . . 16

2 Create and Load Data Sets **19**
2.1 Simulated Data . . . 19
2.2 Load Datasets . . . 20
2.3 Save Datasets . . . 21
2.4 Exercises II . . . 21

3 Simple Programming **23**
3.1 The For-Loop . . . 23
3.2 The If-Condition . . . 24
3.3 The While-Loop . . . 25
3.4 Time Saving and Loops . . . 26
3.5 Exercises III . . . 27

4 Functions **28**
4.1 Function Definition . . . 28
4.2 Recursion . . . 29
4.3 Exercises IV . . . 30

5 Plotting in R **31**
5.1 The Plot-Function . . . 31
5.2 Histograms and Kernel Densities . . . 35
5.3 Exercises V . . . 39

6 Monte Carlo Simulations **40**
6.1 Some Important Concepts . . . 40
6.1.1 Random Sample . . . 40
6.1.2 Estimator . . . 41
6.1.3 Bias, Variance, and Consistency of Estimators . . . 45
6.1.4 Asymptotic Normality and Confidence Intervals . . . 46
6.1.5 Why Monte Carlo Simulations? . . . 48
6.2 Simulation Example: The Sample Mean . . . 50
6.2.1 Step 1: Define the Estimator and the Questions . . . 50
6.2.2 Step 2: Choose the Simulation Design . . . 52
6.2.3 Step 3: Write the Simulation Code . . . 53
6.2.4 Step 4: Present the Results . . . 55
6.2.5 Step 5: Interpret the Results . . . 55
6.2.6 Blueprint: Designing Your Own Simulation . . . 57
6.3 Simulation Example: Hypothesis Testing . . . 58
6.3.1 Step 1: Define the Test and the Questions . . . 58
6.3.2 Step 2: Choose the Simulation Design . . . 59
6.3.3 Step 3: Write the Simulation Code . . . 59
6.3.4 Steps 4 and 5: Present and Interpret the Results . . . 60
6.4 Exercises VI . . . 60

7 Regression Analysis **62**
7.1 Basic Linear Model . . . 62
7.2 Empirical Example . . . 62
7.3 Exercises VII . . . 66

8 Some Notes on Useful Packages **68**
8.1 The tidyverse Family . . . 68
8.2 Tidymodels . . . 68
8.3 lmtest . . . 68
8.4 MASS . . . 68
8.5 AER - “Applied Econometrics with R” . . . 69
8.6 knitr . . . 69
8.7 Shiny . . . 69
8.8 Faster Code . . . 69

II Introduction to Python **70**

9 The First Steps in Python **71**
9.1 Positron for the First Time . . . 71

9.2 Simple Calculations . . . 72
9.3 Assigments and Matrix Algebra . . . 73
9.4 Operators . . . 78
9.5 Further Data Objects . . . 80
9.6 Python Help . . . 81
9.7 Python Packages . . . 81
9.8 Exercises I . . . 82

10 Create and Load Data Sets **84**
10.1 Simulated Data . . . 84
10.2 Load Datasets . . . 85
10.3 Save Datasets . . . 86
10.4 Exercises II . . . 86

11 Simple Programming **88**
11.1 The For-Loop . . . 88
11.2 The If-Condition . . . 89
11.3 The While-Loop . . . 90
11.4 Time Saving and Loops . . . 91
11.5 Exercises III . . . 92

12 Functions **93**
12.1 Function Definition . . . 93
12.2 Recursion . . . 94
12.3 Exercises IV . . . 95

13 Plotting in Python **96**
13.1 The Plot-Function . . . 96
13.2 Histograms and Kernel Densities . . . 101
13.3 Exercises V . . . 105

14 Monte Carlo Simulations **107**
14.1 Some Important Concepts . . . 107
14.1.1 Random Sample . . . 107
14.1.2 Estimator . . . 108
14.1.3 Bias, Variance, and Consistency of Estimators . . . 112
14.1.4 Asymptotic Normality and Confidence Intervals . . . 114
14.1.5 Why Monte Carlo Simulations? . . . 115
14.2 Simulation Example: The Sample Mean . . . 117
14.2.1 Step 1: Define the Estimator and the Questions . . . 117
14.2.2 Step 2: Choose the Simulation Design . . . 120
14.2.3 Step 3: Write the Simulation Code . . . 121
14.2.4 Step 4: Present the Results . . . 122
14.2.5 Step 5: Interpret the Results . . . 124
14.2.6 Blueprint: Designing Your Own Simulation . . . 125

14.3 Simulation Example: Hypothesis Testing . . . 125
14.3.1 Step 1: Define the Test and the Questions . . . 126
14.3.2 Step 2: Choose the Simulation Design . . . 126
14.3.3 Step 3: Write the Simulation Code . . . 126
14.3.4 Steps 4 and 5: Present and Interpret the Results . . . 127
14.4 Exercises VI . . . 127

15 Regression Analysis **129**
15.1 Basic Linear Model . . . 129
15.2 Empirical Example . . . 129
15.3 Exercises VII . . . 134

16 Some Notes on Useful Packages **136**
16.1 The PyData Family . . . 136
16.2 scikit-learn . . . 136
16.3 statsmodels . . . 136
16.4 SciPy . . . 137
16.5 linearmodels . . . 138
16.6 Jupyter and Quarto . . . 138
16.7 Streamlit and Shiny for Python . . . 138
16.8 Faster Code . . . 138

Preface

The purpose of this tutorial is to introduce the programming languages R and Python that are used during the lectures, in problem sets, and in the tutorials. The tutorial consists of two parallel parts:

- **Part I: Introduction to R**
- **Part II: Introduction to Python**

Both parts have the same structure: each chapter of the R part has a counterpart in the Python part that covers the same topics with the same examples and exercises. So you can work through the part of your preferred language, or compare how the same task is done in both languages.

Downloads

- R is free of charge and can be downloaded from https://cran.r-project.org/
- Python is free of charge and can be downloaded from https://www.python.org/downloads/
- The IDE Positron for R and Python is also free of charge and can be downloaded from https://positron.posit.co/download.html

On Using LLMs

Of course, you can also ask large language models (LLMs) such as ChatGPT or Claude. **But:** Don't fool yourself! Getting an answer from an LLM is not the same as learning and understanding.

Why Do We Use R and Python?

- R is free of charge and can be downloaded from https://cran.r-project.org/
- Python is free of charge and can be downloaded from https://www.python.org/downloads/
- The IDE Positron for R and Python is also free of charge and can be downloaded from https://positron.posit.co/download.html
- R is equipped with one of the most flexible and powerful graphic routines available anywhere. For instance, check out one of the following repositories:
  - Clean Graphs

  - Publication ready plots

- Today, R is the de-facto standard for statistical science.
- Python is one of the most popular programming languages overall and the de-facto standard for machine learning and data science.

Some Additional References and Tutorials for R

- The online textbook Introduction to Econometrics with R
- Reference card for R commands
- The official introduction to R
- www.r-project.org
- An R-package for learning R: www.swirl.com
- A book project which covers also advanced issues such as "writing performant code" and "package development": adv-r.had.co.nz
- Another excellent book: R for Data Science
- Creating dynamic graphics
- An introduction to statistical learning with R

Some Additional References and Tutorials for Python

- The official Python tutorial
- The online lectures Python Programming for Economics and Finance by QuantEcon
- Python for Data Analysis by Wes McKinney, the creator of pandas
- NumPy for MATLAB users
- Comparison of pandas with R
- An introduction to statistical learning with Python

Part I

Introduction to R

1 The First Steps in R

1.1 Positron for the First Time

I highly recommend to install Positron. For installation just follow the instructions on https://positron.posit.co/download.html

When you open Positron, its window is typically split into the following areas:

1. The editor (center) showing your script file, which is an ordinary text file with suffix `.R`. For instance, `yourFavoritFileName.R`. This file contains your code.
2. The *Session* pane (right) showing all objects (variables) we have defined in the environment.
3. The *Plots* and *Help* panes (right) showing plots and help files.
4. The *Console* (bottom), which allows you to type commands and display results.
5. The *Explorer* (left) showing the files in your current project folder.

Some other terms that will be useful:

- Working directory: The file-directory you are working in. Useful commands: with `getwd()` you get the location of your current working directory and `setwd()` allows you to set a new location for it. This is very useful when you would like to load or save several files. You can just set your working directory globally and all files can be loaded or will be saved into this directory.
- Workspace: This is a hidden file (stored in the working directory), where all objects you use (e.g., data, matrices, vectors, variables, functions, etc.) are stored. Useful commands: `ls()` shows all elements in our current workspace and `rm(list=ls())` deletes all elements in our current workspace. It is also possible to remove only some objects with `rm(object1, object2)`.

1.2 Simple Calculations

We start with a very basic calculation. Let's type `1+1` into the console window and press enter. We get

```
1+1 # first calculation
```

```
[1] 2
```

i.e. the result of the calculation is returned.

Note that everything that is written after the #-sign is ignored by R, which is very useful to comment your code. The second window above, starting with ##, shows the output.

Let's consider a second calculation

```
1+2+3+ # second calculation
```

```
Error: <text>:2:0: unexpected end of input
1: 1+2+3+ # second calculation
   ^
```

This calculation ended with an error. The reason is that the command can only processed if entered correctly.

```
1+2+3+4 # second calculation
```

```
[1] 10
```

A similar thing happens when parentheses are not closed.

```
 2*(2+3
```

```
Error: <text>:2:0: unexpected end of input
1:  2*(2+3
   ^
```

It will be helpful to use a script file, such as `yourFavoritFileName.R`, to store your R commands. Otherwise, you would have to type your code again when an error occurred. You can send single lines or marked regions of your R-code to the console by pressing the keys **STRG+ENTER**.

1.3 Assigments and Matrix Algebra

The **assignment** operator will be your most often used tool. Here is an example where a **scalar** variable is created:

```
x <- 9
x
```

```
[1] 9
```

```
x+1
```

```
[1] 10
```

Note: The <- assignment operator is an unusual syntax that you do not find in other programming languages. In many cases you can alternatively use the = operator. Typical R style uses the <- assignment operator due to some more in depth-differences and compatibility with older version of R. More on R style recommendations can be found in the tidyverse style guide. In simple settings, alternatives to <- are

```
x = 9
x
```

```
[1] 9
```

or even

```
4 -> y # possible but unusal
y
```

```
[1] 4
```

We consider now a vector, an object you will use frequently:

```
z <- c(1,3,5,6)
z
```

```
[1] 1 3 5 6
```

As discussed, `ls()` states the content of the workspace, whereas `rm(list =ls())` deletes the workspace.

```
ls()
```

```
[1] "x" "y" "z"
```

```
rm(list=ls())
ls()
```

```
character(0)
```

In the next step we will consider some vector multiplication. There are three different ways to multiply a vector, namely element by element, using the inner product, or using the outer product. Element by element gives you a vector of the same dimension.

```
z <- c(1,3,5,6)
z*z                #multiplication element by element
```

```
[1]  1  9 25 36
```

The function `t()` gives the transpose of a vector (or matrix). It is therefore tempting to calculate the inner product as

```
t(z)*z                #multiplication inner product
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    9   25   36
```

but this does not give the desired answer. Insead, you have to use

```
z <- c(1,3,5,6)
t(z)%*%z                 #multiplication inner product
```

```
     [,1]
[1,]   71
```

Note that R stores zTz as a matrix.

```
class(t(z)%*%z)
```

```
[1] "matrix" "array"
```

```
class(z)
```

```
[1] "numeric"
```

Finally, we have the outer product that gives us a matrix

```r
z <- c(1,3,5,6)
z%*%t(z)                #multiplication outer product
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    3    5    6
[2,]    3    9   15   18
[3,]    5   15   25   30
[4,]    6   18   30   36
```

In general, be very careful with `%*%` versus `*`. They are often both feasible, but yield different results!

You can also multiply a vector with a scalar:

```r
z*4
```

```
[1]  4 12 20 24
```

A strength of R is that operations can be conducted for each element of the vector in one statement

```r
z^2
```

```
[1]  1  9 25 36
```

```r
log(z)
```

```
[1] 0.000000 1.098612 1.609438 1.791759
```

```r
(z-mean(z))/sd(z)
```

```
[1] -1.2402159 -0.3382407  0.5637345  1.0147221
```

So far, we have seen the function `c()` that can be used to combine objects. All function calls use the same general notation: a function name is always followed by round parentheses. Sometimes, the parentheses include arguments, such as

```r
z <- seq(from = 1, to = 5, by = 1)
z
```

```
[1] 1 2 3 4 5
```

With square brackets you can access particular elements, such as

```
z[1:3]                    #refer to particular elements
```

```
[1] 1 2 3
```

We will next define a matrix and perform matrix-vector multiplication.

```
M <- matrix(data=1:9, nrow=3, ncol=3)    # define matrix M
M
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
dim(M)                                   # dimension of M
```

```
[1] 3 3
```

```
dim(z)                                   # dimension of z - does not work
 ↪  because z is not a matrix
```

```
NULL
```

```
length(z)                                # number of elements of z
```

```
[1] 5
```

Element-by-element multiplication for M is given by

```
M * M
```

```
     [,1] [,2] [,3]
[1,]    1   16   49
[2,]    4   25   64
[3,]    9   36   81
```

Standard matrix multiplication is again done using %*% such as

```
M %*% M
```

```
     [,1] [,2] [,3]
[1,]   30   66  102
[2,]   36   81  126
[3,]   42   96  150
```

```
t(M) %*% M
```

```
     [,1] [,2] [,3]
[1,]   14   32   50
[2,]   32   77  122
[3,]   50  122  194
```

If matrices and vectors have suitable dimensions, you can multiply them as follows:

```
M %*% z[1:3]
```

```
     [,1]
[1,]   30
[2,]   36
[3,]   42
```

Similar to before, with square brackets you can excess particular elements, such as

```
M[,3]                  # state the third column of M
```

```
[1] 7 8 9
```

```
M[3,3]                 # state element in the third column and third row of
 ↪  M
```

```
[1] 9
```

Here are some more examples:

```
M[,1]                  # first column of M
M[3,3]                 # element in the third column and third row of M
z[c(1,4)]              # first and fourth element of z
```

Logical statements are also very useful. For example, if you want to know which elements in the first column of matrix `M` are strictly greater than 2:

```
# See which elements:
M[,1]>2
```

```
[1] FALSE FALSE  TRUE
```

```
# and extract the elements
M[M[,1]>2,1]
```

```
[1] 3
```

Note: Logical operations return so-called *boolean* objects, i.e., either a `TRUE` or a `FALSE`. For instance, if we ask R whether `1>2` we get the answer `FALSE`.

1.4 Operators

We have alrady seen basic arithmetic operations such as

```
+, -, *, /
```

Decimals are declared with a ".", not a comma

```
1.8 not 1,8
```

Exponents are declared with ^, i.e. 2^3 is written as

```
2^3
```

For mathematical calculations, only parentheses `()` can be used.

Frequently used mathematical functions are

```
sqrt()            # square root
exp()             # exponential function
log()             # nat.  Logarithm
log(...,10)       # Logarithm with Base 10
abs()             # absolute value
round(...,x)      # rounding to x decimals.
```

```
pi
exp(1)            # Eulers number
sin(),cos(),tan() # trigonemetric functions
min(), max()      # returns the lowest/highest value of a vector/matrix
```

Asa few examples enter the following calculations and make sure to understand the commands:

```
1.8+2
1.8-2
1.8*2
1.8/2
2+2*3
(2+2)*3
2^3
8^(1/3)
3^2
9^0.5
2^2*2+2
2^(2*((0.2+0.3)*(1+2)))+4
sqrt(2)
exp(1)
exp(2)
log(7.389056)
log(exp(3))
log(100,10)
abs(1.8-2)
round(sqrt(2),2)
round(sqrt(2),4)
pi
sin(pi/2)
sin(pi/2)
sin(pi)
tan(pi)
x=2+3i
y=4+1i
x
y
x+y
x*y
```

Attention:

```
1.224606e-16
```

is interpreted as 1.224606×10^{-16}. This is often exactly zero, but for internal memory purposes (R uses double-precision floating-point numbers and therefore only has up to 16 decimals), can be a rounding mistake.

1.5 Further Data Objects

Besides classical data objects such as scalars, vectors, and matrices there are three further data objects in R:

1.The **array**: A matrix but with more dimensions. Here is an example of a 2×2×2-dimensional array

```
myFirst.Array <- array(c(1:8), dim=c(2,2,2))
```

2. The **list**: In `lists` you can organize different kinds of data. E.g., consider the following example:

```
myFirst.List <- list("Some_Numbers" = c(66, 76, 55, 12, 4, 66, 8, 99),
                     "Animals"      = c("Rabbit", "Cat", "Elefant"),
                     "My_Series"    = c(30:1))
```

3. The **data frame**: A `data.frame` is a `list`-object but with some more formal restrictions (e.g., equal number of rows for all columns). As indicated by its name, a `data.frame`-object is designed to store data:

```
myFirst.Dataframe <- data.frame("Credit_Default"   = c( 0, 0, 1, 0, 1, 1),
                                "Age"              = c(35,41,55,36,44,26),
                                "Loan_in_1000_EUR" = c(55,65,23,12,98,76))
```

1.6 R Help

There exist help files for all commands in R. For example consider the help file of the function `sum()`. You can access the help file with

```
?sum
```

The file usually contains the categories: Description, Usage, Arguments, Details, Value and References.

After the general description follows the syntax of the command. This usually indicates which (optional) arguments the command allows. Then follows `na.rm`, which indicates whether missing values (na) will be removed (rm). The default is `FALSE`.

```
x<-c(1,2,NA,4,5)
x
```

```
[1]  1  2 NA  4  5
```

```
sum(x)
```

```
[1] NA
```

```
sum(x,na.rm=T)
```

```
[1] 12
```

Details describes the exact calculation (trivial in this case), and some command specific characteristics. Value describes what is returned, in the case of `sum()` it is obviously a sum of numbers, given that numeric data was used as an input.

1.7 R Packages

There are tons of R packages that are useful for different aspects of data analysis. Some packages that can help you make beautiful plots and that you might want check out are the package `ggplot2` and the collection of packages `tidyverse`. You can install the packages with `install.packages(packagename)`. In order to use the package type `library(package)`. The package has to be installed only once, but you have to load the package with `library(package)` every time you want to use it.

1.8 Exercises I

Exercise 1:

Try to shorten the notation of the following vectors as much as possible, using `:` notation:

a. `x <- c(157, 158, 159, 160, 161, 162, 163, 164)`
b. `x <- c(10, 9, 8, 7, 6, 5, 4, 3, 2, 1)`

c. `x <- c(-1071, -1072, -1073, -1074, -1075, -1074, -1073, -1072, -1071)`

Exercise 2:

The ´:´ operator can be used in more complex operations along with arithmetic operators, and variable names. Have a look at the following expressions, and write down what sequence you think they will generate. Then check with R.

a. `(10:20) * 2`
b. `105:(30 * 3)`
c. `10:20*2`
d. `1 + 1:10/10`
e. `2^(0:5)`

Exercise 3:

R has several functions for rounding. Let's start with floor, ceiling, and trunc:

`floor(x)` rounds to the largest integer not greater than `x` `ceiling(x)` rounds to the smallest integer not less than `x` `trunc(x)` returns the integer part of `x`

Below you will find a series of arguments (x), and results (y), that can be obtained by choosing one or more of the 3 functions above (e.g.`y <- floor(x)`). Which of the above 3 functions could have been used in each case? First, choose your answer without using R, then check with R.

a. `x <- c(300.99, 1.6, 583, 42.10)`

   `y <- c(300, 1, 583, 42)`

b. `x <- c(152.34, 1940.63, 1.0001, -2.4, sqrt(26))`

   `y <- c(152, 1940, 1, -2, 5)`

c. `x <- -c(3.2, 444.35, 1/9, 100)`

   `y <- c(-3, -444, 0, -100)`

d. `x <- c(35.6, 670, -5.4, 3^3)`

   `y <- c(36, 670, -5, 27)`

Exercise 4:

Consider the following vectors:

`u <- c(5, 6, 7)`

`v <- c(2, 3, 4)`

Perform the following operations:

- Add `u` and `v`
- subtract `v` from `u`
- multiply `u` by `v` element by element

- divide `u` by `v` element by element
- calculate the inner product of `u` and `v`
- calculate the outer product of `u` and `v`
- raise `u` to the power of `v` element by element

Then check your answer with R.

2 Create and Load Data Sets

We will use R to analyze both simulate and real data. Simulated data will be crucial to study small sample properties of estimators.

2.1 Simulated Data

We start by considering data that is simulated by the user. We can create data sets by using functions like `rnorm()`, `rexp()` or `runif()`. After the data is created we can work with it as shown in the following example:

```
n = 100  # number of observations

Z = rnorm(n, mean = 5, sd = 3)      # draw from normal distribution
mean(Z)
```

```
[1] 5.195167
```

```
sd(Z)
```

```
[1] 3.172004
```

```
E = rexp(n, rate = 3)           # draw data from exponential distribution
mean(E)
```

```
[1] 0.2736041
```

```
sd(E)
```

```
[1] 0.3025521
```

```
U = runif(n, min = 3, max = 5)  # draw data from uniform distribution
mean(U)
```

```
[1] 4.057058
```

```
sd(U)
```

```
[1] 0.5858102
```

Analogous functions exist for many other distributions in R, and you can find them by searching on the web.

2.2 Load Datasets

It is also possible to work with observed data, which will be loaded into R. Here, we will use some real data, namely a movie database from IMDB that you can download here. The data set contains, among others, the budget, the revenue, and the average rating of movies.

Once the data is saved in your working directory, you can load it using

```
data_set = read.csv("tmdb_5000_movies.csv")
```

which stores the data in a data frame. You can use the `View()` function to look the data.

After the file is loaded, we can work with the data using `$` to access certain parts of the data set such as

```
data_set$original_title[1:5]
```

```
[1] "Avatar"
[2] "Pirates of the Caribbean: At World's End"
[3] "Spectre"
[4] "The Dark Knight Rises"
[5] "John Carter"
```

```
data_set$budget[1:5]
```

```
[1] 237000000 300000000 245000000 250000000 260000000
```

```
mean(data_set$budget[1:5])
```

```
[1] 258400000
```

We will return to this data set when we talk about regression analysis later.

Apart from `read.csv` there exist also `read.dta`, `read.xls` and many more to load suitable data sets.

Another important function is `readRDS` that is used to read in a data set that was saved using the function `saveRDS`. These functions can be useful in case properties of your R object are not preserved when saved in other formats, such as a csv file.

2.3 Save Datasets

There are also multiple ways to save your own data. While `read.csv` can be used to read in data from a csv file, its counterpart `write.csv` can be used to save a suitable object as a csv file. The most important arguments for this function are `x`, the object you want to save, and `file`, which provides the path where to save the file.

```
write.csv(data_set, <my_path>)
```

Similarly the function `saveRDS` allows you to save a generic R object to storage. One advantage of this function compared to `write.csv` is that it preserves properties of your object that might be lost when saved as a csv file. For example, conversions from factors to strings might occur when saving a `data.frame` as a csv file.

The package `readxl` and `openxlsx` from the tidyverse family can be used to read in and write to excel files, respectively. This format is usually not be the best option to store your data.

2.4 Exercises II

Exercise 1:

Simulate a vector containing 10000 independent normally distributed random variables with means of 1 and variances of 2. How many of them are negative? What is the expected fraction of negative values?

Exercise 2:

Simulate `u` and `v` as independent log-normally distributed random vectors of size 200 each. Pick means and variances you like. Then let `w` be equal to `u+v`. Now calculate the following:

- the sample means of `u`, `v`, and `w`
- the sample variances of `u`, `v`, and `w`
- all pairwise covariances

Exercise 3:

Take the objects you created in the previous exercises from this chapter and save them to storage in your preferred format. Then close your session, clean your workspace and read them in again. This might be tedious at first, but integrating the export of data into your workflow might save you from time costly mistakes in the future.

3 Simple Programming

We often want to repeat specific operations several times. For example, we might want to repeat an operation for different elements of a given data structure, such as columns of a matrix, or perform an operation until a satisfactory result is reached, such as running an algorithm until the value converges to a prespecified level. In addition, we often need to check whether certain conditions are fulfilled, such as whether elements of a vector are equal to 0, and perform an operation depending on the result.

3.1 The For-Loop

A for-loop is a procedure in which a particular routine is repeated and only an index variable is changed.

```
x=0
for (i in 1:3) {
  x=x+i
  print(x)
}
```

```
[1] 1
[1] 3
[1] 6
```

We can also use it to study the behavior of the arithmetic mean

```
out = rep(0,100)
n = 20
for (i in 1:100) {
  x = rnorm(n, mean = 0,sd = 1)
  out[i]=mean(x)
}
mean(out)
```

```
[1] 0.001503778
```

```
sd(out)
```

```
[1] 0.2526717
```

```
out = rep(0,100)
n = 200
for (i in 1:100) {
  x = rnorm(n,mean = 0,sd = 1)
  out[i]=mean(x)
}
mean(out)
```

```
[1] 0.0125802
```

```
sd(out)
```

```
[1] 0.07676445
```

3.2 The If-Condition

Within a for-loop, it may be useful to perform arithmetic operations only under certain conditions. The basic syntax for this is

```
if (condition){
  statement
  }
else{
  alternative
  }
```

where `else` can remain unspecified.

```
###If-condition###
y <- c(1,4,5,2,7,8,2,4)
N <- length(y)
1:N
##
y.sq <- numeric(N)#creates an empty numeric vector of length N

for(i in 1:N){ #initializing the for-loop
```

```
  y.sq[i] <- y[i]^2#every element of y.sq is replaced by the respective
 ↪  squared value.
  if(i == N){#only when this condition is met, i.e. in the last iteration
   ↪  of the loop
    print(y.sq)#is the vector printed
  }
}
```

In shorter form the `ifelse()` command is an alternative.

```
ifelse(test, yes, no)
```

3.3 The While-Loop

The while-loop performs a program ("statement") as long as certain data conditions ("test expression") are met. The basic syntax is

```
while (testexpression)
{
  #increment (i+1);
  statement
}
```

Here, `testexpression` is evaluated and the body of the loop is entered if the result is `TRUE`.

```
i <- 1

while (i < 6) {
  print(i)
  i = i+1
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

3.4 Time Saving and Loops

One easy way to make your loops in R faster is to preallocate space for your results instead of appending them to a vector. This is caused by the underlying architectural choice of R that is called copy-on-modify. In some cases it leads to your whole data being copied from one place in memory to another, when you simply wanted to append another value to your vector.

If you look at the following code fragments, the second one is about 70 to 80 times faster!

```
# The slow but simple way
time <- system.time(
  {results <- c() # empty vector
  for(i in 1:100000){
    results <- c(results, rnorm(1))
  }}
)
print(time)
```

```
   user  system elapsed
 12.388   0.176  12.567
```

```
# The right way to do it
time2 <- system.time(
  {results2 <- vector(mode = "double", length = 100000)
  for(i in 1:100000){
    results2[i] <- rnorm(1)
  }}
)
print(time2)
```

```
   user  system elapsed
  0.082   0.000   0.082
```

If you can avoid a loop altogether and use vector operations instead, your code will will be even faster.

```
# The right way to do it
time3 <- system.time(
  {results3 <- rnorm(100000)
  }
)
print(time3)
```

```
   user  system elapsed
  0.004   0.000   0.003
```

Try to keep this in mind when designing your own code. In R this is one of the most frequent and easily avoidable reasons for why code is slow.

3.5 Exercises III

Exercise 1:

Write a for-loop that iterates over the numbers 6 to 14 and prints the cube of each number using `print()`

Exercise 2:

Write a while-loop that prints out standard random normal numbers (use `rnorm()`) but stops (breaks) if you get a number bigger than 1.

Exercise 3:

Using a for-loop simulate the flip a coin twenty times, keeping track of the individual outcomes (1 = heads, 0 = tails) in a vector that you preallocated.

4 Functions

If you want to make your code more readable and easily understandable, you should split up complicated tasks into more simple ones. A convenient way to do this is defining a function that generalizes a step of your code. If you define this function in advance, you can call it later to perform the task, which might make the code more easily readable and enables you to use paradigmas such as recursion or metaprogramming.

4.1 Function Definition

A function definition follows a specific structure.

```
my_function <- function(argument1, argument2){
  # here I describe what I want to do with my arguments
  # For example multiply them, assuming that they are numerical.

  my_return_value = argument1 * argument2

  # and using return() we tell our function what to return
  return(my_return_value)
}

my_function(argument1 = 5, argument2 = 7)
```

```
[1] 35
```

The function can also be called without explicitly naming the arguments, but if you don't know your code very well, this might lead to difficulties when somebody else or your future self might try to understand the code.

```
my_function(2,5)
```

```
[1] 10
```

Another useful feature is the definition of default values for your arguments such that they can be omitted when calling a function.

```r
my_2nd_function <- function(argument1, argument2, greetings = FALSE){
  my_return_value <- argument1 * argument2
  if(greetings == TRUE){
    print("Hello user, the cake is a lie.")
  }
  return(my_return_value)
}

my_2nd_function(2, 7)
```

```
[1] 14
```

```r
my_2nd_function(3, 12, TRUE)
```

```
[1] "Hello user, the cake is a lie."
```

```
[1] 36
```

4.2 Recursion

In some settings using a recursive approach instead of loops can create more readable code. In simple terms, recursion in programming is calling a function in its own body. A simple example - even though very inefficient from a computational standpoint - is calculating factorials.

```r
factorial <- function(n){
  if(n == 1) return(1)
  else return(n*factorial(n-1))
}

factorial(17)
```

```
[1] 3.556874e+14
```

Often recursion relies on a principle called "divide and conquer". We test for a simple base case, like n being equal to one in the case of factorials, and otherwise reducing the problem to a more simple one.

Another example is the calculation of Fibonacci numbers using recursion.

```r
fibonacci <- function(n){
  if(n == 1 || n == 2) return(1)
  else return(fibonacci(n-1) + fibonacci(n-2))
}

fibonacci(20)
```

```
[1] 6765
```

4.3 Exercises IV

Exercise 1:

Write a function that takes 2 integers as inputs and outputs whether or not the first integer is divisible by the second as a boolean.

Exercise 2:

Try to solve the problem of adding all squares from 1 to n first using loops and then recursion. Think about the underlying way in which the function works. Which approach do you prefer? Why?

Exercise 3:

Write a function that receives a single vector of real numbers as its argument and returns an ordered vector with the same entries.

If you are interested to see recursion in action look for mergesort, that solves this problem with a cool recursive approach.

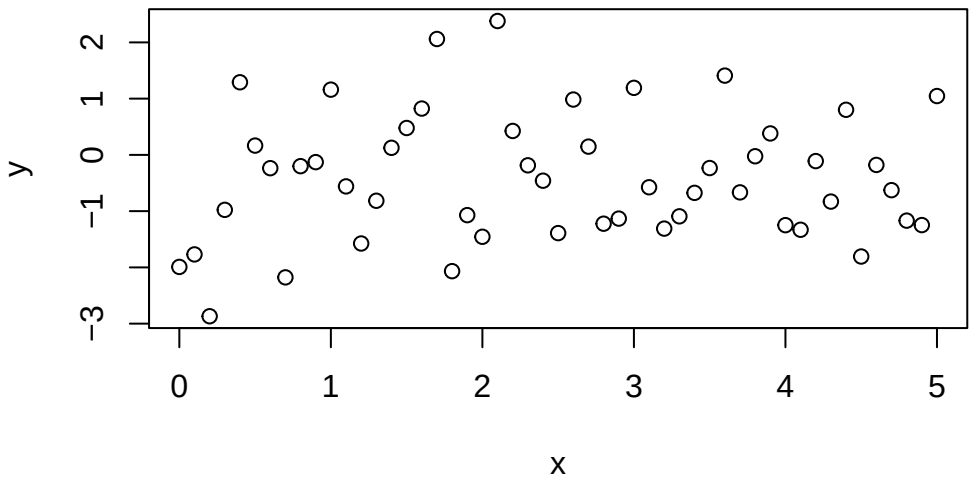
5 Plotting in R

It is often useful to visualize data by using plots. R offers many different possibilities to create plots, ranging from simple scatterplots to animated plots, that you can export as gifs or even interactive webapps. The built-in capabilities of R offer many tools, to quickly generate plots and packages like ggplot2 from the tidyverse family offer a full plotting experience built on the so called "Grammar of Graphics".

5.1 The Plot-Function

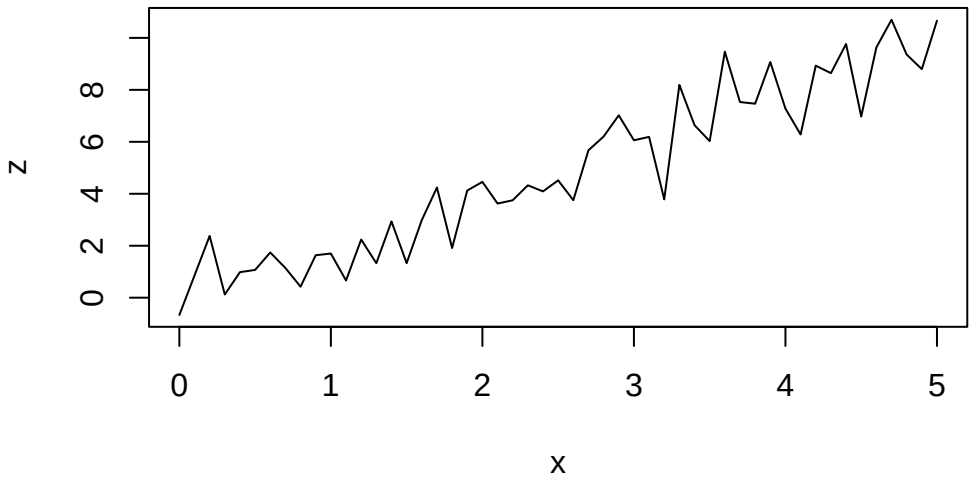
In its simplest form, the `plot` function in R delivers a scatterplot. In the call below, `type = "p"` could be omitted as it is the default value if the argument is not provided in the call.

```
x <- seq(0, 5, 0.1)
y <- rnorm(n = length(x))
plot(x = x, y = y, type = "p")
```

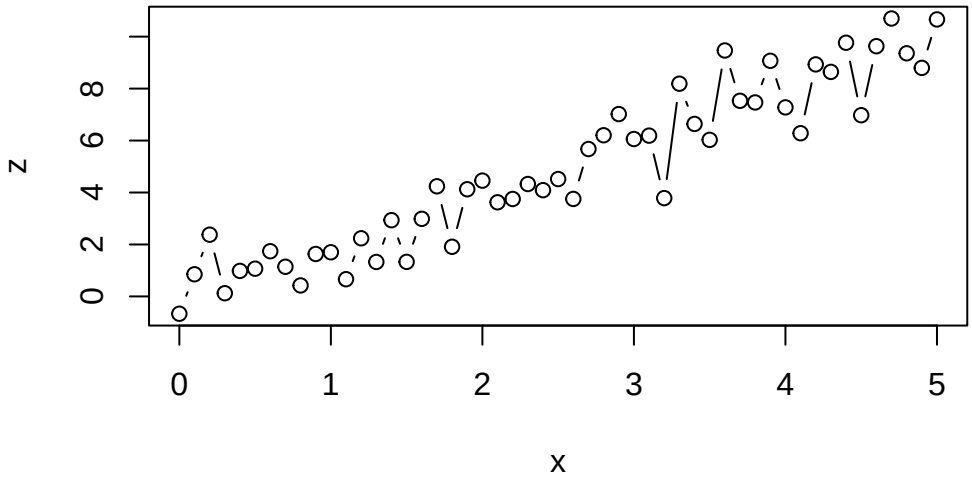


If we instead want a simple line plot or a combination of both, we can specify this behavior using the `type` argument.

```
z <- 2*x + rnorm(n = 51)
plot(x = x, y = z, type = "l")
```



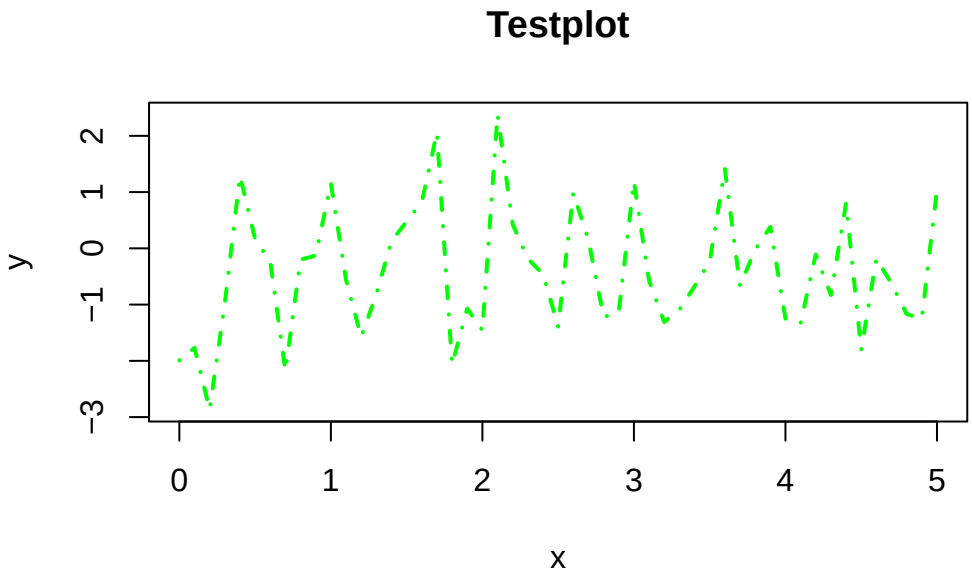

```
plot(x = x, y = z, type = "b")
```



Other arguments to the `plot` function can be used to specify color, linetype (dashed etc.), linewidth, title, subtitle and many more. The help files `?plot` and `?par` can be very helpful when trying to achieve a specific look for your plot.

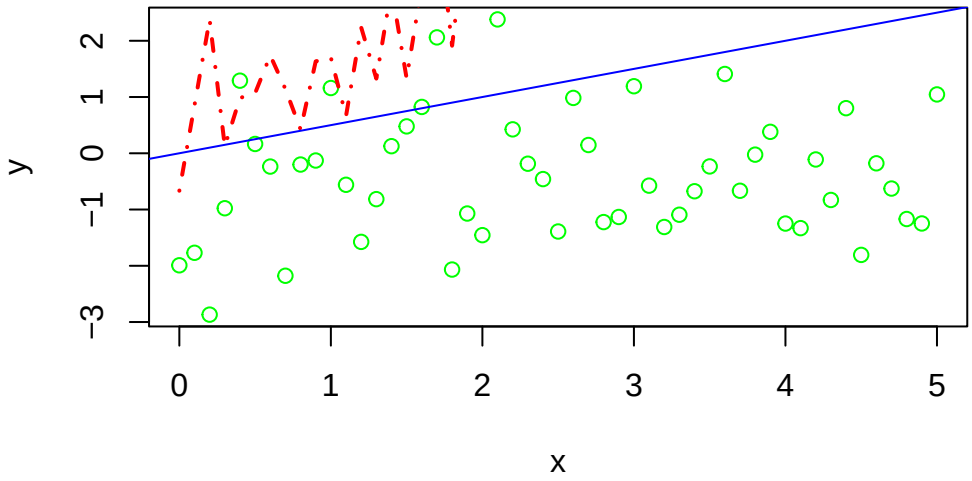
An example could look a little like this:

```
plot(x = x, y = y, type = "l", lty = 4, lwd = 2, col = "green", main =
 ↪  "Testplot")
```



If we want to show multiple things in one diagram it can be useful to plot multiple objects in one coordinate grid. R provides some ways to do so, the most simple ones being the `lines` function that works similar to the `plot` function and the `abline` function which can be used to add straight lines.

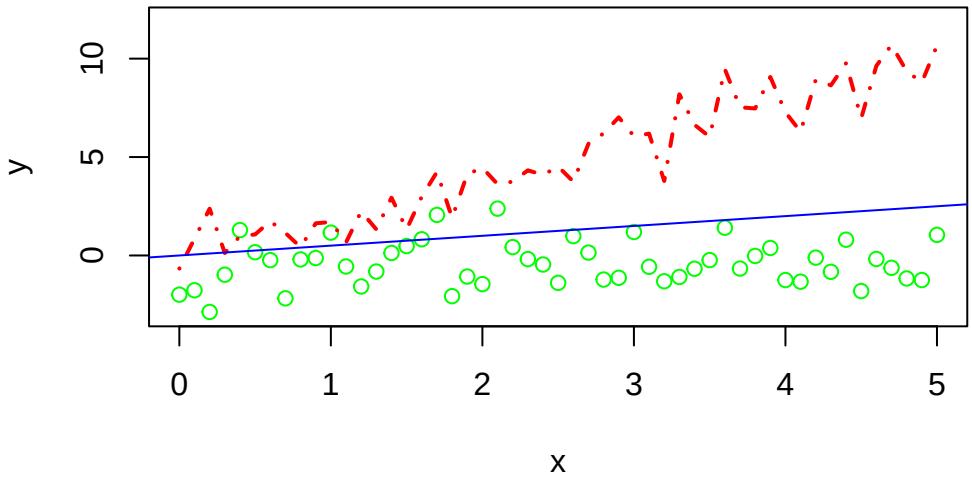
```
plot(x = x, y = y, type = "p", col = "green")
lines(x = x, y = z, type = "l", lwd = 2, lty = 4, col = "red")
abline(a=0, b=0.5, col = "blue")
```



As you may see from this plot, our y-axis is not adjusted to our newly added line. But we have a possibility to manually adjust the range of our axes if we choose to do so. This is done using the `xlim` and `ylim` arguments in the original `plot` function.

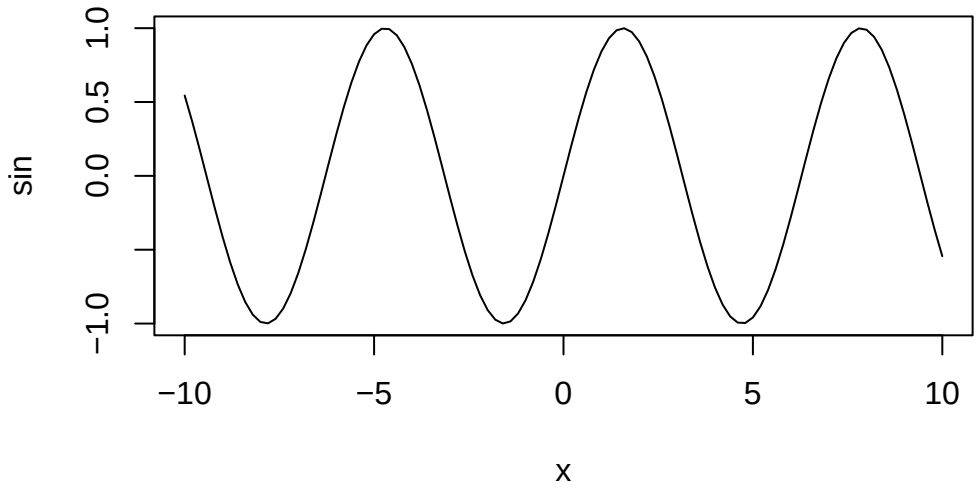
```
plot(x = x, y = y, type = "p", col = "green", xlim = c(0,5), ylim = c(-3,
 ↪  12))
```

```
lines(x = x, y = z, type = "l", lwd = 2, lty = 4, col = "red")
abline(a=0, b=0.5, col = "blue")
```

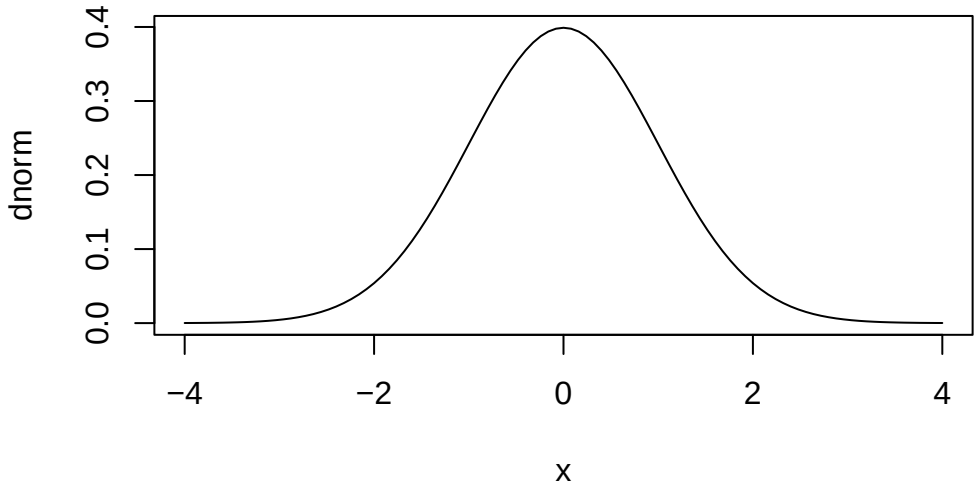


It is also possible to plot functions like $\sin(x)$ or the standard normal density.

```
plot(sin, xlim = c(-10, 10))
```




```
plot(dnorm, xlim = c(-4, 4))
```



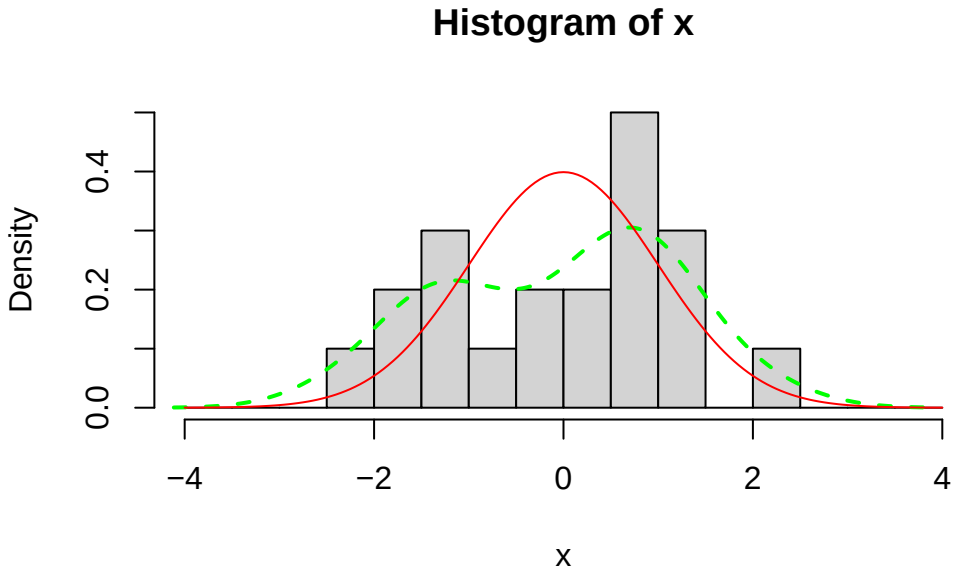
There are many other useful plotting functions that are integrated in base R. We won't cover all of them here, but instead focus on some of the most frequently needed.

5.2 Histograms and Kernel Densities

Other useful plotting functions that are offered in the default R experience include histograms with the function `hist`. With a combination of `plot` and `density` we get access to kernel density plots. A kernel density estimator is a nonparametric estimator of density of random variable based on a sample. Details about the estimator are beyond the scope of these notes. In the settings where we will use them, the default choices of the estimator will yield accurate results. Combining our previous knowledge with these possibilities, we can create plots like the following (note: in `hist` the argument `freq = FALSE` leads to relative frequency instead of counts. See `?hist` for more details.):

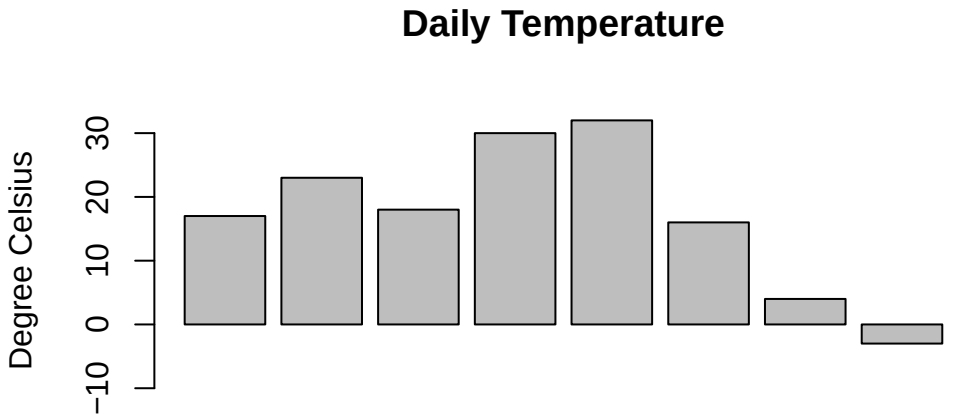
```
x <- rnorm(n = 20)
my_dens <- density(x)

hist(x, freq = FALSE, breaks = seq(-4, 4, 0.5), xlim = c(-4, 4))
lines(my_dens, col = "green", lty = 2, lwd = 2)
lines(x = seq(-4, 4, 0.05), y = dnorm(seq(-4, 4, 0.05)), col = "red", lty =
 ↪  1)
```



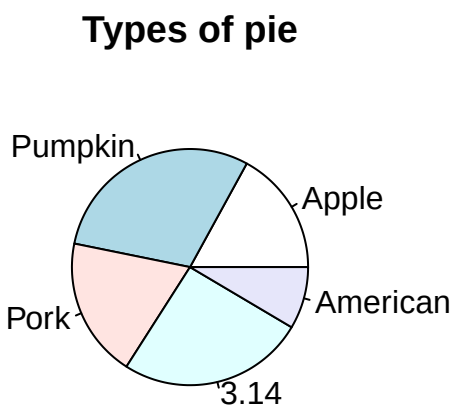
Further functions include barplots:

```
temperatures <- c(17,23,18,30,32,16,4,-3)
barplot(temperatures,
        ylim = c(-15, 35),
        main = "Daily Temperature",
        ylab = "Degree Celsius")
```



and piecharts:

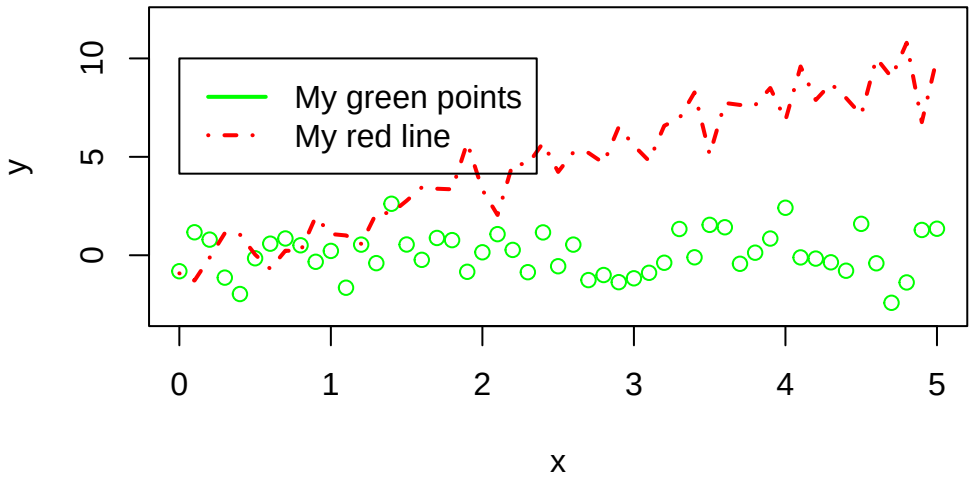
```
slices <- c(8, 14, 9, 12, 4)
labels <- c("Apple", "Pumpkin", "Pork", "3.14", "American")
pie(slices, labels = labels, main = "Types of pie")
```



Another important part of plots - like comments in code - is a legible legend. R has built-in support for adding a legend to your plots. Going back to our combined plot from above, one can add a legend as shown in the following example. Here the `x` and `y` arguments specify the position of the legend in the plot and the other arguments specify its contents. Since a legend has to be customized for each plot, one should have a look in the documentation `?legend`.

```
x <- seq(0, 5, 0.1)
y <- rnorm(n = length(x))
z <- 2*x + rnorm(n = 51)

plot(x = x, y = y, type = "p", col = "green", xlim = c(0,5), ylim = c(-3,
 ↪  12))
lines(x = x, y = z, type = "l", lwd = 2, lty = 4, col = "red")
legend(x = 0, y = 10,
       legend = c("My green points", "My red line"),
       col = c("green", "red"),
       lty = c(1, 4),
       lwd = 2)
```



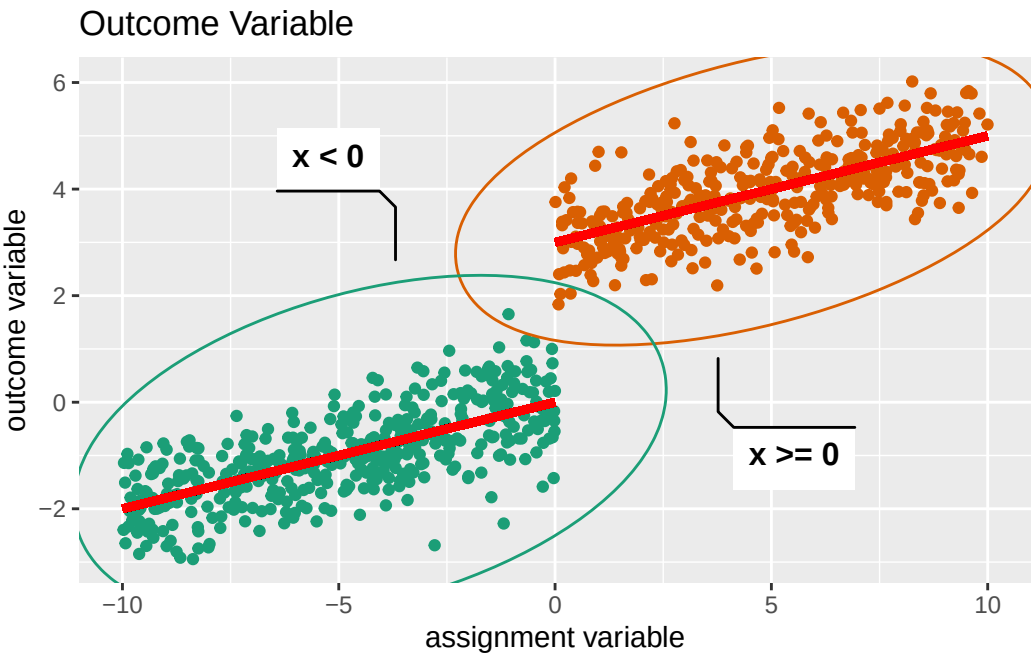
If one decides to pursue a tidyverse approach, the package ggplot allows for beautiful, heavily customized plots. An example can be seen below:

```
library(ggplot2)
library(ggforce)
library(hrbrthemes)

n = 800
x <- runif(n, min = -10, max = 10)
treatment <- ifelse(x>0,1,0)
y <- 0.2*x + treatment*3 + rnorm(n, mean = 0, sd = 0.6)
visualise.data <- data.frame(x, treatment, y)

p1 <- ggplot(visualise.data, aes(x=x, y=y, colour = treatment == 1)) +
        labs(title = "Outcome Variable", y = "outcome variable", x =
         ↪  "assignment variable") +
        geom_point() +
        geom_mark_ellipse(aes(filter = treatment == 1,
                              label = 'x \U02265 0')) +
        geom_mark_ellipse(aes(filter = treatment == 0,
                              label = 'x < 0')) +
        geom_segment(aes(x = -10, y = -2, xend = 0, yend = 0), color =
         ↪  'red', size = 1.5) +
        geom_segment(aes(x = 0, y = 3, xend = 10, yend = 5), color = 'red',
         ↪  size = 1.5) +
        #theme_ipsum(axis_title_size = 18) +
        theme(legend.position = "none",
              legend.title = element_text(size = 18)) +
        scale_colour_brewer(type = 'qual', palette = 'Dark2')

p1
```



The possibilities of ggplot2 are vast and can be intimidating, but it offers a consistent plotting experience integrated into a tidy data workflow using the tidyverse family of packages.

5.3 Exercises V

Exercise 1:

Draw samples of size 1000 from exponential distributions with varying values of the parameter. Use histograms to visualize differences in the distributions.

Exercise 2:

Take the `mtcars` dataset that is included in base R and plot `mpg` (miles per gallon) against `wt` (weight) as a scatterplot. Add a red line with `abline` by eyeballing the parameters that could come out of a regression.

Exercise 3:

Write a function that takes as its arguments a sample size n and a boolean that indicates, if plotting should take place. Let this function return a sample of size n drawn from a standard normal. If the plotting argument was true, let it also plot your sample in a way that seems sensible to you.

6 Monte Carlo Simulations

6.1 Some Important Concepts

6.1.1 Random Sample

Let

$$\{X_1, X_2, \dots, X_n\}$$

denote an **(i.i.d.) random sample**, i.e., a collection of n random variables that all have the same distribution F_X,

$$X_i \sim F_X \quad \text{for all} \quad i = 1, \dots, n,$$

and that are independent of each other,

$$X_i \perp\!\!\!\perp X_j \quad \text{for all} \quad i \neq j.$$

- The symbol $\perp\!\!\!\perp$ means **independent of**.
- The abbreviation **i.i.d.** stands for **independent and identically distributed**.

Examples: F_X and Parameters of Interest θ

1. **Normal distribution:** $F_X = \mathcal{N}(\mu_X, \sigma_X^2)$
   - $\theta = E(X) = \mu_X$, or
   - $\theta = \text{Var}(X) = \sigma_X^2$
2. **Chi-squared distribution:** $F_X = \chi^2_{\text{df}}$
   - $\theta = E(X) = \text{df}$, or
   - $\theta = \text{Var}(X) = 2 \cdot \text{df}$
3. **A distribution without a name:** F_X is some unnamed distribution (most distributions have no name)
   - $\theta = E(X)$, or
   - $\theta = \text{Median}(X)$, or
   - $\theta = p\text{-Quantile}(X)$
4. **A multivariate distribution:** F_{YX} is the joint distribution of $(Y, X^\top) \in \mathbb{R}^{K+1}$
   - $\theta = \beta$, the coefficient vector of the regression model $Y_i = X_i^\top \beta + \varepsilon_i$

6.1.2 Estimator

To estimate the unknown parameter θ, we use an **estimator**, which is simply a function of the random sample,

$$\hat{\theta}_n \equiv \hat{\theta}(X_1, X_2, \dots, X_n).$$

Since estimators are functions of random variables, they are random variables themselves.

Estimators are Random Variables

Thus, to understand an estimator, we need to know its **statistical properties**, i.e., its distribution or, at least, its mean and variance.

Observed data are just one **obs**erved realization of the random sample,

$$\{X_{1,\text{obs}}, X_{2,\text{obs}}, \dots, X_{n,\text{obs}}\}.$$

Each observed realization of the random sample yields a realization of the estimator,

$$\hat{\theta}_{n,\text{obs}} \equiv \hat{\theta}(X_{1,\text{obs}}, X_{2,\text{obs}}, \dots, X_{n,\text{obs}}).$$

This can be confusing: in practice, we typically have only one data set and, therefore, only one realization of the estimator. Still, we should keep in mind all the other data sets we could have obtained.

The following examples illustrate two random samples and their estimators:

Example: Estimating the Mean from an i.i.d. Random Sample

Let

$$\{X_1, \dots, X_n\}$$

be an i.i.d. random sample with $X_i \sim F_X$ for all $i = 1, \dots, n$.

Psst … Secret Information

In this simulation, $F_X = \mathcal{N}(\mu_X, \sigma_X^2)$ with $\mu_X = 5$ and $\sigma_X^2 = 10$. In practice, F_X, and thus μ_X and σ_X^2, are **unknown**.

We estimate the unknown mean $\theta \equiv \mu_X$ by the **sample mean**

$$\hat{\theta}_n \equiv \hat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} X_i.$$

The following code generates one realization of the random sample. The fixed seed, `set.seed(321)`, makes the pseudo-random numbers reproducible.

```r
set.seed(321)

mean_X <- 5
var_X  <- 10
n      <- 10

sample_X <- rnorm(n = n, mean = mean_X, sd = sqrt(var_X))
```

The observed realization, stored in `sample_X`,

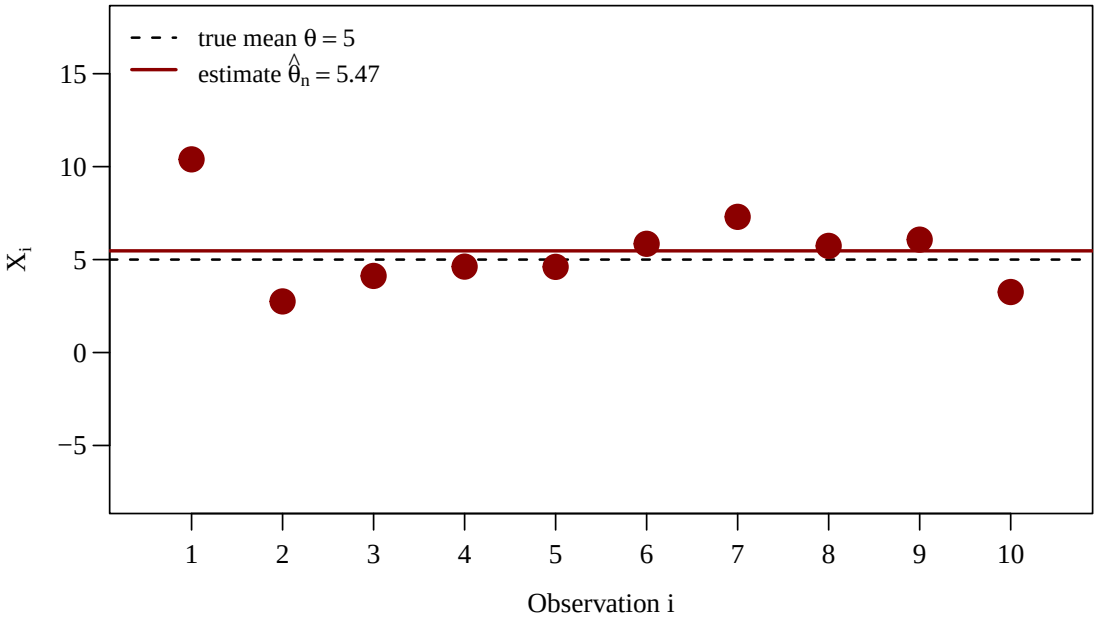
$$\{X_{1,\text{obs}}, \dots, X_{n,\text{obs}}\} = \{10.39,\ 2.75,\ \dots,\ 3.25\}$$

yields the estimate

$$\hat{\theta}_{n,\text{obs}} = \frac{1}{n}\sum_{i=1}^{n} X_{i,\text{obs}} = 5.47.$$

A new sample yields a new estimate. The animation shows the observed sample (first frame, red) followed by further realizations of the random sample and of the estimator $\hat{\theta}_n$ (blue).

Observed realization of the random sample



Summary:

- The data change from realization to realization, and so does the estimate. That is, $\hat{\theta}_n$ is a **random variable**, and $\hat{\theta}_{n,\text{obs}}$ is just one of its realizations.
- The realizations of $\hat{\theta}_n$ lie close to the (usually unknown) true mean $\mu_X = 5$. That is, the distribution of $\hat{\theta}_n$ is **concentrated** around μ_X. This concentration increases with the sample size n.

Example: Estimating Regression Coefficients from an i.i.d. Random Sample

Let

$$\{(Y_1, X_1^\top), \dots, (Y_n, X_n^\top)\}$$

be an i.i.d. random sample with $(Y_i, X_i^\top) \sim F_{YX}$ for all $i = 1, \dots, n$, where

- $Y_i = X_i^\top \beta + \varepsilon_i$ is the outcome, and
- $X_i \in \mathbb{R}^K$ is the vector of predictors of observation i.

Psst … Secret Information

In this simulation, $X_i = (1, X_{i2})^\top$ with $X_{i2} \sim \mathcal{U}[0, 4]$, $\varepsilon_i \sim \mathcal{N}(0, 1)$ independent of X_i, and $\beta = (2, -1)^\top$. Together, these choices fully specify the joint distribution F_{YX}.
In practice, we assume (hopefully correctly) that we know the type of regression model, here the simple linear regression model

$$Y_i = X_i^\top \beta + \varepsilon_i \quad \text{with} \quad X_i = (1, X_{i2})^\top.$$

However, we know only the model's structure (here: an intercept and one predictor), not the entire model. In particular, we typically do **not know**:

- the distribution of X_i,
- the distribution of ε_i, and
- the value of β.

Thus, essential parts of F_{YX}, such as β, are **unknown**.

We estimate the unknown coefficient vector $\theta \equiv \beta$ by the **OLS estimator**

$$\hat{\theta}_n \equiv \hat{\beta}_n = (X^\top X)^{-1} X^\top Y,$$

where

$$X = \begin{pmatrix} X_1^\top \\ X_2^\top \\ \vdots \\ X_n^\top \end{pmatrix} = \begin{pmatrix} X_{11} & X_{12} & \cdots & X_{1K} \\ X_{21} & X_{22} & \cdots & X_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ X_{n1} & X_{n2} & \cdots & X_{nK} \end{pmatrix} \quad \text{and} \quad Y = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}$$

are the $n \times K$ matrix of predictors and the $n \times 1$ vector of outcomes.
The following code generates one realization of the random sample and computes the estimate.

```r
set.seed(321)

n    <- 20
beta <- c(2, -1)

X   <- cbind(1, runif(n, min = 0, max = 4))   # first column: intercept
eps <- rnorm(n, mean = 0, sd = 1)
Y   <- X %*% beta + eps

beta_hat <- solve(t(X) %*% X) %*% t(X) %*% Y
```

The observed realization, stored in `Y` and `X`,

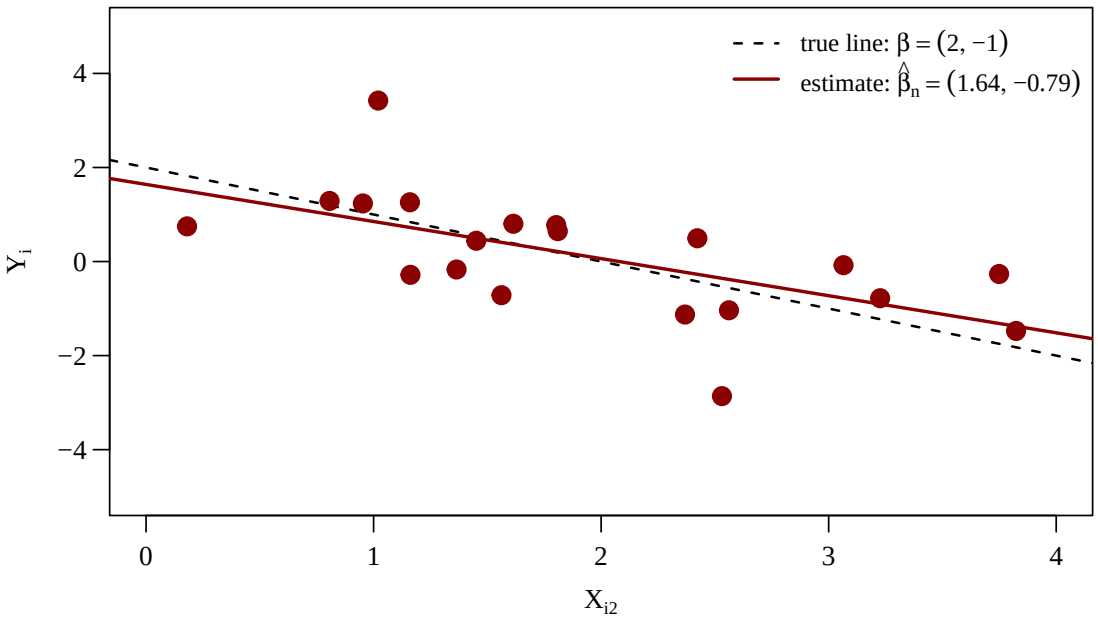
$$\{(Y_{1,\text{obs}}, X_{1,\text{obs}}^\top), \dots, (Y_{n,\text{obs}}, X_{n,\text{obs}}^\top)\} = \{(-1.48,\ 1,\ 3.82),\ \dots,\ (-1.04,\ 1,\ 2.56)\}$$

yields the estimate, stored in `beta_hat`,

$$\hat{\theta}_{n,\text{obs}} = (1.64,\ -0.79)^\top.$$

A new sample yields a new estimate. The animation shows the observed sample (first frame, red) followed by further realizations of the random sample and of the estimator $\hat{\theta}_n$ (blue).

Observed realization of the random sample



Summary:

- The data change from realization to realization, and so does the estimate. That is, $\hat{\theta}_n$ is a **random vector**, and $\hat{\theta}_{n,\text{obs}}$ is just one of its realizations.

- The estimated regression lines lie close to the (usually unknown) true line with $\beta = (2, -1)^\top$. That is, the distribution of $\hat{\theta}_n$ is **concentrated** around β. This concentration increases with the sample size n.

6.1.3 Bias, Variance, and Consistency of Estimators

Since an estimator is a random variable, it has a distribution and, typically, a mean and a variance. We use them to assess the quality of an estimator.

Definition 6.1 (Bias, Variance, Standard Error, and Mean Squared Error of $\hat{\theta}_n$)**.**

- The **bias** of $\hat{\theta}_n$ is
$$\text{Bias}\left(\hat{\theta}_n\right) = E\left(\hat{\theta}_n\right) - \theta.$$
- The **variance** and the **standard error** of $\hat{\theta}_n$ are
$$\text{Var}\left(\hat{\theta}_n\right) = E\left(\left(\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right)^2\right) \quad \text{and} \quad \text{SE}\left(\hat{\theta}_n\right) = \sqrt{\text{Var}\left(\hat{\theta}_n\right)}.$$
- The **mean squared error** of $\hat{\theta}_n$ combines bias and variance,
$$\text{MSE}\left(\hat{\theta}_n\right) = E\left(\left(\hat{\theta}_n - \theta\right)^2\right) = \left(\text{Bias}\left(\hat{\theta}_n\right)\right)^2 + \text{Var}\left(\hat{\theta}_n\right).$$

Bias. An estimator is called **unbiased** if
$$\text{Bias}\left(\hat{\theta}_n\right) = 0 \quad \text{for all} \quad n.$$
That is, averaged over *all* hypothetical data sets, the estimator equals θ. Unbiasedness holds for every fixed n; it is a **small-sample property**. Many estimators, however, are biased for fixed n. Then, we want them to be at least **asymptotically unbiased**,
$$\text{Bias}\left(\hat{\theta}_n\right) \to 0 \quad \text{as} \quad n \to \infty.$$

Variance. (Asymptotic) unbiasedness alone does not guarantee that the realizations of $\hat{\theta}_n$ are close to θ: if $\text{Var}(\hat{\theta}_n)$ is large, the realizations scatter widely around their mean, even in large samples. Therefore, we also want estimators to be **consistent**.

Definition 6.2 (Consistency of $\hat{\theta}_n$)**.** An estimator $\hat{\theta}_n$ is called **(weakly) consistent** if, for any (arbitrarily small) $\varepsilon > 0$,
$$P\left(\left|\hat{\theta}_n - \theta\right| > \varepsilon\right) \to 0 \quad \text{as} \quad n \to \infty.$$
In short, we write
$$\hat{\theta}_n \to_P \theta \quad \text{as} \quad n \to \infty.$$

That is, the realizations of a consistent estimator concentrate around θ as the sample size grows.

A convenient way to check consistency is to verify that both the bias and the variance vanish,

$$\text{Bias}\left(\hat{\theta}_n\right) \to 0 \quad \text{and} \quad \text{Var}\left(\hat{\theta}_n\right) \to 0 \quad \text{as} \quad n \to \infty,$$

or, equivalently, that $\text{MSE}(\hat{\theta}_n) \to 0$. This condition is sufficient (but not necessary) for consistency; see the box below for a proof.

Nerd's Box: A Sufficient Condition for Consistency

An estimator $\hat{\theta}_n$ with

$$\text{Bias}\left(\hat{\theta}_n\right) \to 0 \quad \text{and} \quad \text{Var}\left(\hat{\theta}_n\right) \to 0 \quad \text{as} \quad n \to \infty$$

is (weakly) consistent.

Proof. We use **Chebyshev's inequality**: for any random variable Z with finite variance and any $c > 0$,

$$P\left(|Z - E(Z)| > c\right) \leq \frac{\text{Var}(Z)}{c^2}.$$

Let $\varepsilon > 0$. Adding a zero, $0 = -E(\hat{\theta}_n) + E(\hat{\theta}_n)$, and applying the triangle inequality, $|a + b| \leq |a| + |b|$, yields

$$\left|\hat{\theta}_n - \theta\right| \leq \left|\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right| + \left|\text{Bias}\left(\hat{\theta}_n\right)\right|.$$

Since $\text{Bias}(\hat{\theta}_n) \to 0$, there is an n_0 such that $|\,\text{Bias}(\hat{\theta}_n)| \leq \varepsilon/2$ for all $n \geq n_0$. Thus, for all $n \geq n_0$, the event $|\hat{\theta}_n - \theta| > \varepsilon$ implies $|\hat{\theta}_n - E(\hat{\theta}_n)| > \varepsilon/2$, i.e.,

$$\left\{\left|\hat{\theta}_n - \theta\right| > \varepsilon\right\} \subseteq \left\{\left|\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right| > \frac{\varepsilon}{2}\right\}.$$

Applying Chebyshev's inequality with $Z = \hat{\theta}_n$ and $c = \varepsilon/2$ yields

$$P\left(\left|\hat{\theta}_n - \theta\right| > \varepsilon\right) \leq P\left(\left|\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right| > \frac{\varepsilon}{2}\right) \leq \frac{4\,\text{Var}\left(\hat{\theta}_n\right)}{\varepsilon^2} \to 0 \quad \text{as} \quad n \to \infty. \quad \square$$

6.1.4 Asymptotic Normality and Confidence Intervals

(Asymptotic) unbiasedness and consistency are minimum requirements for any estimator. For hypothesis tests and confidence intervals, however, we need more: the **distribution** of $\hat{\theta}_n$.

Problem: The distribution of $\hat{\theta}_n = \hat{\theta}(X_1, \ldots, X_n)$ depends on the distribution F_X of the random sample, which is typically unknown.

Solution: Many estimators are **asymptotically normal**, which usually follows from a **Central Limit Theorem (CLT)**:

$$\sqrt{n}\left(\hat{\theta}_n - \theta\right) \to_D \mathcal{N}\left(0, \operatorname{AVar}\left(\hat{\theta}_n\right)\right) \quad \text{as} \quad n \to \infty.$$

Equivalently, for large n,

$$\hat{\theta}_n \overset{a}{\sim} \mathcal{N}\left(\theta, \frac{1}{n}\operatorname{AVar}\left(\hat{\theta}_n\right)\right).$$

Here, the **asymptotic variance** of $\hat{\theta}_n$ is (using $\operatorname{Var}(aZ+b) = a^2\operatorname{Var}(Z)$ for constants a, b)

$$\operatorname{AVar}\left(\hat{\theta}_n\right) = \lim_{n\to\infty} \operatorname{Var}\left(\sqrt{n}\left(\hat{\theta}_n - \theta\right)\right) = \lim_{n\to\infty} n \operatorname{Var}\left(\hat{\theta}_n\right).$$

- **Why scale by $\sqrt{n}$?** For a consistent estimator, $\operatorname{Var}(\hat{\theta}_n) \to 0$. To obtain a non-degenerate limit distribution, we must scale appropriately. For most parametric estimators, the correct scaling factor is $\sqrt{n}$.
- **Interpretation:** The estimator, centered by θ and scaled by $\sqrt{n}$, is approximately normally distributed for large n.

Such results hold only in the limit and are called **large-sample properties**. They allow us to test hypotheses and to construct confidence intervals. Let $\widehat{\operatorname{AVar}}(\hat{\theta}_n)$ be a consistent estimator of $\operatorname{AVar}(\hat{\theta}_n)$, and let $\alpha \in (0,1)$ be the significance level. Define the interval

$$\mathrm{CI}_n^{1-\alpha} = \left[\hat{\theta}_n - z_{1-\alpha/2}\sqrt{\frac{\widehat{\operatorname{AVar}}\left(\hat{\theta}_n\right)}{n}},\ \hat{\theta}_n + z_{1-\alpha/2}\sqrt{\frac{\widehat{\operatorname{AVar}}\left(\hat{\theta}_n\right)}{n}}\right],$$

where

$$z_{1-\alpha/2} = \Phi^{-1}\left(1 - \frac{\alpha}{2}\right)$$

denotes the $(1-\alpha/2)$-quantile of the standard normal distribution, and Φ denotes the standard normal distribution function. Then

$$P\left(\theta \in \mathrm{CI}_n^{1-\alpha}\right) \to 1-\alpha \quad \text{as} \quad n \to \infty,$$

so $\mathrm{CI}_n^{1-\alpha}$ is an (asymptotic) $(1-\alpha)\cdot 100\%$ **confidence interval** for θ. For example, $\alpha = 0.05$ yields $z_{0.975} \approx 1.96$ and a 95% confidence interval.

- **Interpretation:** Across all hypothetical data sets, $(1-\alpha)\cdot 100\%$ of the intervals contain θ. The interval changes from data set to data set; the parameter θ does not.
- **Caveat:** The $(1-\alpha)\cdot 100\%$ coverage holds only in the limit. For a fixed sample size n, the **actual coverage probability** $P(\theta \in \mathrm{CI}_n^{1-\alpha})$ can differ considerably from $1-\alpha$: it can be smaller (**under-coverage**) or larger (**over-coverage**). In applications, the actual coverage probability is typically unknown.

6.1.5 Why Monte Carlo Simulations?

In practice, we always work with a **fixed** sample size n. We therefore want to know the small-sample properties of $\hat{\theta}_n$ for this n, e.g.,

- the bias $\text{Bias}(\hat{\theta}_n)$,
- the variance $\text{Var}(\hat{\theta}_n)$ and the standard error $\text{SE}(\hat{\theta}_n)$,
- the mean squared error $\text{MSE}(\hat{\theta}_n)$, and
- the actual coverage probability $P(\theta \in \text{CI}_n^{1-\alpha})$.

Problem: All of these depend on the distribution of $\hat{\theta}_n$ for the given n, which is typically unknown. There are three ways to derive or approximate it:

1. **Exact derivation** under a *complete* assumption on F_X. For example, if $X_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2)$, then $\bar{X}_n \sim \mathcal{N}(\mu, \sigma^2/n)$ for every n.
   - Pro: exact for every n.
   - Con: feasible only for simple estimators and distributions; the derivations are often cumbersome; wrong if the assumption on F_X is wrong.
2. **Asymptotic approximation** under *weak* assumptions on F_X (e.g., i.i.d. with finite variance), as in the previous section.
   - Pro: requires only weak assumptions on F_X.
   - Con: exact only as $n \to \infty$; can be poor for small n.
3. **Monte Carlo simulation** under a *complete* assumption on F_X.
   - Pro: approximates the exact distribution of $\hat{\theta}_n$ with arbitrary precision for basically any F_X, any estimator, and any n; no derivations needed.
   - Con: the results hold only for the chosen F_X.

In particular, Monte Carlo simulations let us check whether the asymptotic results of Option 2 are reliable for the sample sizes we face in practice.

How Do Monte Carlo Simulations Work?

Idea: Bias, variance, MSE, and coverage probability are all **expectations**, e.g.,

$$\text{MSE}\left(\hat{\theta}_n\right) = E\left(\left(\hat{\theta}_n - \theta\right)^2\right)$$

and

$$P\left(\theta \in \text{CI}_n^{1-\alpha}\right) = E\left(\mathbf{1}\left(\theta \in \text{CI}_n^{1-\alpha}\right)\right),$$

where $\mathbf{1}(\cdot)$ denotes the indicator function. By the **law of large numbers**, we can approximate expectations by **sample means** over many independent realizations, and a computer can generate as many realizations as we want.

Monte Carlo Simulation Algorithm

Choose a distribution F_X, a sample size n, and a large number of Monte Carlo replications B (e.g., $B = 10{,}000$). For each $b = 1, \dots, B$:

1. **Simulate data:** Draw a realization of the random sample from F_X using a (pseudo-)random number generator,
$$\{X_{1,b}, \dots, X_{n,b}\}.$$
2. **Compute the estimate:** Compute the corresponding realization of the estimator (and, if needed, of the confidence interval $\mathrm{CI}_{n,b}^{1-\alpha}$ or a test statistic),
$$\hat{\theta}_{n,b} = \hat{\theta}\left(X_{1,b}, \dots, X_{n,b}\right).$$

This yields B independent realizations $\hat{\theta}_{n,1}, \dots, \hat{\theta}_{n,B}$ of the estimator.

From these B realizations, we compute the **Monte Carlo approximations**:

- **Bias:**
$$\widehat{\mathrm{Bias}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) = \bar{\theta}_{n,B} - \theta, \quad \text{where} \quad \bar{\theta}_{n,B} = \frac{1}{B}\sum_{b=1}^{B} \hat{\theta}_{n,b}.$$
- **Variance:**
$$\widehat{\mathrm{Var}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) = \frac{1}{B}\sum_{b=1}^{B}\left(\hat{\theta}_{n,b} - \bar{\theta}_{n,B}\right)^2.$$
- **Mean squared error:**
$$\widehat{\mathrm{MSE}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) = \frac{1}{B}\sum_{b=1}^{B}\left(\hat{\theta}_{n,b} - \theta\right)^2.$$
- **Coverage probability** (the share of simulated intervals that contain θ):
$$\widehat{P}_{\mathrm{MC}}\left(\theta \in \mathrm{CI}_n^{1-\alpha}\right) = \frac{1}{B}\sum_{b=1}^{B}\mathbf{1}\left(\theta \in \mathrm{CI}_{n,b}^{1-\alpha}\right).$$

By the law of large numbers, these approximations become arbitrarily precise as $B \to \infty$, e.g.,
$$\widehat{\mathrm{MSE}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) \to_P \mathrm{MSE}\left(\hat{\theta}_n\right) \quad \text{as} \quad B \to \infty.$$
Likewise, the empirical distribution function of $\hat{\theta}_{n,1}, \dots, \hat{\theta}_{n,B}$ approximates the entire distribution function of $\hat{\theta}_n$ as $B \to \infty$ (Glivenko–Cantelli theorem).

Two Different "Sample Sizes"

- n is the sample size of **one** data set. It is part of the setup we investigate: $\text{Bias}(\hat{\theta}_n)$, $\text{Var}(\hat{\theta}_n)$, etc. depend on it.
- B is the number of **simulated** data sets. It controls only the precision of the Monte Carlo approximations and should be as large as computationally feasible.

Monte Carlo results hold only for the chosen F_X and n. Therefore, we typically repeat the simulation for several distributions F_X (e.g., normal, skewed, heavy-tailed) and several sample sizes n (small, intermediate, and large).

6.2 Simulation Example: The Sample Mean

This example walks through a complete Monte Carlo simulation in five steps. Use it as a **blueprint** for your own simulations.

6.2.1 Step 1: Define the Estimator and the Questions

Let

$$\{X_1, \dots, X_n\}$$

be an i.i.d. random sample with

$$X_i \sim F_X,$$

and let

- $\mu = E(X)$
- $\sigma^2 = \text{Var}(X)$.

The parameter of interest is thus the mean, $\theta = \mu$. We estimate it by the **sample mean**

$$\hat{\mu}_n = \bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$$

and construct the **asymptotic confidence interval** from above:

$$\text{CI}_n^{1-\alpha} = \left[\bar{X}_n - z_{1-\alpha/2}\frac{s_n}{\sqrt{n}},\ \bar{X}_n + z_{1-\alpha/2}\frac{s_n}{\sqrt{n}}\right],$$

where

$$s_n = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X}_n)^2}$$

is the sample standard deviation, a consistent estimator of $\sigma = \sqrt{\text{AVar}(\bar{X}_n)}$.

Nerd's Box: The Asymptotic Variance of $\bar{X}_n$

Since $X_1, \ldots, X_n$ are independent, the variance of their sum is the sum of their variances. Since they are identically distributed, $\mathrm{Var}(X_i) = \sigma^2$ for all i. Using $\mathrm{Var}(aZ) = a^2 \mathrm{Var}(Z)$, we get

$$\mathrm{Var}\left(\bar{X}_n\right) = \mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n^2}\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right)$$

$$= \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}\left(X_i\right) = \frac{1}{n^2}\, n\,\sigma^2 = \frac{\sigma^2}{n}.$$

Thus, by the definition of the asymptotic variance,

$$\mathrm{AVar}\left(\bar{X}_n\right) = \lim_{n\to\infty} n\,\mathrm{Var}\left(\bar{X}_n\right) = \lim_{n\to\infty} n\,\frac{\sigma^2}{n} = \sigma^2.$$

Hence, $\sqrt{\mathrm{AVar}(\bar{X}_n)/n} = \sigma/\sqrt{n}$, which $\mathrm{CI}_n^{1-\alpha}$ estimates by $s_n/\sqrt{n}$.

What Does Theory Already Tell Us?

For *any* F_X with finite variance and *any* n,

$$\mathrm{Bias}\left(\bar{X}_n\right) = 0$$

and

$$\mathrm{Var}\left(\bar{X}_n\right) = \mathrm{MSE}\left(\bar{X}_n\right) = \frac{\sigma^2}{n}.$$

The coverage probability, however, is known only in the limit:

$$P(\mu \in \mathrm{CI}_n^{1-\alpha}) \to 1-\alpha \quad \text{as} \quad n \to \infty.$$

Nerd's Box: Derivations of Bias, Variance, MSE, and Coverage

Bias. By the linearity of the expectation and $E(X_i) = \mu$ for all i,

$$E\left(\bar{X}_n\right) = \frac{1}{n}\sum_{i=1}^{n} E\left(X_i\right) = \frac{1}{n}\, n\,\mu = \mu \quad \Rightarrow \quad \mathrm{Bias}\left(\bar{X}_n\right) = E\left(\bar{X}_n\right) - \mu = 0.$$

Variance. As shown in the previous box (using independence),

$$\mathrm{Var}\left(\bar{X}_n\right) = \frac{\sigma^2}{n}.$$

MSE. By the bias–variance decomposition,

$$\mathrm{MSE}\left(\bar{X}_n\right) = \left(\mathrm{Bias}\left(\bar{X}_n\right)\right)^2 + \mathrm{Var}\left(\bar{X}_n\right) = 0 + \frac{\sigma^2}{n} = \frac{\sigma^2}{n}.$$

Coverage probability. The interval contains μ if and only if the t-statistic is small:

$$\mu \in \mathrm{CI}_n^{1-\alpha} \quad \Leftrightarrow \quad |T_n| \leq z_{1-\alpha/2}, \quad \text{where} \quad T_n = \frac{\sqrt{n}\,(\bar{X}_n - \mu)}{s_n}.$$

Thus, $P(\mu \in \mathrm{CI}_n^{1-\alpha}) = P(|T_n| \leq z_{1-\alpha/2})$. As $n \to \infty$,

- $\sqrt{n}(\bar{X}_n - \mu)/\sigma \to_D \mathcal{N}(0,1)$ by the CLT, and
- $s_n \to_P \sigma$ by the law of large numbers.

Together (Slutsky's theorem), $T_n \to_D \mathcal{N}(0,1)$. Therefore, as $n \to \infty$,

$$P\left(|T_n| \leq z_{1-\alpha/2}\right) \to \Phi\left(z_{1-\alpha/2}\right) - \Phi\left(-z_{1-\alpha/2}\right) = \left(1 - \frac{\alpha}{2}\right) - \frac{\alpha}{2} = 1 - \alpha.$$

Why only in the limit? For fixed n, the distribution of T_n depends on the shape of F_X and is typically unknown.
One exception is normal data: if $F_X = \mathcal{N}(\mu, \sigma^2)$, then T_n follows a t-distribution with $n-1$ degrees of freedom. Hence, the exact coverage is

$$P\left(\mu \in \mathrm{CI}_n^{1-\alpha}\right) = 2\,F_{t_{n-1}}\left(z_{1-\alpha/2}\right) - 1 < 1 - \alpha,$$

where $F_{t_{n-1}}$ denotes the distribution function of the t_{n-1}-distribution. For $\alpha = 0.05$, this gives 0.878 for $n = 5$, 0.935 for $n = 20$, and 0.947 for $n = 100$. These values serve as a benchmark for the simulation results below. For skewed data such as χ_1^2, no such formula is available, so we need a simulation.

Questions for the Simulation

1. Do the Monte Carlo approximations of bias, variance, and MSE match the theory? (This is mainly a **sanity check** of our code.)
2. How close is the actual coverage probability to $1 - \alpha$ for small n, and does it depend on the shape of F_X? (This is the **open question**.)

6.2.2 Step 2: Choose the Simulation Design

We compare two distributions with the **same mean and variance** but different shapes:

- **Symmetric:** $F_X = \mathcal{N}(1, 2)$.
- **Right-skewed:** $F_X = \chi_1^2$, which also has $E(X) = 1$ and $\mathrm{Var}(X) = 2$.

Thus, $\mu = 1$ and $\sigma^2 = 2$ in both cases. Any difference between the two cases is due to the **shape** of F_X alone. The remaining design parameters are:

- sample sizes $n \in \{5, 10, 20, 50, 100, 500\}$,
- significance level $\alpha = 0.05$, i.e., a nominal coverage of 95%,
- $B = 10{,}000$ Monte Carlo replications per design, and
- a fixed seed for reproducibility.

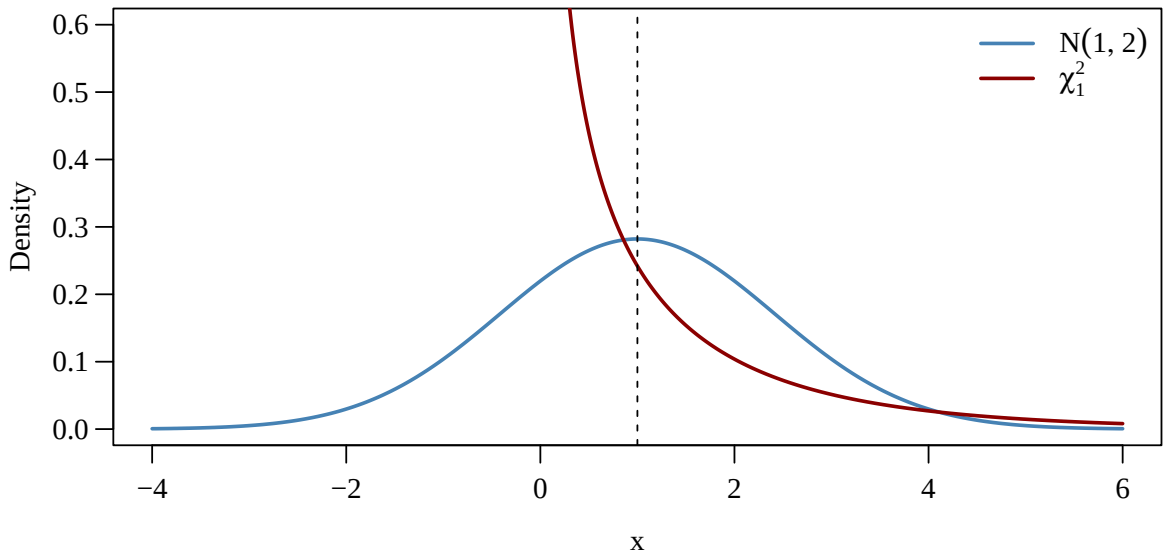


Figure 6.1: The two data distributions F_X. Both have mean $\mu = 1$ (dashed line) and variance $\sigma^2 = 2$.

6.2.3 Step 3: Write the Simulation Code

We implement the algorithm from above in three parts.

(a) Set the design parameters.

```
set.seed(123)

mu    <- 1                              # true mean E(X)
sigma <- sqrt(2)                        # true standard deviation sd(X)
n_seq <- c(5, 10, 20, 50, 100, 500)     # sample sizes
B     <- 10000                          # Monte Carlo replications
alpha <- 0.05                           # significance level
z     <- qnorm(1 - alpha / 2)           # standard normal quantile

## Random number generators for the two distributions F_X
r_FX <- list(
  "Normal"     = function(n) rnorm(n, mean = mu, sd = sigma),
  "Chi-square" = function(n) rchisq(n, df = 1)
)
```

(b) Write a function for one replication. It performs Steps 1 and 2 of the algorithm: it simulates one data set and returns the estimate and whether the confidence interval contains μ.

```
one_replication <- function(n, r_X) {
  X_sim    <- r_X(n)                                  # 1. simulate data
  mu_hat   <- mean(X_sim)                             # 2. compute estimate
  se_hat   <- sd(X_sim) / sqrt(n)                     #    and the CI
  CI_lower <- mu_hat - z * se_hat
  CI_upper <- mu_hat + z * se_hat
  covers   <- (CI_lower <= mu) & (mu <= CI_upper)
  c(mu_hat = mu_hat, covers = covers)
}
```

(c) Repeat B times for each design and compute the Monte Carlo approximations.

```
results   <- data.frame()
estimates <- list()                                   # keep realizations for
 ↪  plots

for (dist in names(r_FX)) {
  for (n in n_seq) {
    sim <- replicate(B, one_replication(n = n, r_X = r_FX[[dist]]))
    mu_hat_B <- sim["mu_hat", ]                       # B realizations of the
 ↪  estimator
    covers_B <- sim["covers", ]                       # B coverage indicators
 ↪  (0/1)

    estimates[[paste(dist, n)]] <- mu_hat_B
    results <- rbind(results, data.frame(
      dist     = dist,
      n        = n,
      Bias     = mean(mu_hat_B) - mu,
      Var      = mean((mu_hat_B - mean(mu_hat_B))^2),
      Var_theo = sigma^2 / n,
      MSE      = mean((mu_hat_B - mu)^2),
      Coverage = mean(covers_B)
    ))
  }
}
```

6.2.4 Step 4: Present the Results

Table 6.1 reports the Monte Carlo approximations for all designs. Figure 6.2 shows the simulated distributions of $\bar{X}_n$, and Figure 6.3 shows the MSE and the coverage probability as functions of n.

Table 6.1: Monte Carlo approximations ($B = 10{,}000$) for the sample mean. `Var (theory)` is the exact variance σ^2/n. The nominal coverage is 0.95.

F_X	n	Bias	Var	Var (theory)	MSE	Coverage
Normal	5	-0.0025	0.3944	0.4000	0.3945	0.880
Normal	10	0.0051	0.1984	0.2000	0.1984	0.919
Normal	20	0.0020	0.1000	0.1000	0.1000	0.934
Normal	50	-0.0017	0.0401	0.0400	0.0401	0.943
Normal	100	-0.0023	0.0201	0.0200	0.0201	0.945
Normal	500	-0.0007	0.0040	0.0040	0.0040	0.947
Chi-square	5	-0.0038	0.4020	0.4000	0.4020	0.759
Chi-square	10	0.0046	0.2058	0.2000	0.2058	0.826
Chi-square	20	-0.0050	0.0982	0.1000	0.0983	0.876
Chi-square	50	-0.0023	0.0399	0.0400	0.0399	0.913
Chi-square	100	-0.0011	0.0200	0.0200	0.0200	0.931
Chi-square	500	0.0005	0.0039	0.0040	0.0039	0.951

6.2.5 Step 5: Interpret the Results

Question 1: Bias, variance, and MSE. The simulation confirms the theory (Table 6.1 and Figure 6.3, left):

- The bias is close to zero for all designs. The remaining deviations are of the order of the Monte Carlo standard error of the bias, $\sqrt{\text{Var}(\bar{X}_n)/B}$, e.g., $\sqrt{0.4/10{,}000} \approx 0.006$ for $n = 5$.
- The variance and the MSE match σ^2/n and decrease at rate $1/n$, i.e., with slope -1 on log-log scales.
- Both distributions yield the same results, since these properties depend on F_X only through σ^2.

Question 2: Coverage. Here the simulation reveals what asymptotic theory cannot (Table 6.1 and Figure 6.3, right):

- **Under-coverage for small n:** For $n = 5$, the coverage is only 0.880 (normal) and 0.759 (χ^2_1) instead of 0.95. Even for normal data, the interval is too short because it ignores the estimation uncertainty in s_n. (Using the t_{n-1}-quantile instead of $z_{1-\alpha/2}$ would fix this for normal data, but not for skewed data.)

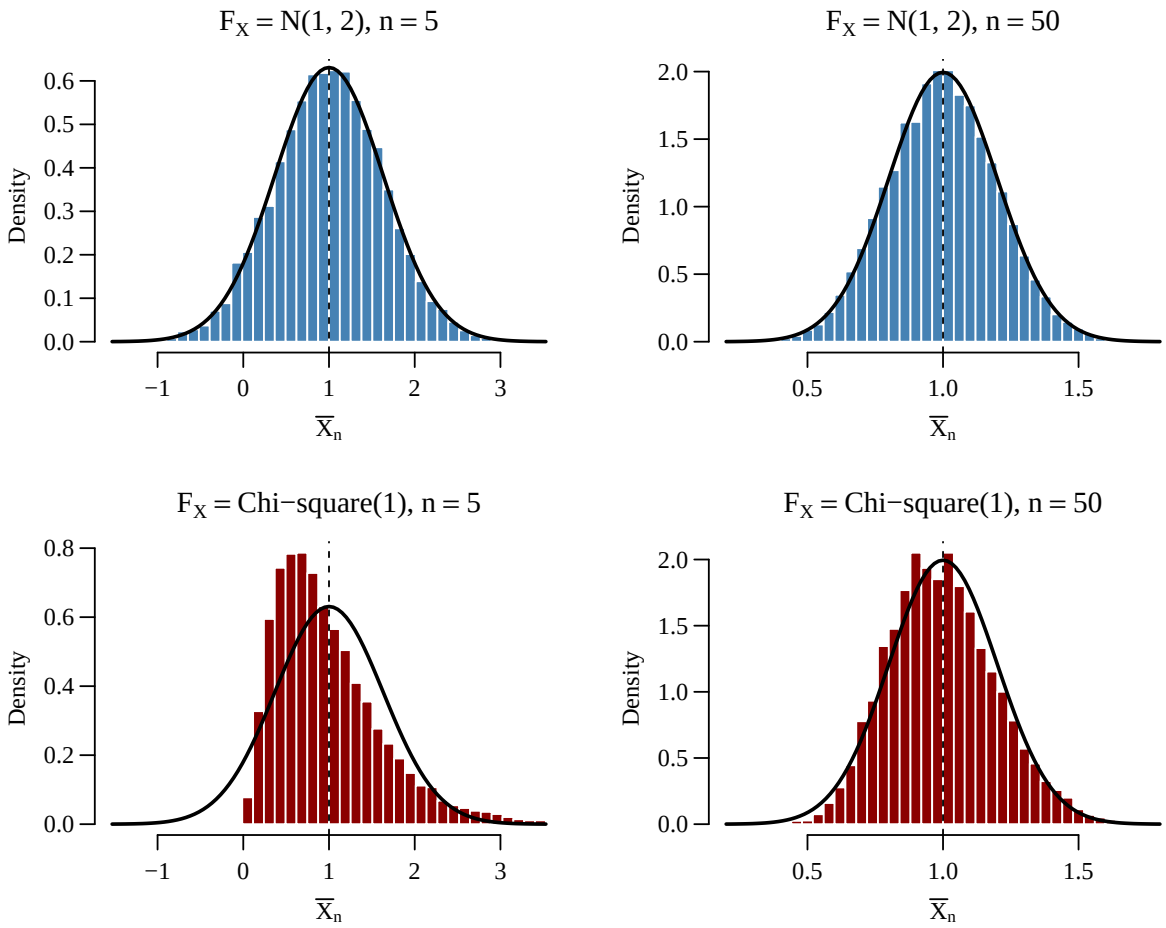


Figure 6.2: Histograms of the $B = 10{,}000$ realizations of $\bar{X}_n$. The black curve is the normal approximation $\mathcal{N}(\mu, \sigma^2/n)$ implied by the CLT; the dashed line marks $\mu = 1$.

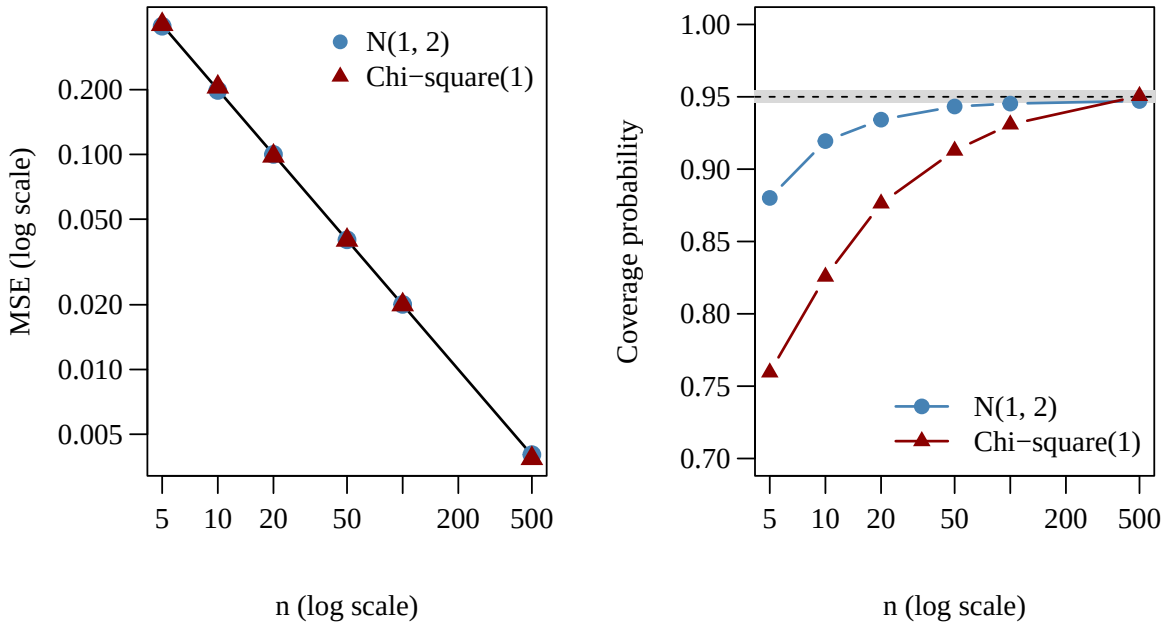


Figure 6.3: Left: MSE versus n on log-log scales; the black line is the theoretical MSE σ^2/n. Right: coverage probability versus n; the gray band shows the range $0.95 \pm 1.96 \cdot \sqrt{0.95 \cdot 0.05/B}$ expected from Monte Carlo error alone.

- **The shape of F_X matters:** Under the skewed χ^2_1 distribution, the convergence to 0.95 is much slower. Even for $n = 100$, the coverage is 0.931. The reason is visible in Figure 6.2: for small n, the distribution of $\bar{X}_n$ is still skewed, so the normal approximation behind the interval is poor.
- **Large n:** For $n = 500$, the coverage is close to 0.95 in both cases, as the CLT predicts.

How Precise Are the Monte Carlo Results?

Monte Carlo approximations are themselves estimates based on B replications. For a coverage probability p, the **Monte Carlo standard error** is

$$\sqrt{\frac{p(1-p)}{B}} \approx \sqrt{\frac{0.95 \cdot 0.05}{10{,}000}} \approx 0.002.$$

Thus, deviations from 0.95 within about ± 0.004 (the gray band in Figure 6.3) can be pure Monte Carlo error, while the under-coverage found above is real. To halve the Monte Carlo error, quadruple B.

6.2.6 Blueprint: Designing Your Own Simulation

1. **Define the estimator and the questions.** Which estimator (or test, or interval)? Which properties (bias, variance, MSE, coverage, rejection rates)? What does theory

already tell you?

2. **Choose the design.** Which distributions F_X (include cases where the assumptions are stressed, e.g., skewness or heavy tails)? Which sample sizes n? Which number of replications B? Fix a seed for the (pseudo) random number generator(s).
3. **Write the code.** Write a function for *one* replication (simulate data, compute estimate), repeat it B times for each design, and store the results in a data frame.
4. **Present the results.** Use tables for exact numbers and figures for patterns across n and F_X. Always show the target value (e.g., μ, σ^2/n, or $1-\alpha$) for comparison.
5. **Interpret the results.** Answer each question from Step 1, and account for Monte Carlo error before drawing conclusions.

6.3 Simulation Example: Hypothesis Testing

We now apply the blueprint to a hypothesis test about μ, using the same design as above: $F_X \in \{\mathcal{N}(1,2), \chi^2_1\}$, $\sigma^2 = 2$, and $B = 10{,}000$.

6.3.1 Step 1: Define the Test and the Questions

Consider the test of

$$H_0 \colon \mu = \mu_0 \quad \text{against} \quad H_1 \colon \mu \neq \mu_0$$

based on the t-statistic

$$T_n = \frac{\sqrt{n}\,(\bar{X}_n - \mu_0)}{s_n}.$$

We reject H_0 if $|T_n| > z_{1-\alpha/2}$. By the CLT, as $n \to \infty$,

- the **size**, i.e., the rejection probability if H_0 is true, converges to α, and
- the **power**, i.e., the rejection probability if H_0 is false, converges to 1.

How large are size and power for small n?

> Tests and Confidence Intervals
>
> We reject $H_0 \colon \mu = \mu_0$ if and only if $\mu_0 \notin \mathrm{CI}_n^{1-\alpha}$. Thus, under H_0,
>
> $$P\,(\text{reject } H_0) = 1 - P\left(\mu_0 \in \mathrm{CI}_n^{1-\alpha}\right),$$
>
> i.e., the size is one minus the coverage probability. The under-coverage found above therefore implies that the test rejects a true H_0 too often.

6.3.2 Step 2: Choose the Simulation Design

We test $\mu_0 = 1$ and vary the **true** mean μ over a grid from 0 to 2. To shift the mean without changing the shape of F_X, we generate

$$X_i = \mu + U_i,$$

where U_i has mean zero and variance 2: either $U_i \sim \mathcal{N}(0, 2)$ or $U_i \sim \chi^2_1 - 1$. We consider $n \in \{20, 100\}$.

6.3.3 Step 3: Write the Simulation Code

As before, we write a function for one replication and repeat it B times for each design.

```
mu_0    <- 1                                    # hypothesized mean
mu_grid <- seq(0, 2, by = 0.1)                  # true means
n_test  <- c(20, 100)                           # sample sizes

## Random number generators for the mean-zero errors U
r_U <- list(
  "Normal"     = function(n) rnorm(n, mean = 0, sd = sigma),
  "Chi-square" = function(n) rchisq(n, df = 1) - 1
)

one_test <- function(n, mu_true, r_U_dist) {
  X_sim  <- mu_true + r_U_dist(n)                          # 1. simulate
 ↪  data
  T_stat <- sqrt(n) * (mean(X_sim) - mu_0) / sd(X_sim)   # 2. test
 ↪  statistic
  abs(T_stat) > z                                          #    reject H0?
}

test_results <- data.frame()
for (dist in names(r_U)) {
  for (n in n_test) {
    for (mu_true in mu_grid) {
      reject_B <- replicate(B, one_test(n = n, mu_true = mu_true,
                                        r_U_dist = r_U[[dist]]))
      test_results <- rbind(test_results, data.frame(
        dist = dist, n = n, mu = mu_true,
        Reject = mean(reject_B)                 # MC rejection probability
      ))
    }
  }
}
```

6.3.4 Steps 4 and 5: Present and Interpret the Results

Figure 6.4 shows the rejection probabilities as functions of the true mean μ.

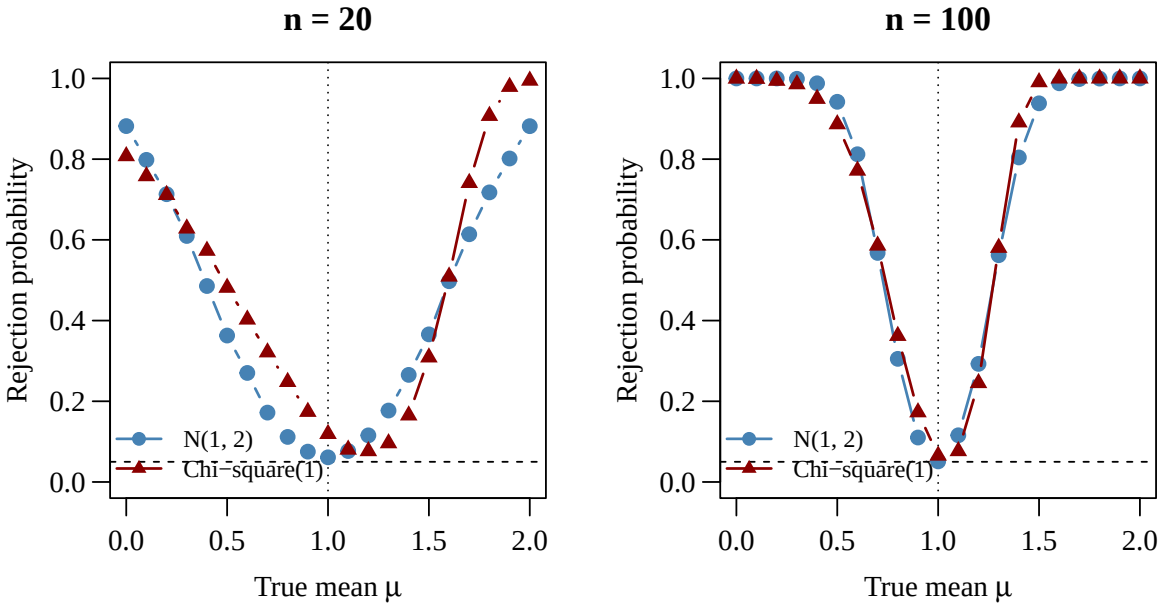


Figure 6.4: Monte Carlo rejection probabilities of the t-test of $H_0 : \mu = 1$ as functions of the true mean μ. At $\mu = 1$ (dotted line), the curves show the size; elsewhere, they show the power. The dashed line marks $\alpha = 0.05$.

- **Size:** At $\mu = \mu_0$, the rejection probability for $n = 20$ is 0.061 (normal) and 0.119 (χ^2_1) instead of $\alpha = 0.05$. As expected, this is approximately one minus the coverage probabilities in Table 6.1; the small differences are Monte Carlo error.
- **Power:** The further μ is from μ_0, the higher the power. For $n = 100$, the power is close to 1 already for $|\mu - \mu_0| \geq 0.6$; for $n = 20$, a much larger distance is needed.
- **Skewness:** Under χ^2_1, the power curve is **asymmetric**. For $n = 20$ and $\mu = 1.3$, the power (0.096) is even smaller than the size. Again, the normal approximation is poor for skewed data and small n.

6.4 Exercises VI

Use the blueprint above for both exercises.

Exercise 1:

Suppose $X \sim \mathcal{U}[0, 2]$, where $\mathcal{U}[0, 2]$ denotes the uniform distribution on the interval $[0, 2]$. It is easy to show (make sure to verify this!) that $E(X) = 1$. Now repeat the following two steps at least 1,000 times:

1. Draw a random sample of size $n = 20$ from the distribution of X.
2. Compute the sample mean and a 95% confidence interval for $E(X)$ from this sample.

You now have 1,000 sample means and 1,000 confidence intervals. Use them to answer the following questions:

a. What is the average of your 1,000 sample means?

b. What is the standard deviation of your 1,000 sample means?

c. How often does the 95% confidence interval contain $E(X)$?

Exercise 2:

In this exercise, we use the Monte Carlo method to approximate the number π. Randomly draw n points from a square and count how many of them lie inside the square's incircle. Based on this count and the formula for the area of a circle, compute an approximation of π.

7 Regression Analysis

7.1 Basic Linear Model

You have probably all learnt about the linear regression model in a previous econometrics class where

$$Y = \beta_1 X_1 + \beta_2 X_2 + ... + \beta_k X_k + \beta_{k+1} + e$$

and

$$E(e \mid X_1, X_2, ..., X_k) = 0.$$

That is, the conditional mean of the outcome variable Y is a linear function of the regressors $X_1, X_2, ..., X_k$.

When we have a random sample of size n, we can also write

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_k X_{ik} + \beta_{k+1} + e_i, \qquad i = 1, 2, ..., n.$$

Our goal is to estimate the coefficients $\beta_1, ..., \beta_{k+1}$ using the data $\{Y_i, X_{i1}, X_{i2}, ..., X_{ik}\}_{i=1}^n$.

7.2 Empirical Example

We will now see how to run regressions in R using the movie data set. Again, you can load it using

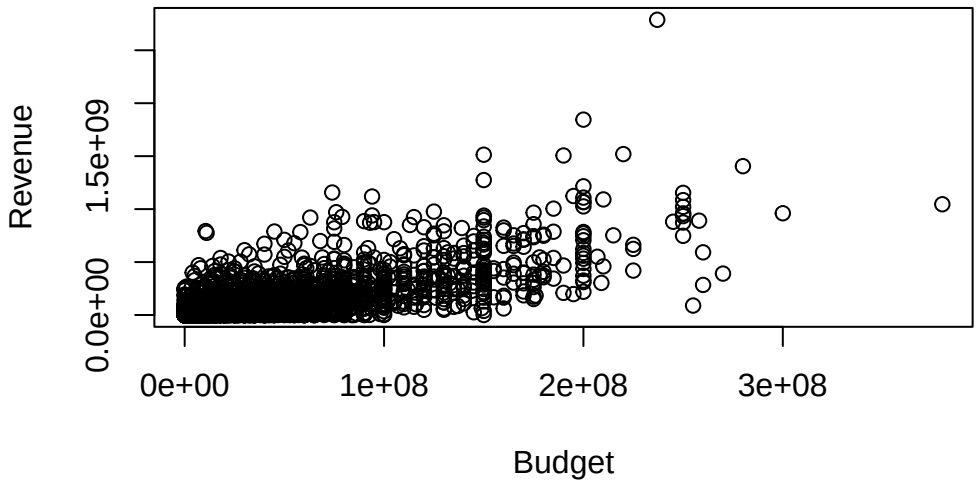
```
data_set = read.csv("tmdb_5000_movies.csv")
```

Suppose we are interested in studying how the budget of a movie is related to its revenue and we model the relationship as:

$$revenue_i = \beta_1 budget_i + \beta_2 + e_i$$

Before we run a regression, let's look at a scatter plot of the relevant variables:

```
plot(data_set$budget, data_set$revenue, xlab = "Budget", ylab = "Revenue")
```



Clearly, movies with higher budgets tend to have higher revenues. To estimate the regression coefficients, you can use the `lm()` command:

```
lm_result <- lm(formula = data_set$revenue~data_set$budget)
lm_summary  <- summary(lm_result)
lm_summary
```

```
Call:
lm(formula = data_set$revenue ~ data_set$budget)

Residuals:
       Min          1Q      Median          3Q         Max
-653371282  -35365659     2250851     8486969 2097912654

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)     -2.630e+06  1.970e+06  -1.335    0.182
data_set$budget  2.923e+00  3.940e-02  74.188   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 111200000 on 4801 degrees of freedom
Multiple R-squared:  0.5341,    Adjusted R-squared:  0.534
F-statistic:  5504 on 1 and 4801 DF,  p-value: < 2.2e-16
```

Think about how you can interpret the estimated slope coefficient.

We can also plot the regression line in the scatter plot:

```r
b0 <- lm_summary$coefficients[1]
b1  <- lm_summary$coefficients[2]

## Plot all:
plot(y=data_set$revenue, x=data_set$budget,
     xlab="Budget",
     ylab="Revenue",
     main="Scatter plot and regression line")
abline(a=b0, b=b1, col="red")
```



Let's now see if a higher budget also implies a higher rating:

```r
lm_result_rating <- lm(data_set$vote_average~data_set$budget)
lm_summary_rating  <- summary(lm_result_rating)
lm_summary_rating
```

```
Call:
lm(formula = data_set$vote_average ~ data_set$budget)

Residuals:
    Min      1Q  Median      3Q     Max
-6.0224 -0.5131  0.1139  0.7462  3.9872

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)     6.013e+00  2.108e-02 285.190  < 2e-16 ***
data_set$budget 2.732e-09  4.215e-10   6.482 9.95e-11 ***
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.19 on 4801 degrees of freedom
Multiple R-squared:  0.008676,  Adjusted R-squared:  0.00847
F-statistic: 42.02 on 1 and 4801 DF,  p-value: 9.949e-11
```

Notice that the estimated slope coefficient is tiny. Does that mean that the rating is unrelated to the budget?

The budget is often huge. For a better interpretation of the coefficients, define the budget in millions as

```
data_set$budget_mil <- data_set$budget/1000000
```

look at summary statistics of the variables

```
summary(data_set$vote_average)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.000   5.600   6.200   6.092   6.800  10.000
```

```
summary(data_set$budget_mil )
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00    0.79   15.00   29.05   40.00  380.00
```

and redo the analysis:

```
lm_result_rating <- lm(formula = data_set$vote_average~data_set$budget_mil)
lm_summary_rating  <- summary(lm_result_rating)
lm_summary_rating
```

```
Call:
lm(formula = data_set$vote_average ~ data_set$budget_mil)

Residuals:
    Min      1Q  Median      3Q     Max
-6.0224 -0.5131  0.1139  0.7462  3.9872

Coefficients:
```

```
                     Estimate Std. Error t value Pr(>|t|)
(Intercept)         6.0128066  0.0210835 285.190  < 2e-16 ***
data_set$budget_mil 0.0027325  0.0004215   6.482 9.95e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.19 on 4801 degrees of freedom
Multiple R-squared:  0.008676,  Adjusted R-squared:  0.00847
F-statistic: 42.02 on 1 and 4801 DF,  p-value: 9.949e-11
```

Do think that the budget has a large impact on the rating?

Of course, we are not limited to one regressors. To include three regressors, can use `lm(formula = Y~X1+X2+X3)`. We will return to regression analysis later in the semester.

7.3 Exercises VII

Exercise 1:

In the previous section, we looked at Monte Carlo simulations in a very simple setting. We can analyze analogous issues in other settings, such as the linear regression model. Also in this case, we have parameters (the regression coefficients) and estimators (the OLS estimators). The parameters are random and have certain known small and large sample properties that you discussed in the theoretical part of the class. Many other small sample properties, such as actual coverage rates of confidence intervals are unknown, but can be studies in Monte Carlo simulations.

Let

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i2} + \beta_3 + e_i, \qquad i = 1, 2, \dots, n.$$

where

$$\begin{pmatrix} X_{i1} \\ X_{i2} \end{pmatrix} \sim N\left(\begin{pmatrix} 2 \\ -3 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \right)$$

and $(\beta_1, \beta_2, \beta_3) = (2, 1, -1)$. Let $e_i \sim U[-2, 2]$ and assume that e_i is independent of (X_{i1}, X_{i2}). Assume that the data is a random sample. Let $(\hat{\beta}_1, \hat{\beta}_2, \hat{\beta}_3)$ be the OLS estimator of $(\beta_1, \beta_2, \beta_3)$.

For different values of n and ρ use simulations to calculate

- $E(\hat{\beta}_1)$,
- $Var(\hat{\beta}_1)$,
- the probability that β_1 is in a 95% confidence interval,
- the expected length of the confidence interval,
- the probability that we reject $H_0 : \beta_1 = 2$ using a two-sided t-test and a 1% significance level, and
- the probability that we reject $H_0 : \beta_1 = 1.8$ using a two-sided t-test and a 1% significance level.

Discuss the results.

How do you think the results will change if X_{i1} and e_i are correlated?

Exercise 2:

Start with the help file `?lm()` of the linear model - one of the functions most often used in R - and try to answer these questions:

- How do you estimate a model without a constant?
- In what setting could `na.action = na.exclude` be more useful to you than the function's default value?
- For a given lm-object, what does the function `summary` do?

Exercise 3:

Assume that $X \sim U[0,1]$ and $Y \sim U[0,1]$ are independent random variables. For different sample sizes ($n \in \{2, 5, 10, 100, 1000\}$) draw two i.i.d. samples of X and Y and regress Y on X for both samples. Using a Monte-Carlo simulation, approximate the probability that the two corresponding regression lines intersect on $[0,1] \times [0,1]$. Do the results depend on the sample size?

8 Some Notes on Useful Packages

8.1 The tidyverse Family

`"The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures."` The tidyverse family of R packages provides a comprehensive framework for many useful tools we need: from reading in data with readr [csv, tsv etc.] and haven [SPSS, Stata, SAS], wrangling it into the shape one desires with dplyr (mostly so called tidy-data) to visualization with ggplot2. These packages are not necessary to use R efficiently, but have a large following in the R community and often provide a more comfortable way to perform tedious tasks. A comprehensive guide to the tidyverse is provided in Hadley Wickham's book R for Data Science.

8.2 Tidymodels

A set of modeling packages built on the tidyverse architecture that can be used for a multitude of tasks. https://www.tidymodels.org/

8.3 lmtest

`"A collection of tests, data sets, and examples for diagnostic checking in linear regression models. Furthermore, some generic tools for inference in parametric models are provided."`

8.4 MASS

Collection of many useful functions that originated as a support package for a book on applied statistics with S (the predecessor language of R). One example is its function `mvrnorm` that enables its user to draw from a multivariate normal with a given mean vector and covariance matrix.

```r
library(MASS)
my_sample <-mvrnorm(n = 100, mu = c(0,0), Sigma = matrix(data =
 ↪  c(1,0.5,0.5,1), nrow = 2, byrow = T))
head(my_sample)
```

```
           [,1]       [,2]
[1,]  0.3696608  0.8793679
[2,] -1.5625274 -0.2190073
[3,] -0.4088915  0.7413431
[4,]  0.4536224  1.1629451
[5,]  0.8545593  0.3811284
[6,] -0.2113240 -0.6704835
```

8.5 AER - "Applied Econometrics with R"

Provides many useful function in the context of applied Econometrics. One prominent example is the function `ivreg` used for IV-estimation.

8.6 knitr

Formally an engine for dynamic report generation, knitr can be especially useful in your studies, if you want to use R in conjunction with markdown and output to different files. For example. this introduction is created from a bunch of Rmd files [R Markdown]. It can also be used to utilize R in a similar way to a Jupyter notebook, intermingling code and text freely.

8.7 Shiny

Used to build interactive web-aps with R. Somewhat more advanced, but very cool.

8.8 Faster Code

You might want to look at these packages:

- `Rcpp`
- `parallel`
- `data.table`
- `future` & `future.apply`

These can get complicated very quickly, so beware.

Part II

Introduction to Python

9 The First Steps in Python

9.1 Positron for the First Time

Positron is not only an IDE for R, but also for Python. For installation just follow the instructions on https://positron.posit.co/download.html

In addition, you need a Python installation. Positron detects installed Python versions automatically and lets you pick one as *interpreter* (the button in the upper right corner of the window). A good practice is to create a *virtual environment* for each project, i.e., a folder (usually called `.venv`) that contains the Python interpreter together with all packages you install for this project. In Positron you can create one via the command palette (**Ctrl+Shift+P**) and the command *Python: Create Environment*.

When you open Positron, its window is typically split into the following areas:

1. The editor showing your script file, which is an ordinary text file with suffix `.py`. For instance, `yourFavoritFileName.py`. This file contains your code.
2. The *Session* pane showing all objects (variables) we have defined in the environment.
3. The *Plots* and *Help* panes showing plots and help files.
4. The *Console*, which allows you to type commands and display results.
5. The *Explorer* (left) showing the files in your current project folder.

Some other terms that will be useful:

- Working directory: The file-directory you are working in. Useful commands: with `os.getcwd()` you get the location of your current working directory and `os.chdir()` allows you to set a new location for it (both functions are part of the module `os`, which you load with `import os`). This is very useful when you would like to load or save several files. You can just set your working directory globally and all files can be loaded or will be saved into this directory.
- Workspace (namespace): This is where all objects you use (e.g., data, matrices, vectors, variables, functions, etc.) are stored during your session. Useful commands in the Positron console: `%who` shows all variables in our current workspace and `%reset -f` deletes all elements in our current workspace. It is also possible to remove only some objects with `del object1, object2`.

9.2 Simple Calculations

We start with a very basic calculation. Let's type `1+1` into the console window and press enter. We get

```
1+1 # first calculation
```

```
2
```

i.e. the result of the calculation is returned.

Note that everything that is written after the #-sign is ignored by Python, which is very useful to comment your code. The second window above shows the output.

Let's consider a second calculation

```
1+2+3+ # second calculation
```

```
SyntaxError: invalid syntax (2415499698.py, line 1)
  Cell In[2], line 1
    1+2+3+ # second calculation
           ^
SyntaxError: invalid syntax
```

This calculation ended with an error. The reason is that the command can only processed if entered correctly.

```
1+2+3+4 # second calculation
```

```
10
```

A similar thing happens when parentheses are not closed.

```
2*(2+3
```

```
_IncompleteInputError: incomplete input (2817090546.py, line 1)
  Cell In[4], line 1
    2*(2+3
          ^
_IncompleteInputError: incomplete input
```

It will be helpful to use a script file, such as `yourFavoritFileName.py`, to store your Python commands. Otherwise, you would have to type your code again when an error occurred. You can send single lines or marked regions of your Python code to the console by pressing the keys **STRG+ENTER**.

9.3 Assigments and Matrix Algebra

The **assignment** operator will be your most often used tool. Here is an example where a **scalar** variable is created:

```
x = 9
x
```

```
9
```

```
x+1
```

```
10
```

Note: If you run several lines at once (e.g., a code block), Python only shows the result of the *last* line. If you want to see several results, use the `print()` function:

```
x = 9
print(x)
print(x+1)
```

```
9
10
```

In Python, assignments are always done with the `=` operator (there is no `<-` as in R). More on Python style recommendations can be found in the PEP 8 style guide.

We consider now a vector, an object you will use frequently. Python itself does not have vectors and matrices in the mathematical sense. They are provided by the package **NumPy**, which we load (under the usual short name `np`) with

```
import numpy as np
```

A vector is then created with `np.array()`:

```
z = np.array([1,3,5,6])
z
```

```
array([1, 3, 5, 6])
```

As discussed, `%who` states the content of the workspace, whereas `%reset -f` deletes the workspace.

```
%who
```

```
np   x   z
```

```
%reset -f
%who
```

```
Interactive namespace is empty.
```

Note that `%reset -f` also removed the NumPy package from the workspace, so we have to load it again.

```
import numpy as np
```

In the next step we will consider some vector multiplication. There are three different ways to multiply a vector, namely element by element, using the inner product, or using the outer product. Element by element gives you a vector of the same dimension.

```
z = np.array([1,3,5,6])
z*z                #multiplication element by element
```

```
array([ 1,  9, 25, 36])
```

The attribute `.T` gives the transpose of a vector (or matrix). It is therefore tempting to calculate the inner product as

```
z.T*z              #multiplication inner product?
```

```
array([ 1,  9, 25, 36])
```

but this does not give the desired answer. (A one-dimensional NumPy array is neither a row nor a column vector, so transposing it does not change anything.) Instead, you have to use the matrix multiplication operator `@`

```
z = np.array([1,3,5,6])
z @ z              #multiplication inner product
```

```
np.int64(71)
```

The output `np.int64(71)` simply means: the number 71, stored as a NumPy integer. (With `print(z @ z)` you would only see `71`.) Note that, in contrast to R, NumPy stores zTz as a number and not as a matrix.

```
type(z @ z)
```

```
numpy.int64
```

```
type(z)
```

```
numpy.ndarray
```

Finally, we have the outer product that gives us a matrix

```
z = np.array([1,3,5,6])
np.outer(z, z)                #multiplication outer product
```

```
array([[ 1,  3,  5,  6],
       [ 3,  9, 15, 18],
       [ 5, 15, 25, 30],
       [ 6, 18, 30, 36]])
```

In general, be very careful with `@` versus `*`. They are often both feasible, but yield different results!

You can also multiply a vector with a scalar:

```
z*4
```

```
array([ 4, 12, 20, 24])
```

A strength of NumPy is that operations can be conducted for each element of the vector in one statement

```
print(z**2)
print(np.log(z))
print((z-np.mean(z))/np.std(z, ddof=1))
```

```
[ 1  9 25 36]
[0.         1.09861229 1.60943791 1.79175947]
[-1.24021595 -0.33824071  0.56373452  1.01472214]
```

Note: `np.std()` divides by n by default. With the argument `ddof=1` it divides by $n-1$, as R's `sd()` does.

So far, we have seen the function `np.array()` that can be used to combine objects. All function calls use the same general notation: a function name is always followed by round parentheses. Sometimes, the parentheses include arguments, such as

```
z = np.arange(start = 1, stop = 6, step = 1)
z
```

```
array([1, 2, 3, 4, 5])
```

Note that the `stop` value is *not* included.

With square brackets you can access particular elements, such as

```
z[0:3]                  #refer to particular elements
```

```
array([1, 2, 3])
```

Attention: In Python, counting starts at 0, i.e., `z[0]` is the first element of `z`. Moreover, `z[0:3]` includes the elements at positions 0, 1, and 2, but not the element at position 3.

We will next define a matrix and perform matrix-vector multiplication.

```
M = np.arange(1, 10).reshape((3,3), order = "F")    # define matrix M
 ↪  (filled column by column, as in R)
print(M)
print(M.shape)                                      # dimension of M
print(z.shape)                                      # dimension of z: (5,)
 ↪  means one-dimensional with 5 elements
print(len(z))                                       # number of elements of
 ↪  z
```

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
(3, 3)
(5,)
5
```

Element-by-element multiplication for M is given by

```
M * M
```

```
array([[ 1, 16, 49],
       [ 4, 25, 64],
       [ 9, 36, 81]])
```

Standard matrix multiplication is again done using `@` such as

```
M @ M
```

```
array([[ 30,  66, 102],
       [ 36,  81, 126],
       [ 42,  96, 150]])
```

```
M.T @ M
```

```
array([[ 14,  32,  50],
       [ 32,  77, 122],
       [ 50, 122, 194]])
```

If matrices and vectors have suitable dimensions, you can multiply them as follows:

```
M @ z[0:3]
```

```
array([30, 36, 42])
```

Similar to before, with square brackets you can access particular elements, such as

```
print(M[:,2])                # state the third column of M
print(M[2,2])                # state element in the third column and third
 ↪  row of M
```

```
[7 8 9]
9
```

Here are some more examples:

```
M[:,0]                  # first column of M
M[2,2]                  # element in the third column and third row of M
z[[0,3]]                # first and fourth element of z
```

Logical statements are also very useful. For example, if you want to know which elements in the first column of matrix `M` are strictly greater than 2:

```
# See which elements:
print(M[:,0]>2)
# and extract the elements
print(M[M[:,0]>2,0])
```

```
[False False  True]
[3]
```

Note: Logical operations return so-called *boolean* objects, i.e., either a `True` or a `False`. For instance, if we ask Python whether `1>2` we get the answer `False`.

9.4 Operators

We have already seen basic arithmetic operations such as

```
+, -, *, /
```

Decimals are declared with a ".", not a comma

```
1.8 not 1,8
```

Exponents are declared with `**`, i.e. 2^3 is written as

```
2**3
```

Attention: In contrast to R, `2^3` does *not* compute 2^3 in Python (`^` is the "bitwise exclusive or" operator).

For mathematical calculations, only parentheses `()` can be used.

Frequently used mathematical functions are (most of them are provided by NumPy)

```
np.sqrt()                 # square root
np.exp()                  # exponential function
np.log()                  # nat.  Logarithm
np.log10()                # Logarithm with Base 10
abs()                     # absolute value
round(...,x)              # rounding to x decimals.
np.pi
np.e                      # Eulers number
np.sin(),np.cos(),np.tan() # trigonemetric functions
np.min(), np.max()        # returns the lowest/highest value of a
 ↪  vector/matrix
```

As a few examples enter the following calculations and make sure to understand the commands:

```
1.8+2
1.8-2
1.8*2
1.8/2
2+2*3
(2+2)*3
2**3
8**(1/3)
3**2
9**0.5
2**2*2+2
2**(2*((0.2+0.3)*(1+2)))+4
np.sqrt(2)
np.exp(1)
np.exp(2)
np.log(7.389056)
np.log(np.exp(3))
np.log10(100)
abs(1.8-2)
round(np.sqrt(2),2)
round(np.sqrt(2),4)
np.pi
np.sin(np.pi/2)
np.sin(np.pi)
np.tan(np.pi)
x=2+3j     # complex numbers: Python uses j instead of i
y=4+1j
x
y
x+y
```

```
x*y
```

Attention:

```
1.2246467991473532e-16
```

is interpreted as $1.2246467991473532 \times 10^{-16}$. This is often exactly zero, but for internal memory purposes (Python uses double-precision floating-point numbers and therefore only has up to 16 decimals), can be a rounding mistake.

9.5 Further Data Objects

Besides classical data objects such as scalars, vectors, and matrices there are three further data objects that we will use frequently:

1. The **array**: A NumPy array can have more than two dimensions. Here is an example of a 2×2×2-dimensional array

```
myFirst_Array = np.arange(1, 9).reshape((2,2,2), order = "F")
```

2. The **dictionary**: In `dict`s you can organize different kinds of data (this is the counterpart of R's `list`). E.g., consider the following example:

```
myFirst_Dict = {"Some_Numbers": [66, 76, 55, 12, 4, 66, 8, 99],
                "Animals"     : ["Rabbit", "Cat", "Elefant"],
                "My_Series"   : list(range(30, 0, -1))}
```

Python also has a data object called `list`, e.g., `["Rabbit", "Cat", "Elefant"]`, which is simply an ordered collection of arbitrary objects.

3. The **data frame**: Data frames are provided by the package **pandas**. A `DataFrame` is designed to store data, where each column can have a different type, but all columns have the same number of rows:

```
import pandas as pd

myFirst_Dataframe = pd.DataFrame({"Credit_Default"  : [ 0, 0, 1, 0, 1, 1],
                                  "Age"             : [35,41,55,36,44,26],
                                  "Loan_in_1000_EUR": [55,65,23,12,98,76]})
```

9.6 Python Help

There exist help files for all commands in Python. For example consider the help file of the function `np.sum()`. You can access the help file with

```
help(np.sum)
```

or, in the Positron console, with `?np.sum`.

The file usually contains the categories: Parameters, Returns, See Also, Notes and Examples.

After the general description follows the syntax of the command. This usually indicates which (optional) arguments the command allows. Missing values are denoted by `np.nan` ("not a number"). If a vector contains missing values, `np.sum()` returns `nan`. The function `np.nansum()` ignores missing values.

```
x = np.array([1,2,np.nan,4,5])
print(x)
print(np.sum(x))
print(np.nansum(x))
```

```
[ 1.  2. nan  4.  5.]
nan
12.0
```

Note that the behavior of pandas is different: Here, missing values are ignored by default (argument `skipna=True`).

```
pd.Series(x).sum()
```

```
np.float64(12.0)
```

9.7 Python Packages

There are tons of Python packages that are useful for different aspects of data analysis. Some packages that you will use all the time are `numpy` (vectors and matrices), `pandas` (data frames), `matplotlib` and `seaborn` (plots), and `scipy` and `statsmodels` (statistics). You can install packages in the terminal with `pip install packagename`, or directly in the Positron console with `%pip install packagename`. In order to use the package type `import packagename`, or, with a short name, `import packagename as short_name`. The package has to be installed only once (per virtual environment), but you have to load the package with `import` every time you want to use it.

9.8 Exercises I

Exercise 1:

Try to shorten the notation of the following vectors as much as possible, using `np.arange()` and `np.concatenate()`:

a. `x = np.array([157, 158, 159, 160, 161, 162, 163, 164])`
b. `x = np.array([10, 9, 8, 7, 6, 5, 4, 3, 2, 1])`
c. `x = np.array([-1071, -1072, -1073, -1074, -1075, -1074, -1073, -1072, -1071])`

Exercise 2:

The `np.arange()` function can be used in more complex operations along with arithmetic operators, and variable names. Have a look at the following expressions, and write down what sequence you think they will generate. Then check with Python.

a. `np.arange(10, 21) * 2`
b. `np.arange(105, 89, -1)`
c. `np.arange(10, 21*2)`
d. `1 + np.arange(1, 11)/10`
e. `2**np.arange(0, 6)`

Exercise 3:

NumPy has several functions for rounding. Let's start with floor, ceil, and trunc:

`np.floor(x)` rounds to the largest integer not greater than `x` `np.ceil(x)` rounds to the smallest integer not less than `x` `np.trunc(x)` returns the integer part of `x`

Below you will find a series of arguments (x), and results (y), that can be obtained by choosing one or more of the 3 functions above (e.g.`y = np.floor(x)`). Which of the above 3 functions could have been used in each case? First, choose your answer without using Python, then check with Python.

a. `x = np.array([300.99, 1.6, 583, 42.10])`

   `y = np.array([300, 1, 583, 42])`

b. `x = np.array([152.34, 1940.63, 1.0001, -2.4, np.sqrt(26)])`

   `y = np.array([152, 1940, 1, -2, 5])`

c. `x = -np.array([3.2, 444.35, 1/9, 100])`

   `y = np.array([-3, -444, 0, -100])`

d. `x = np.array([35.6, 670, -5.4, 3**3])`

   `y = np.array([36, 670, -5, 27])`

Exercise 4:

Consider the following vectors:

```
u = np.array([5, 6, 7])
```

```
v = np.array([2, 3, 4])
```

Perform the following operations:

- Add `u` and `v`
- subtract `v` from `u`
- multiply `u` by `v` element by element
- divide `u` by `v` element by element
- calculate the inner product of `u` and `v`
- calculate the outer product of `u` and `v`
- raise `u` to the power of `v` element by element

Then check your answer with Python.

10 Create and Load Data Sets

We will use Python to analyze both simulate and real data. Simulated data will be crucial to study small sample properties of estimators.

10.1 Simulated Data

We start by considering data that is simulated by the user. In NumPy, random numbers are drawn from a *random number generator*, which we create with `np.random.default_rng()`. We can then create data sets by using its methods like `normal()`, `exponential()` or `uniform()`. After the data is created we can work with it as shown in the following example:

```
import numpy as np

rng = np.random.default_rng()  # random number generator

n = 100  # number of observations

Z = rng.normal(loc = 5, scale = 3, size = n)      # draw from normal
 ↪  distribution
print(np.mean(Z))
print(np.std(Z, ddof = 1))

E = rng.exponential(scale = 1/3, size = n)        # draw data from
 ↪  exponential distribution
print(np.mean(E))
print(np.std(E, ddof = 1))


U = rng.uniform(low = 3, high = 5, size = n)      # draw data from uniform
 ↪  distribution
print(np.mean(U))
print(np.std(U, ddof = 1))
```

```
5.79675190693358
2.811374770078481
0.36976002427819216
```

```
0.35468862459920053
4.003209397431291
0.5407155794713873
```

Note that NumPy parametrizes the exponential distribution by its scale, i.e., by 1/rate.

Analogous functions exist for many other distributions, and you can find them in the NumPy documentation. Densities, distribution functions, and quantile functions are provided by the module `scipy.stats`.

10.2 Load Datasets

It is also possible to work with observed data, which will be loaded into Python. Here, we will use some real data, namely a movie database from IMDB that you can download here. The data set contains, among others, the budget, the revenue, and the average rating of movies.

Once the data is saved in your working directory, you can load it with the pandas function `pd.read_csv()`

```
import pandas as pd

data_set = pd.read_csv("tmdb_5000_movies.csv")
```

which stores the data in a data frame. You can use `data_set.head()` or the *Session* pane in Positron (click on the table symbol next to `data_set`) to look at the data.

After the file is loaded, we can work with the data using `data_set["column_name"]` (or `data_set.column_name`) to access certain parts of the data set such as

```
print(data_set["original_title"][0:5])
print(data_set["budget"][0:5])
print(data_set["budget"][0:5].mean())
```

```
0                                     Avatar
1    Pirates of the Caribbean: At World's End
2                                      Spectre
3                      The Dark Knight Rises
4                                John Carter
Name: original_title, dtype: str
0    237000000
1    300000000
2    245000000
3    250000000
```

```
4    260000000
Name: budget, dtype: int64
258400000.0
```

We will return to this data set when we talk about regression analysis later.

Apart from `pd.read_csv` there exist also `pd.read_stata`, `pd.read_excel` and many more to load suitable data sets.

Another important function is `pd.read_pickle` that is used to read in a data set that was saved using the method `to_pickle`. These functions can be useful in case properties of your Python object are not preserved when saved in other formats, such as a csv file. (Only read pickle files from sources you trust, since they can contain executable code.)

10.3 Save Datasets

There are also multiple ways to save your own data. While `pd.read_csv` can be used to read in data from a csv file, its counterpart, the data frame method `to_csv`, can be used to save a data frame as a csv file. The most important arguments of this method are `path_or_buf`, which provides the path where to save the file, and `index`, which specifies whether the row names should be saved as well.

```
data_set.to_csv(<my_path>, index = False)
```

Similarly, the method `to_pickle` allows you to save a data frame to storage. (For arbitrary Python objects you can use the module `pickle`.) One advantage compared to `to_csv` is that it preserves properties of your object that might be lost when saved as a csv file. For example, the data types of columns (e.g., dates or categorical variables) might get lost when saving a data frame as a csv file.

The methods `to_excel` and `pd.read_excel` (which require the package `openpyxl`) can be used to write to and read in excel files, respectively. This format is usually not be the best option to store your data.

10.4 Exercises II

Exercise 1:

Simulate a vector containing 10000 independent normally distributed random variables with means of 1 and variances of 2. How many of them are negative? What is the expected fraction of negative values?

Exercise 2:

Simulate `u` and `v` as independent log-normally distributed random vectors of size 200 each. Pick means and variances you like. Then let `w` be equal to `u+v`. Now calculate the following:

- the sample means of `u`, `v`, and `w`
- the sample variances of `u`, `v`, and `w`
- all pairwise covariances

Exercise 3:

Take the objects you created in the previous exercises from this chapter and save them to storage in your preferred format. Then close your session, clean your workspace and read them in again. This might be tedious at first, but integrating the export of data into your workflow might save you from time costly mistakes in the future.

11 Simple Programming

We often want to repeat specific operations several times. For example, we might want to repeat an operation for different elements of a given data structure, such as columns of a matrix, or perform an operation until a satisfactory result is reached, such as running an algorithm until the value converges to a prespecified level. In addition, we often need to check whether certain conditions are fulfilled, such as whether elements of a vector are equal to 0, and perform an operation depending on the result.

Attention: In Python, blocks of code (e.g., the body of a loop) are not enclosed by curly brackets `{}` as in R, but are defined by their *indentation*. By convention, one uses four spaces per level of indentation.

11.1 The For-Loop

A for-loop is a procedure in which a particular routine is repeated and only an index variable is changed.

```
x = 0
for i in range(1, 4):
    x = x + i
    print(x)
```

```
1
3
6
```

Note that `range(1, 4)` generates the numbers 1, 2, and 3, i.e., the `stop` value 4 is not included.

We can also use it to study the behavior of the arithmetic mean

```
import numpy as np

rng = np.random.default_rng()

out = np.zeros(100)
n = 20
```

```
for i in range(100):
    x = rng.normal(loc = 0, scale = 1, size = n)
    out[i] = np.mean(x)

print(np.mean(out))
print(np.std(out, ddof = 1))



out = np.zeros(100)
n = 200
for i in range(100):
    x = rng.normal(loc = 0, scale = 1, size = n)
    out[i] = np.mean(x)

print(np.mean(out))
print(np.std(out, ddof = 1))
```

```
0.011813343353710926
0.23018756229463744
0.01295534498887953
0.07327405143756376
```

11.2 The If-Condition

Within a for-loop, it may be useful to perform arithmetic operations only under certain conditions. The basic syntax for this is

```
if condition:
    statement
else:
    alternative
```

where `else` can remain unspecified. Further conditions can be checked with `elif condition:`.

```
###If-condition###
y = np.array([1,4,5,2,7,8,2,4])
N = len(y)

y_sq = np.zeros(N)  # creates a numeric vector of zeros of length N
```

```
for i in range(N):  # initializing the for-loop
    y_sq[i] = y[i]**2  # every element of y_sq is replaced by the
 ↪  respective squared value.
    if i == N-1:  # only when this condition is met, i.e. in the last
     ↪  iteration of the loop
        print(y_sq)  # is the vector printed
```

```
[ 1. 16. 25.  4. 49. 64.  4. 16.]
```

In shorter form the `np.where()` command is an alternative, which works element-wise on vectors.

```
np.where(test, yes, no)
```

11.3 The While-Loop

The while-loop performs a program ("statement") as long as certain data conditions ("test expression") are met. The basic syntax is

```
while test_expression:
    # increment (i+1);
    statement
```

Here, `test_expression` is evaluated and the body of the loop is entered if the result is `True`.

```
i = 1

while i < 6:
    print(i)
    i = i + 1
```

```
1
2
3
4
5
```

11.4 Time Saving and Loops

One easy way to make your loops faster is to preallocate space for your results instead of appending them to a vector. NumPy arrays have a fixed size. Appending a value with `np.append()` therefore creates a new array and copies the whole data from one place in memory to another, when you simply wanted to append another value to your vector.

If you look at the following code fragments, the second one is much faster! (We use only 20000 iterations here, since the first version takes very long otherwise.)

```
import time

# The slow but simple way
start = time.perf_counter()
results = np.array([])  # empty vector
for i in range(20000):
    results = np.append(results, rng.normal())
time1 = time.perf_counter() - start
print(time1)
```

```
0.08956197299994528
```

```
# The right way to do it
start = time.perf_counter()
results2 = np.zeros(20000)
for i in range(20000):
    results2[i] = rng.normal()
time2 = time.perf_counter() - start
print(time2)
```

```
0.016706811991753057
```

If you can avoid a loop altogether and use vector operations instead, your code will will be even faster.

```
# The right way to do it
start = time.perf_counter()
results3 = rng.normal(size = 20000)
time3 = time.perf_counter() - start
print(time3)
```

```
0.00042457800009287894
```

Try to keep this in mind when designing your own code. In Python this is one of the most frequent and easily avoidable reasons for why code is slow. (Note: Python's built-in `list` objects are an exception. Appending to a list with `my_list.append(value)` is fast.)

11.5 Exercises III

Exercise 1:

Write a for-loop that iterates over the numbers 6 to 14 and prints the cube of each number using `print()`

Exercise 2:

Write a while-loop that prints out standard random normal numbers (use `rng.normal()`) but stops (breaks) if you get a number bigger than 1.

Exercise 3:

Using a for-loop simulate the flip a coin twenty times, keeping track of the individual outcomes (1 = heads, 0 = tails) in a vector that you preallocated.

12 Functions

If you want to make your code more readable and easily understandable, you should split up complicated tasks into more simple ones. A convenient way to do this is defining a function that generalizes a step of your code. If you define this function in advance, you can call it later to perform the task, which might make the code more easily readable and enables you to use paradigmas such as recursion or metaprogramming.

12.1 Function Definition

A function definition follows a specific structure. In Python, functions are defined with the keyword `def`, and, as for loops, the body of the function is defined by its indentation.

```
def my_function(argument1, argument2):
    # here I describe what I want to do with my arguments
    # For example multiply them, assuming that they are numerical.

    my_return_value = argument1 * argument2

    # and using return we tell our function what to return
    return my_return_value

my_function(argument1 = 5, argument2 = 7)
```

```
35
```

The function can also be called without explicitly naming the arguments, but if you don't know your code very well, this might lead to difficulties when somebody else or your future self might try to understand the code.

```
my_function(2, 5)
```

```
10
```

Another useful feature is the definition of default values for your arguments such that they can be omitted when calling a function.

```
def my_2nd_function(argument1, argument2, greetings = False):
    my_return_value = argument1 * argument2
    if greetings:  # the same as: if greetings == True:
        print("Hello user, the cake is a lie.")
    return my_return_value

print(my_2nd_function(2, 7))
print(my_2nd_function(3, 12, True))
```

```
14
Hello user, the cake is a lie.
36
```

12.2 Recursion

In some settings using a recursive approach instead of loops can create more readable code. In simple terms, recursion in programming is calling a function in its own body. A simple example - even though very inefficient from a computational standpoint - is calculating factorials.

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)

factorial(17)
```

```
355687428096000
```

Note: Python limits the depth of recursion (by default to about 1000 nested calls), so `factorial(5000)` would end with a `RecursionError`.

Often recursion relies on a principle called "divide and conquer". We test for a simple base case, like n being equal to one in the case of factorials, and otherwise reducing the problem to a more simple one.

Another example is the calculation of Fibonacci numbers using recursion.

```
def fibonacci(n):
    if n == 1 or n == 2:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

fibonacci(20)
```

```
6765
```

12.3 Exercises IV

Exercise 1:

Write a function that takes 2 integers as inputs and outputs whether or not the first integer is divisible by the second as a boolean.

Exercise 2:

Try to solve the problem of adding all squares from 1 to n first using loops and then recursion. Think about the underlying way in which the function works. Which approach do you prefer? Why?

Exercise 3:

Write a function that receives a single vector of real numbers as its argument and returns an ordered vector with the same entries.

If you are interested to see recursion in action look for mergesort, that solves this problem with a cool recursive approach.

13 Plotting in Python

It is often useful to visualize data by using plots. Python offers many different possibilities to create plots, ranging from simple scatterplots to animated plots, that you can export as gifs or even interactive webapps. The standard plotting package in Python is **matplotlib**, which offers many tools to quickly generate plots. Packages like **seaborn**, which is built on top of matplotlib, offer a more convenient interface for statistical plots based on data frames.

We load matplotlib's plotting module under its usual short name `plt`:

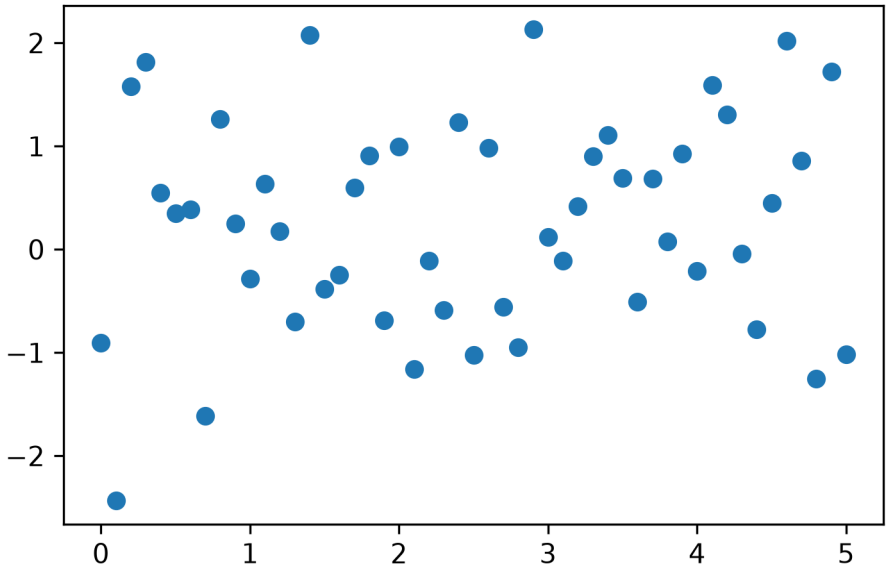
```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng()
```

13.1 The Plot-Function

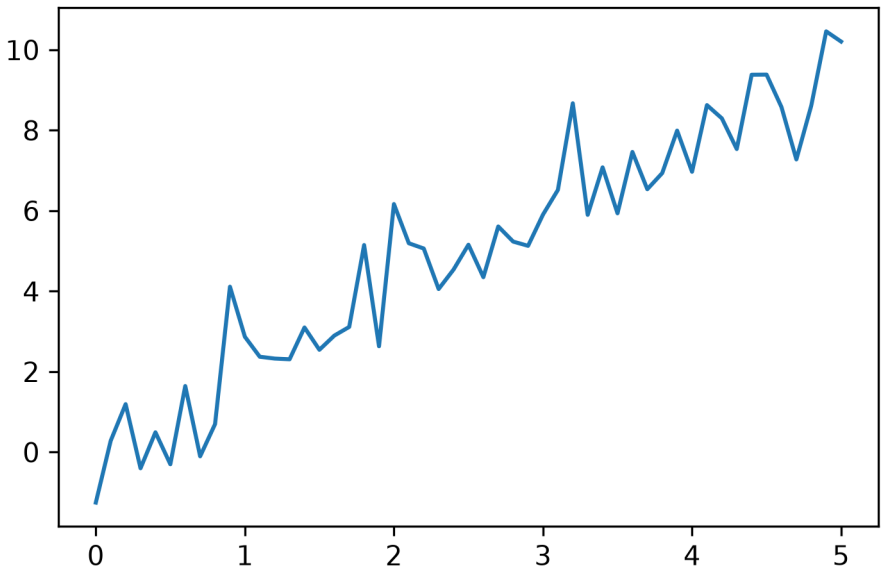
In its simplest form, the `plt.scatter()` function delivers a scatterplot.

```
x = np.linspace(0, 5, 51)   # 51 equally spaced values from 0 to 5
y = rng.normal(size = len(x))
plt.scatter(x, y)
plt.show()
```

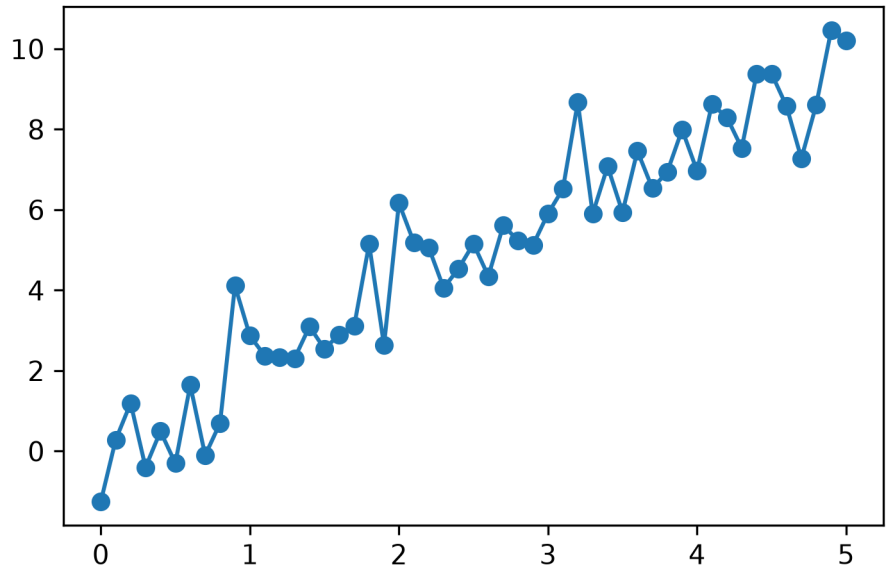


If we instead want a simple line plot or a combination of both, we can use the `plt.plot()` function, where the third argument (the *format string*) specifies this behavior (`"-"` for lines, `"o"` for points, and `"o-"` for both).

```
z = 2*x + rng.normal(size = 51)
plt.plot(x, z, "-")
plt.show()
```



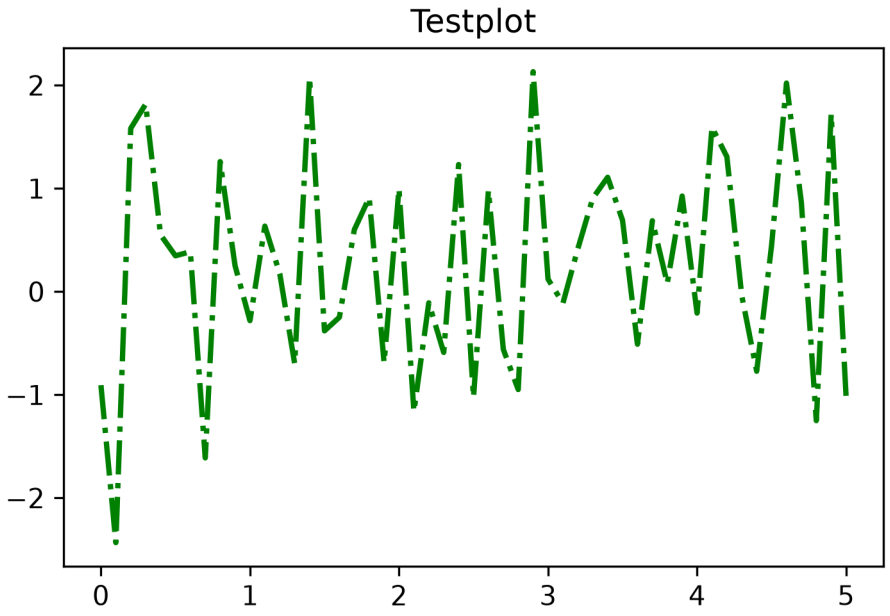
```
plt.plot(x, z, "o-")
plt.show()
```



Other arguments to the `plt.plot()` function can be used to specify color, linestyle (dashed etc.), linewidth, title, subtitle and many more. The help file `help(plt.plot)` and the matplotlib documentation can be very helpful when trying to achieve a specific look for your plot.

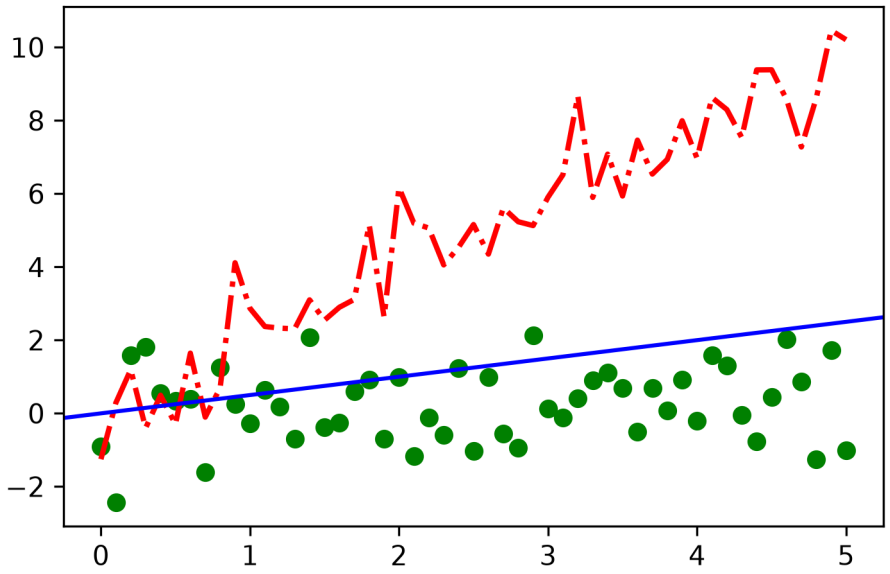
An example could look a little like this:

```
plt.plot(x, y, linestyle = "-.", linewidth = 2, color = "green")
plt.title("Testplot")
plt.show()
```



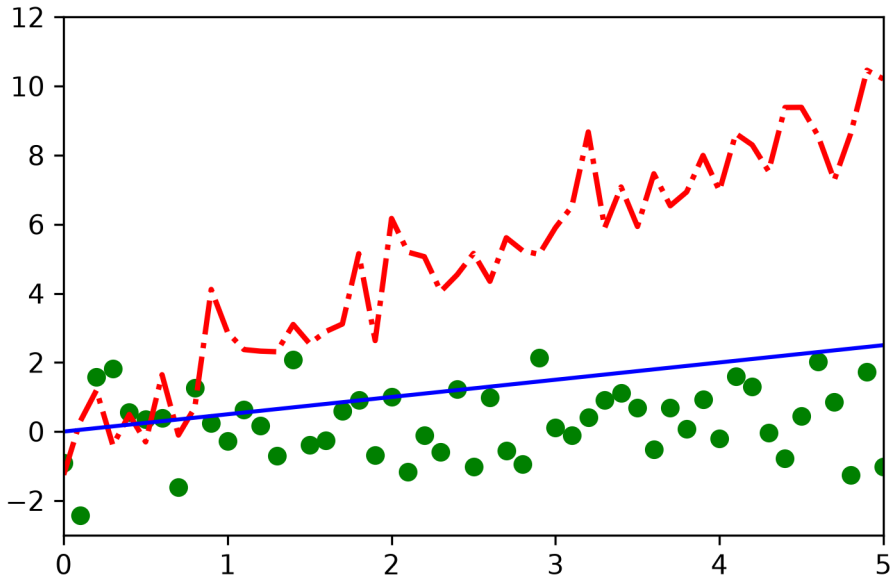
If we want to show multiple things in one diagram it can be useful to plot multiple objects in one coordinate grid. In matplotlib, all plotting commands before `plt.show()` are drawn into the same figure. The function `plt.axline()` can be used to add straight lines.

```
plt.scatter(x, y, color = "green")
plt.plot(x, z, linewidth = 2, linestyle = "-.", color = "red")
plt.axline((0, 0), slope = 0.5, color = "blue")
plt.show()
```



In contrast to R, matplotlib adjusts the range of the axes automatically to all plotted objects. But we can also manually adjust the range of our axes if we choose to do so. This is done using the functions `plt.xlim()` and `plt.ylim()`.

```
plt.scatter(x, y, color = "green")
plt.plot(x, z, linewidth = 2, linestyle = "-.", color = "red")
plt.axline((0, 0), slope = 0.5, color = "blue")
plt.xlim(0, 5)
plt.ylim(-3, 12)
plt.show()
```

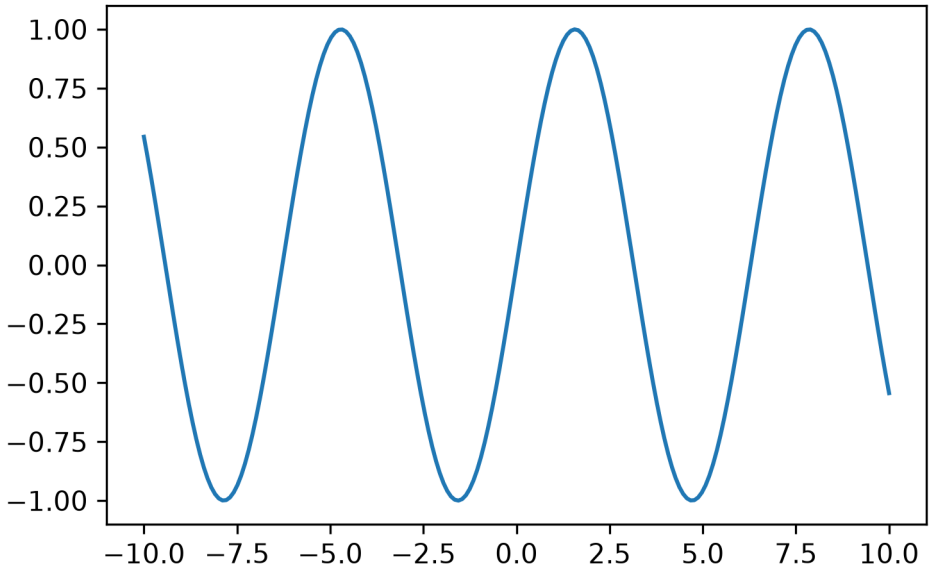


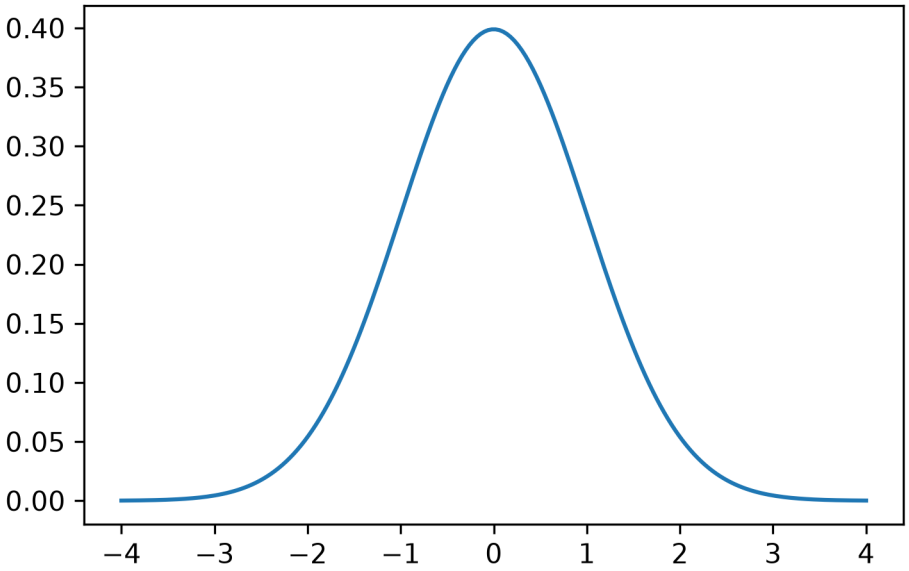
To plot functions like $\sin(x)$ or the standard normal density, we evaluate them on a fine grid of x-values. The standard normal density is provided by the module `scipy.stats`.

```
from scipy.stats import norm

grid = np.linspace(-10, 10, 200)
plt.plot(grid, np.sin(grid))
plt.show()

grid = np.linspace(-4, 4, 200)
plt.plot(grid, norm.pdf(grid))
plt.show()
```





There are many other useful plotting functions in matplotlib. We won't cover all of them here, but instead focus on some of the most frequently needed.

13.2 Histograms and Kernel Densities

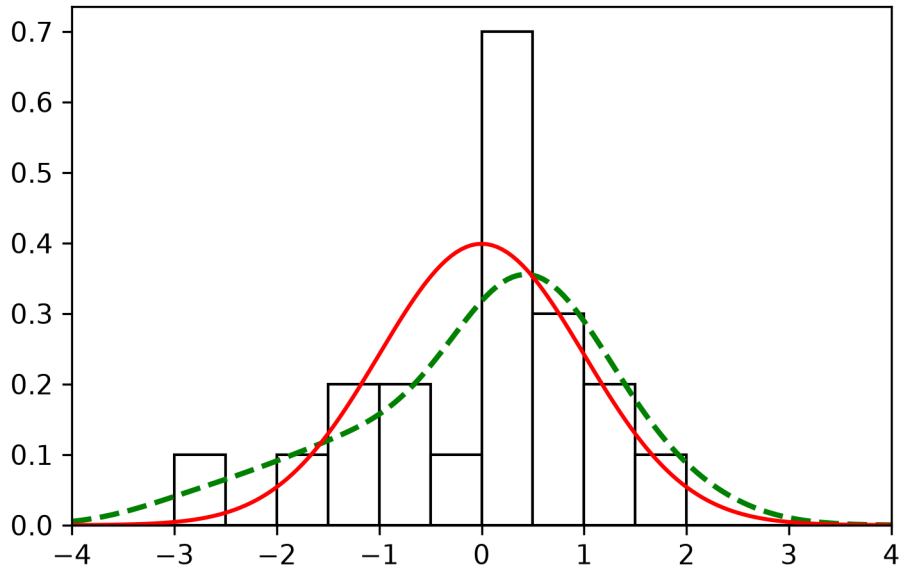
Other useful plotting functions include histograms with the function `plt.hist()`. With the function `gaussian_kde()` from `scipy.stats` we get access to kernel density estimates. A kernel density estimator is a nonparametric estimator of density of random variable based on a sample. Details about the estimator are beyond the scope of these notes. In the settings where we will use them, the default choices of the estimator will yield accurate

results. Combining our previous knowledge with these possibilities, we can create plots like the following (note: in `plt.hist()` the argument `density = True` leads to relative frequency instead of counts. See `help(plt.hist)` for more details.):

```
from scipy.stats import gaussian_kde

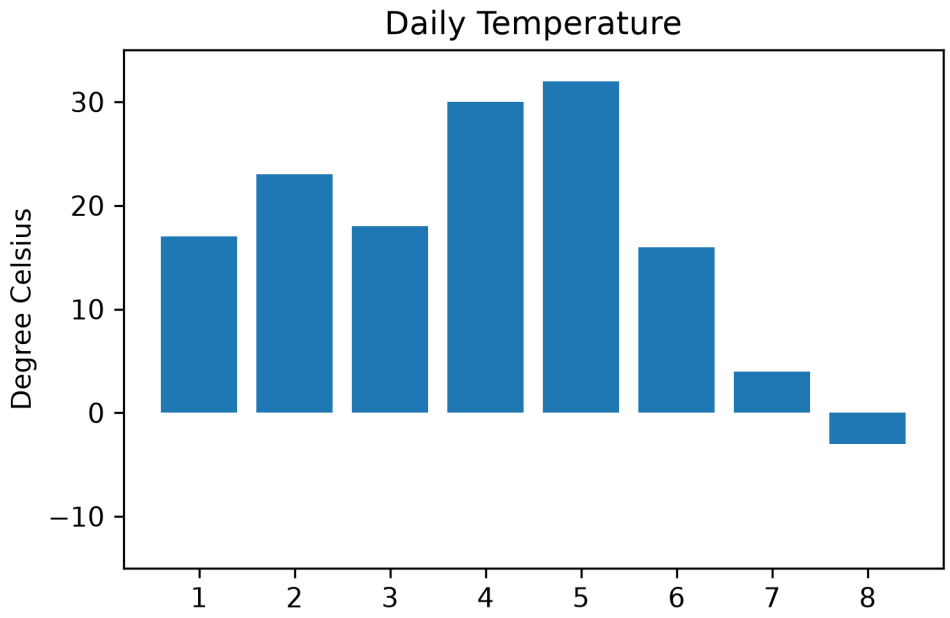
x = rng.normal(size = 20)
my_dens = gaussian_kde(x)
grid = np.linspace(-4, 4, 161)

plt.hist(x, density = True, bins = np.linspace(-4, 4, 17),
         edgecolor = "black", color = "white")
plt.xlim(-4, 4)
plt.plot(grid, my_dens(grid), color = "green", linestyle = "--", linewidth
 ↪  = 2)
plt.plot(grid, norm.pdf(grid), color = "red", linestyle = "-")
plt.show()
```



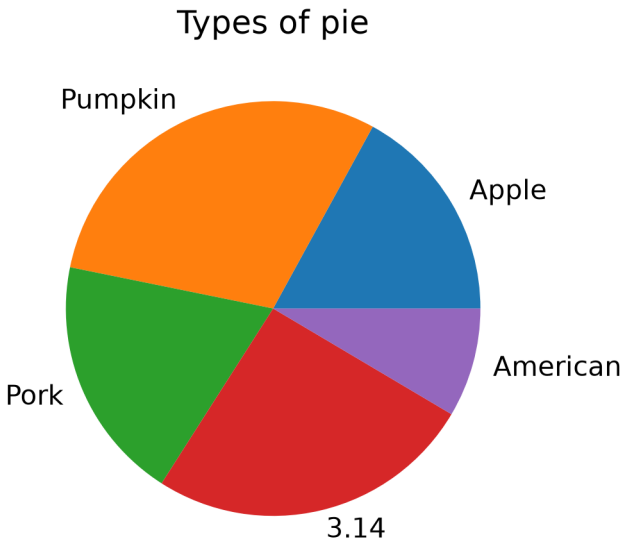
Further functions include barplots:

```
temperatures = [17,23,18,30,32,16,4,-3]
plt.bar(range(1, 9), temperatures)
plt.ylim(-15, 35)
plt.title("Daily Temperature")
plt.ylabel("Degree Celsius")
plt.show()
```



and piecharts:

```
slices = [8, 14, 9, 12, 4]
labels = ["Apple", "Pumpkin", "Pork", "3.14", "American"]
plt.pie(slices, labels = labels)
plt.title("Types of pie")
plt.show()
```

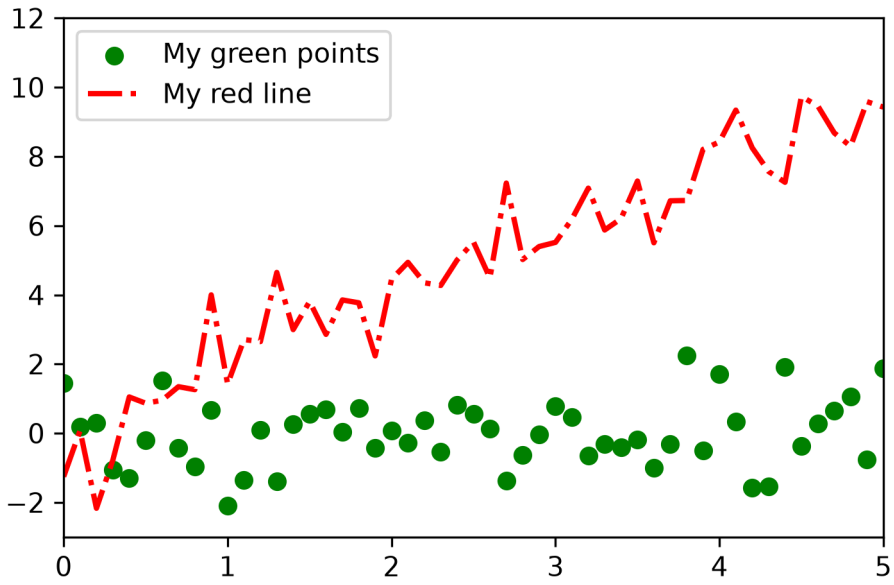


Another important part of plots - like comments in code - is a legible legend. Matplotlib has built-in support for adding a legend to your plots. Going back to our combined plot from

above, one can add a legend as shown in the following example. Here, the `label` arguments specify the contents of the legend and the `loc` argument of `plt.legend()` specifies its position in the plot. Since a legend has to be customized for each plot, one should have a look in the documentation `help(plt.legend)`.

```
x = np.linspace(0, 5, 51)   # 51 equally spaced values from 0 to 5
y = rng.normal(size = len(x))
z = 2*x + rng.normal(size = 51)

plt.scatter(x, y, color = "green", label = "My green points")
plt.plot(x, z, linewidth = 2, linestyle = "-.", color = "red", label = "My
 ↪  red line")
plt.xlim(0, 5)
plt.ylim(-3, 12)
plt.legend(loc = "upper left")
plt.show()
```



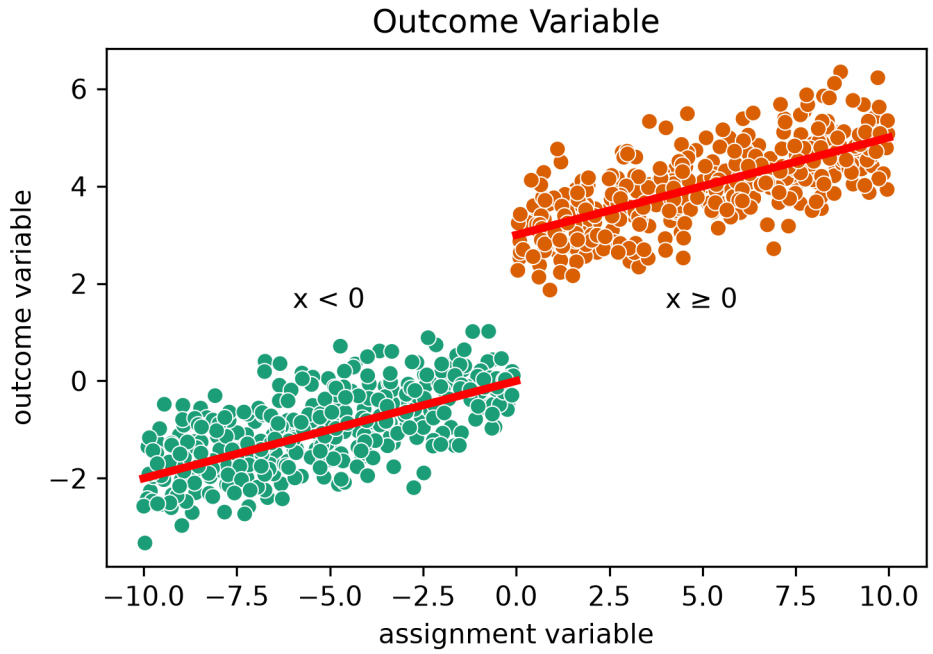
If one decides to work with data frames, the package seaborn allows for beautiful, heavily customized plots. An example can be seen below:

```
import pandas as pd
import seaborn as sns

n = 800
x = rng.uniform(low = -10, high = 10, size = n)
treatment = np.where(x > 0, 1, 0)
y = 0.2*x + treatment*3 + rng.normal(loc = 0, scale = 0.6, size = n)
```

```
visualise_data = pd.DataFrame({"x": x, "treatment": treatment, "y": y})

ax = sns.scatterplot(data = visualise_data, x = "x", y = "y",
                     hue = "treatment", palette = "Dark2", legend = False)
ax.plot([-10, 0], [-2, 0], color = "red", linewidth = 3)
ax.plot([0, 10], [3, 5], color = "red", linewidth = 3)
ax.text(-6, 1.5, "x < 0")
ax.text(4, 1.5, "x   0")
ax.set(title = "Outcome Variable",
       xlabel = "assignment variable", ylabel = "outcome variable")
plt.show()
```



The possibilities of matplotlib and seaborn are vast and can be intimidating, but seaborn offers a consistent plotting experience integrated into a data frame workflow using pandas. (If you like R's ggplot2, have a look at the Python package plotnine, which implements the "Grammar of Graphics" in Python.)

13.3 Exercises V

Exercise 1:

Draw samples of size 1000 from exponential distributions with varying values of the parameter. Use histograms to visualize differences in the distributions.

Exercise 2:

Take the `mtcars` dataset (you can load it with `import statsmodels.api as sm` and `mtcars = sm.datasets.get_rdataset("mtcars").data`, which requires an internet connection) and plot `mpg` (miles per gallon) against `wt` (weight) as a scatterplot. Add a red line with `plt.axline` by eyeballing the parameters that could come out of a regression.

Exercise 3:

Write a function that takes as its arguments a sample size n and a boolean that indicates, if plotting should take place. Let this function return a sample of size n drawn from a standard normal. If the plotting argument was true, let it also plot your sample in a way that seems sensible to you.

14 Monte Carlo Simulations

14.1 Some Important Concepts

14.1.1 Random Sample

Let

$$\{X_1, X_2, \dots, X_n\}$$

denote an **(i.i.d.) random sample**, i.e., a collection of n random variables that all have the same distribution F_X,

$$X_i \sim F_X \quad \text{for all} \quad i = 1, \dots, n,$$

and that are independent of each other,

$$X_i \perp\!\!\!\perp X_j \quad \text{for all} \quad i \neq j.$$

- The symbol $\perp\!\!\!\perp$ means **independent of**.
- The abbreviation **i.i.d.** stands for **independent and identically distributed**.

Examples: F_X and Parameters of Interest θ

1. **Normal distribution:** $F_X = \mathcal{N}(\mu_X, \sigma_X^2)$
   - $\theta = E(X) = \mu_X$, or
   - $\theta = \text{Var}(X) = \sigma_X^2$
2. **Chi-squared distribution:** $F_X = \chi^2_{\text{df}}$
   - $\theta = E(X) = \text{df}$, or
   - $\theta = \text{Var}(X) = 2 \cdot \text{df}$
3. **A distribution without a name:** F_X is some unnamed distribution (most distributions have no name)
   - $\theta = E(X)$, or
   - $\theta = \text{Median}(X)$, or
   - $\theta = p\text{-Quantile}(X)$
4. **A multivariate distribution:** F_{YX} is the joint distribution of $(Y, X^\top) \in \mathbb{R}^{K+1}$
   - $\theta = \beta$, the coefficient vector of the regression model $Y_i = X_i^\top \beta + \varepsilon_i$

14.1.2 Estimator

To estimate the unknown parameter θ, we use an **estimator**, which is simply a function of the random sample,

$$\hat{\theta}_n \equiv \hat{\theta}(X_1, X_2, \dots, X_n).$$

Since estimators are functions of random variables, they are random variables themselves.

Estimators are Random Variables

Thus, to understand an estimator, we need to know its **statistical properties**, i.e., its distribution or, at least, its mean and variance.

Observed data are just one **obs**erved realization of the random sample,

$$\{X_{1,\text{obs}}, X_{2,\text{obs}}, \dots, X_{n,\text{obs}}\}.$$

Each observed realization of the random sample yields a realization of the estimator,

$$\hat{\theta}_{n,\text{obs}} \equiv \hat{\theta}(X_{1,\text{obs}}, X_{2,\text{obs}}, \dots, X_{n,\text{obs}}).$$

This can be confusing: in practice, we typically have only one data set and, therefore, only one realization of the estimator. Still, we should keep in mind all the other data sets we could have obtained.

The following examples illustrate two random samples and their estimators:

Example: Estimating the Mean from an i.i.d. Random Sample

Let

$$\{X_1, \dots, X_n\}$$

be an i.i.d. random sample with $X_i \sim F_X$ for all $i = 1, \dots, n$.

Psst … Secret Information

In this simulation, $F_X = \mathcal{N}(\mu_X, \sigma_X^2)$ with $\mu_X = 5$ and $\sigma_X^2 = 10$. In practice, F_X, and thus μ_X and σ_X^2, are **unknown**.

We estimate the unknown mean $\theta \equiv \mu_X$ by the **sample mean**

$$\hat{\theta}_n \equiv \hat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} X_i.$$

The following code generates one realization of the random sample. The fixed seed, `default_rng(seed = 321)`, makes the pseudo-random numbers reproducible.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, chi2

rng = np.random.default_rng(seed = 321)

mean_X = 5
var_X  = 10
n      = 10

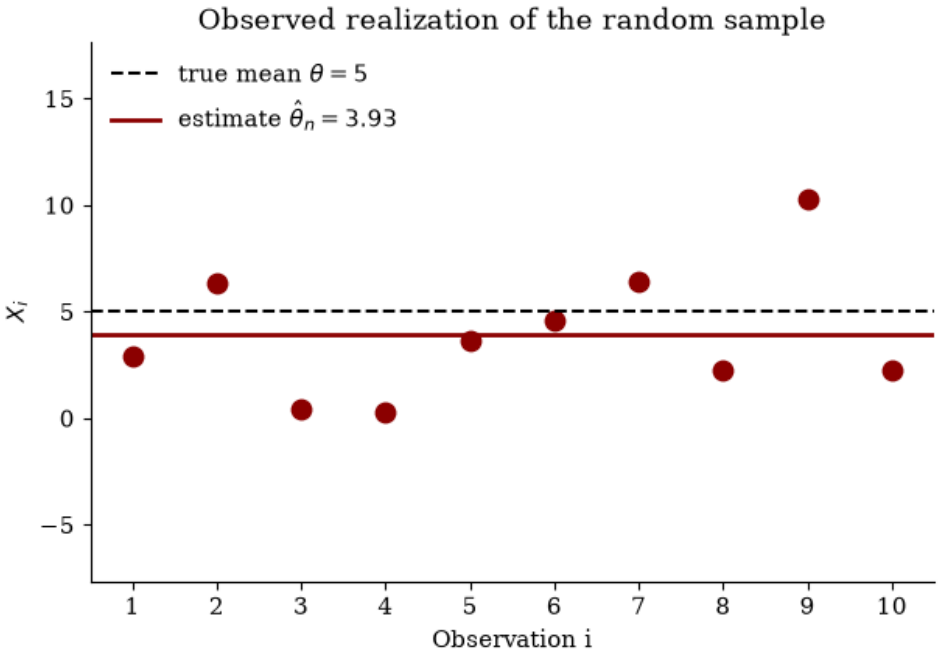
sample_X = rng.normal(loc = mean_X, scale = np.sqrt(var_X), size = n)
```

The observed realization, stored in `sample_X`, and the resulting estimate are

$$\{X_{1,\text{obs}}, \dots, X_{n,\text{obs}}\} = \{2.88,\ 6.33,\ \dots,\ 2.24\}$$

$$\hat{\theta}_{n,\text{obs}} = \frac{1}{n}\sum_{i=1}^{n} X_{i,\text{obs}} = 3.93.$$

A new sample yields a new estimate. The animation shows the observed sample (first frame, red) followed by further realizations of the random sample and of the estimator $\hat{\theta}_n$ (blue).



Summary:

- The data change from realization to realization, and so does the estimate. That is, $\hat{\theta}_n$ is a **random variable**, and $\hat{\theta}_{n,\text{obs}}$ is just one of its realizations.
- The realizations of $\hat{\theta}_n$ lie close to the (usually unknown) true mean $\mu_X = 5$. That is, the distribution of $\hat{\theta}_n$ is **concentrated** around μ_X. This concentration increases with the sample size n.

i Example: Estimating Regression Coefficients from an i.i.d. Random Sample

Let

$$\{(Y_1, X_1^\top), \dots, (Y_n, X_n^\top)\}$$

be an i.i.d. random sample with $(Y_i, X_i^\top) \sim F_{YX}$ for all $i = 1, \dots, n$, where

- $Y_i = X_i^\top \beta + \varepsilon_i$ is the outcome, and
- $X_i \in \mathbb{R}^K$ is the vector of predictors of observation i.

Psst … Secret Information

In this simulation, $X_i = (1, X_{i2})^\top$ with $X_{i2} \sim \mathcal{U}[0, 4]$, $\varepsilon_i \sim \mathcal{N}(0, 1)$ independent of X_i, and $\beta = (2, -1)^\top$. Together, these choices fully specify the joint distribution F_{YX}.
In practice, we assume (hopefully correctly) that we know the type of regression model, here the simple linear regression model

$$Y_i = X_i^\top \beta + \varepsilon_i \quad \text{with} \quad X_i = (1, X_{i2})^\top.$$

However, we know only the model's structure (here: an intercept and one predictor), not the entire model. In particular, we typically do **not know**:

- the distribution of X_i,
- the distribution of ε_i, and
- the value of β.

Thus, essential parts of F_{YX}, such as β, are **unknown**.

We estimate the unknown coefficient vector $\theta \equiv \beta$ by the **OLS estimator**

$$\hat{\theta}_n \equiv \hat{\beta}_n = (X^\top X)^{-1} X^\top Y,$$

where

$$X = \begin{pmatrix} X_1^\top \\ X_2^\top \\ \vdots \\ X_n^\top \end{pmatrix} = \begin{pmatrix} X_{11} & X_{12} & \cdots & X_{1K} \\ X_{21} & X_{22} & \cdots & X_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ X_{n1} & X_{n2} & \cdots & X_{nK} \end{pmatrix} \quad \text{and} \quad Y = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}$$

are the $n \times K$ matrix of predictors and the $n \times 1$ vector of outcomes.
The following code generates one realization of the random sample and computes the estimate.

```
rng = np.random.default_rng(seed = 321)

n    = 20
beta = np.array([2, -1])

X    = np.column_stack([np.ones(n), rng.uniform(0, 4, size = n)])   #
 ↪  first column: intercept
eps = rng.normal(loc = 0, scale = 1, size = n)
Y   = X @ beta + eps

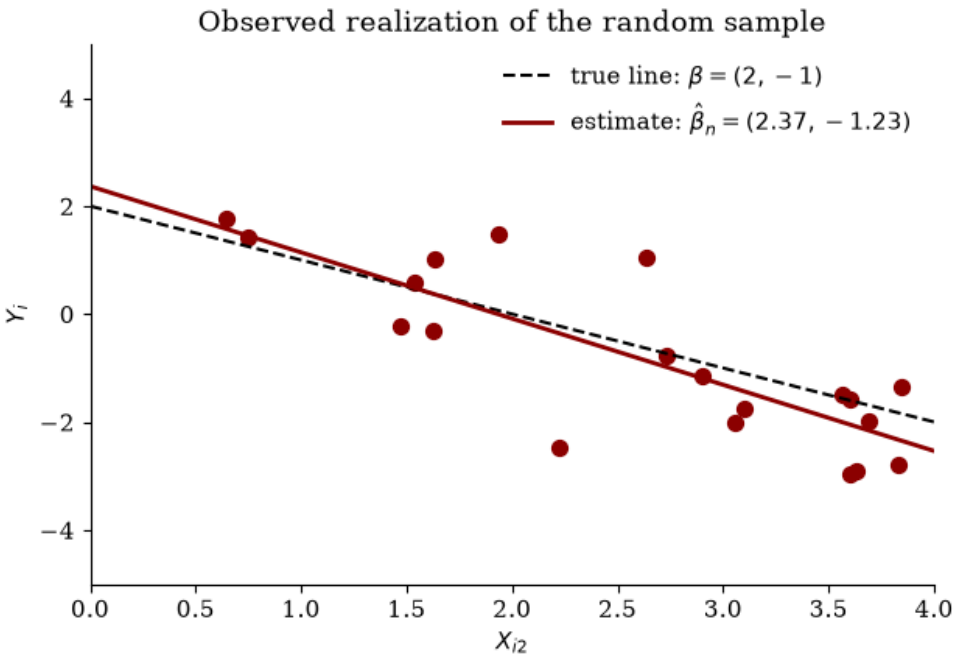
beta_hat = np.linalg.inv(X.T @ X) @ X.T @ Y
```

The observed realization, stored in `Y` and `X`, and the resulting estimate, stored in `beta_hat`, are

$$\{(Y_{1,\text{obs}}, X_{1,\text{obs}}^\top), \dots, (Y_{n,\text{obs}}, X_{n,\text{obs}}^\top)\} = \{(1.04,\ 1,\ 2.64),\ \dots,\ (1.01,\ 1,\ 1.64)\}$$

$$\hat{\theta}_{n,\text{obs}} = (2.37,\ -1.23)^\top.$$

A new sample yields a new estimate. The animation shows the observed sample (first frame, red) followed by further realizations of the random sample and of the estimator $\hat{\theta}_n$ (blue).



Summary:

- The data change from realization to realization, and so does the estimate. That is, $\hat{\theta}_n$ is a **random vector**, and $\hat{\theta}_{n,\text{obs}}$ is just one of its realizations.
- The estimated regression lines lie close to the (usually unknown) true line with $\beta = (2, -1)^\top$. That is, the distribution of $\hat{\theta}_n$ is **concentrated** around β. This concentration increases with the sample size n.

14.1.3 Bias, Variance, and Consistency of Estimators

Since an estimator is a random variable, it has a distribution and, typically, a mean and a variance. We use them to assess the quality of an estimator.

Definition 14.1 (Bias, Variance, Standard Error, and Mean Squared Error of $\hat{\theta}_n$)**.**

- The **bias** of $\hat{\theta}_n$ is
$$\text{Bias}\left(\hat{\theta}_n\right) = E\left(\hat{\theta}_n\right) - \theta.$$
- The **variance** and the **standard error** of $\hat{\theta}_n$ are
$$\text{Var}\left(\hat{\theta}_n\right) = E\left(\left(\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right)^2\right) \quad \text{and} \quad \text{SE}\left(\hat{\theta}_n\right) = \sqrt{\text{Var}\left(\hat{\theta}_n\right)}.$$
- The **mean squared error** of $\hat{\theta}_n$ combines bias and variance,
$$\text{MSE}\left(\hat{\theta}_n\right) = E\left(\left(\hat{\theta}_n - \theta\right)^2\right) = \left(\text{Bias}\left(\hat{\theta}_n\right)\right)^2 + \text{Var}\left(\hat{\theta}_n\right).$$

Bias. An estimator is called **unbiased** if

$$\text{Bias}\left(\hat{\theta}_n\right) = 0 \quad \text{for all} \quad n.$$

That is, averaged over *all* hypothetical data sets, the estimator equals θ. Unbiasedness holds for every fixed n; it is a **small-sample property**. Many estimators, however, are biased for fixed n. Then, we want them to be at least **asymptotically unbiased**,

$$\text{Bias}\left(\hat{\theta}_n\right) \to 0 \quad \text{as} \quad n \to \infty.$$

Variance. (Asymptotic) unbiasedness alone does not guarantee that the realizations of $\hat{\theta}_n$ are close to θ: if $\text{Var}(\hat{\theta}_n)$ is large, the realizations scatter widely around their mean, even in large samples. Therefore, we also want estimators to be **consistent**.

Definition 14.2 (Consistency of $\hat{\theta}_n$)**.** An estimator $\hat{\theta}_n$ is called **(weakly) consistent** if, for any (arbitrarily small) $\varepsilon > 0$,

$$P\left(\left|\hat{\theta}_n - \theta\right| > \varepsilon\right) \to 0 \quad \text{as} \quad n \to \infty.$$

In short, we write

$$\hat{\theta}_n \to_P \theta \quad \text{as} \quad n \to \infty.$$

That is, the realizations of a consistent estimator concentrate around θ as the sample size grows.

A convenient way to check consistency is to verify that both the bias and the variance vanish,

$$\text{Bias}\left(\hat{\theta}_n\right) \to 0 \quad \text{and} \quad \text{Var}\left(\hat{\theta}_n\right) \to 0 \quad \text{as} \quad n \to \infty,$$

or, equivalently, that $\text{MSE}(\hat{\theta}_n) \to 0$. This condition is sufficient (but not necessary) for consistency; see the box below for a proof.

Nerd's Box: A Sufficient Condition for Consistency

An estimator $\hat{\theta}_n$ with

$$\text{Bias}\left(\hat{\theta}_n\right) \to 0 \quad \text{and} \quad \text{Var}\left(\hat{\theta}_n\right) \to 0 \quad \text{as} \quad n \to \infty$$

is (weakly) consistent.

Proof. We use **Chebyshev's inequality**: for any random variable Z with finite variance and any $c > 0$,

$$P\left(|Z - E(Z)| > c\right) \leq \frac{\text{Var}(Z)}{c^2}.$$

Let $\varepsilon > 0$. Adding a zero, $0 = -E(\hat{\theta}_n) + E(\hat{\theta}_n)$, and applying the triangle inequality, $|a + b| \leq |a| + |b|$, yields

$$\left|\hat{\theta}_n - \theta\right| \leq \left|\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right| + \left|\text{Bias}\left(\hat{\theta}_n\right)\right|.$$

Since $\text{Bias}(\hat{\theta}_n) \to 0$, there is an n_0 such that $|\text{Bias}(\hat{\theta}_n)| \leq \varepsilon/2$ for all $n \geq n_0$. Thus, for all $n \geq n_0$, the event $|\hat{\theta}_n - \theta| > \varepsilon$ implies $|\hat{\theta}_n - E(\hat{\theta}_n)| > \varepsilon/2$, i.e.,

$$\left\{\left|\hat{\theta}_n - \theta\right| > \varepsilon\right\} \subseteq \left\{\left|\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right| > \frac{\varepsilon}{2}\right\}.$$

Applying Chebyshev's inequality with $Z = \hat{\theta}_n$ and $c = \varepsilon/2$ yields

$$P\left(\left|\hat{\theta}_n - \theta\right| > \varepsilon\right) \leq P\left(\left|\hat{\theta}_n - E\left(\hat{\theta}_n\right)\right| > \frac{\varepsilon}{2}\right) \leq \frac{4\,\text{Var}\left(\hat{\theta}_n\right)}{\varepsilon^2} \to 0 \quad \text{as} \quad n \to \infty. \quad \square$$

14.1.4 Asymptotic Normality and Confidence Intervals

(Asymptotic) unbiasedness and consistency are minimum requirements for any estimator. For hypothesis tests and confidence intervals, however, we need more: the **distribution** of $\hat{\theta}_n$.

Problem: The distribution of $\hat{\theta}_n = \hat{\theta}(X_1, \dots, X_n)$ depends on the distribution F_X of the random sample, which is typically unknown.

Solution: Many estimators are **asymptotically normal**, which usually follows from a **Central Limit Theorem (CLT)**:

$$\sqrt{n}\left(\hat{\theta}_n - \theta\right) \to_D \mathcal{N}\left(0, \operatorname{AVar}\left(\hat{\theta}_n\right)\right) \quad \text{as} \quad n \to \infty.$$

Equivalently, for large n,

$$\hat{\theta}_n \overset{a}{\sim} \mathcal{N}\left(\theta, \frac{1}{n}\operatorname{AVar}\left(\hat{\theta}_n\right)\right).$$

Here, the **asymptotic variance** of $\hat{\theta}_n$ is (using $\operatorname{Var}(aZ + b) = a^2 \operatorname{Var}(Z)$ for constants a, b)

$$\operatorname{AVar}\left(\hat{\theta}_n\right) = \lim_{n\to\infty} \operatorname{Var}\left(\sqrt{n}\left(\hat{\theta}_n - \theta\right)\right) = \lim_{n\to\infty} n \operatorname{Var}\left(\hat{\theta}_n\right).$$

- **Why scale by $\sqrt{n}$?** For a consistent estimator, $\operatorname{Var}(\hat{\theta}_n) \to 0$. To obtain a non-degenerate limit distribution, we must scale appropriately. For most parametric estimators, the correct scaling factor is $\sqrt{n}$.
- **Interpretation:** The estimator, centered by θ and scaled by $\sqrt{n}$, is approximately normally distributed for large n.

Such results hold only in the limit and are called **large-sample properties**. They allow us to test hypotheses and to construct confidence intervals. Let $\widehat{\operatorname{AVar}}(\hat{\theta}_n)$ be a consistent estimator of $\operatorname{AVar}(\hat{\theta}_n)$, and let $\alpha \in (0, 1)$ be the significance level. Define the interval

$$\mathrm{CI}_n^{1-\alpha} = \left[\hat{\theta}_n - z_{1-\alpha/2}\sqrt{\frac{\widehat{\operatorname{AVar}}\left(\hat{\theta}_n\right)}{n}},\ \hat{\theta}_n + z_{1-\alpha/2}\sqrt{\frac{\widehat{\operatorname{AVar}}\left(\hat{\theta}_n\right)}{n}}\right],$$

where

$$z_{1-\alpha/2} = \Phi^{-1}\left(1 - \frac{\alpha}{2}\right)$$

denotes the $(1 - \alpha/2)$-quantile of the standard normal distribution, and Φ denotes the standard normal distribution function. Then

$$P\left(\theta \in \mathrm{CI}_n^{1-\alpha}\right) \to 1 - \alpha \quad \text{as} \quad n \to \infty,$$

so $\mathrm{CI}_n^{1-\alpha}$ is an (asymptotic) $(1-\alpha)\cdot 100\%$ **confidence interval** for θ. For example, $\alpha = 0.05$ yields $z_{0.975} \approx 1.96$ and a 95% confidence interval.

- **Interpretation:** Across all hypothetical data sets, $(1 - \alpha) \cdot 100\%$ of the intervals contain θ. The interval changes from data set to data set; the parameter θ does not.

- **Caveat:** The $(1-\alpha)\cdot 100\%$ coverage holds only in the limit. For a fixed sample size n, the **actual coverage probability** $P(\theta \in \mathrm{CI}_n^{1-\alpha})$ can differ considerably from $1-\alpha$: it can be smaller (**under-coverage**) or larger (**over-coverage**). In applications, the actual coverage probability is typically unknown.

14.1.5 Why Monte Carlo Simulations?

In practice, we always work with a **fixed** sample size n. We therefore want to know the small-sample properties of $\hat{\theta}_n$ for this n, e.g.,

- the bias $\mathrm{Bias}(\hat{\theta}_n)$,
- the variance $\mathrm{Var}(\hat{\theta}_n)$ and the standard error $\mathrm{SE}(\hat{\theta}_n)$,
- the mean squared error $\mathrm{MSE}(\hat{\theta}_n)$, and
- the actual coverage probability $P(\theta \in \mathrm{CI}_n^{1-\alpha})$.

Problem: All of these depend on the distribution of $\hat{\theta}_n$ for the given n, which is typically unknown. There are three ways to derive or approximate it:

1. **Exact derivation** under a *complete* assumption on F_X. For example, if $X_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2)$, then $\bar{X}_n \sim \mathcal{N}(\mu, \sigma^2/n)$ for every n.
   - Pro: exact for every n.
   - Con: feasible only for simple estimators and distributions; the derivations are often cumbersome; wrong if the assumption on F_X is wrong.
2. **Asymptotic approximation** under *weak* assumptions on F_X (e.g., i.i.d. with finite variance), as in the previous section.
   - Pro: requires only weak assumptions on F_X.
   - Con: exact only as $n \to \infty$; can be poor for small n.
3. **Monte Carlo simulation** under a *complete* assumption on F_X.
   - Pro: approximates the exact distribution of $\hat{\theta}_n$ with arbitrary precision for basically any F_X, any estimator, and any n; no derivations needed.
   - Con: the results hold only for the chosen F_X.

In particular, Monte Carlo simulations let us check whether the asymptotic results of Option 2 are reliable for the sample sizes we face in practice.

How Do Monte Carlo Simulations Work?

Idea: Bias, variance, MSE, and coverage probability are all **expectations**, e.g.,

$$\operatorname{MSE}\left(\hat{\theta}_n\right) = E\left(\left(\hat{\theta}_n - \theta\right)^2\right)$$

and

$$P\left(\theta \in \operatorname{CI}_n^{1-\alpha}\right) = E\left(\mathbf{1}\left(\theta \in \operatorname{CI}_n^{1-\alpha}\right)\right),$$

where $\mathbf{1}(\cdot)$ denotes the indicator function. By the **law of large numbers**, we can approximate expectations by **sample means** over many independent realizations, and a computer can generate as many realizations as we want.

Monte Carlo Simulation Algorithm

Choose a distribution F_X, a sample size n, and a large number of Monte Carlo replications B (e.g., $B = 10{,}000$). For each $b = 1, \ldots, B$:

1. **Simulate data:** Draw a realization of the random sample from F_X using a (pseudo-)random number generator,

$$\left\{X_{1,b}, \ldots, X_{n,b}\right\}.$$

2. **Compute the estimate:** Compute the corresponding realization of the estimator (and, if needed, of the confidence interval $\operatorname{CI}_{n,b}^{1-\alpha}$ or a test statistic),

$$\hat{\theta}_{n,b} = \hat{\theta}\left(X_{1,b}, \ldots, X_{n,b}\right).$$

This yields B independent realizations $\hat{\theta}_{n,1}, \ldots, \hat{\theta}_{n,B}$ of the estimator.

From these B realizations, we compute the **Monte Carlo approximations**:

- **Bias:**

$$\widehat{\operatorname{Bias}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) = \bar{\theta}_{n,B} - \theta, \quad \text{where} \quad \bar{\theta}_{n,B} = \frac{1}{B}\sum_{b=1}^{B}\hat{\theta}_{n,b}.$$

- **Variance:**

$$\widehat{\operatorname{Var}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) = \frac{1}{B}\sum_{b=1}^{B}\left(\hat{\theta}_{n,b} - \bar{\theta}_{n,B}\right)^2.$$

- **Mean squared error:**

$$\widehat{\operatorname{MSE}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) = \frac{1}{B}\sum_{b=1}^{B}\left(\hat{\theta}_{n,b} - \theta\right)^2.$$

- **Coverage probability** (the share of simulated intervals that contain θ):

$$\widehat{P}_{\mathrm{MC}}\left(\theta \in \mathrm{CI}_n^{1-\alpha}\right) = \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}\left(\theta \in \mathrm{CI}_{n,b}^{1-\alpha}\right).$$

By the law of large numbers, these approximations become arbitrarily precise as $B \to \infty$, e.g.,

$$\widehat{\mathrm{MSE}}_{\mathrm{MC}}\left(\hat{\theta}_n\right) \to_P \mathrm{MSE}\left(\hat{\theta}_n\right) \quad \text{as} \quad B \to \infty.$$

Likewise, the empirical distribution function of $\hat{\theta}_{n,1}, \ldots, \hat{\theta}_{n,B}$ approximates the entire distribution function of $\hat{\theta}_n$ as $B \to \infty$ (Glivenko–Cantelli theorem).

Two Different "Sample Sizes"

- n is the sample size of **one** data set. It is part of the setup we investigate: $\mathrm{Bias}(\hat{\theta}_n)$, $\mathrm{Var}(\hat{\theta}_n)$, etc. depend on it.
- B is the number of **simulated** data sets. It controls only the precision of the Monte Carlo approximations and should be as large as computationally feasible.

Monte Carlo results hold only for the chosen F_X and n. Therefore, we typically repeat the simulation for several distributions F_X (e.g., normal, skewed, heavy-tailed) and several sample sizes n (small, intermediate, and large).

14.2 Simulation Example: The Sample Mean

This example walks through a complete Monte Carlo simulation in five steps. Use it as a **blueprint** for your own simulations.

14.2.1 Step 1: Define the Estimator and the Questions

Let

$$\{X_1, \ldots, X_n\}$$

be an i.i.d. random sample with

$$X_i \sim F_X,$$

and let

- $\mu = E(X)$
- $\sigma^2 = \mathrm{Var}(X)$.

The parameter of interest is thus the mean, $\theta = \mu$. We estimate it by the **sample mean**

$$\hat{\mu}_n = \bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$$

and construct the **asymptotic confidence interval** from above:

$$\mathrm{CI}_n^{1-\alpha} = \left[\bar{X}_n - z_{1-\alpha/2}\frac{s_n}{\sqrt{n}},\ \bar{X}_n + z_{1-\alpha/2}\frac{s_n}{\sqrt{n}}\right],$$

where

$$s_n = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X}_n)^2}$$

is the sample standard deviation, a consistent estimator of $\sigma = \sqrt{\mathrm{AVar}(\bar{X}_n)}$.

Nerd's Box: The Asymptotic Variance of $\bar{X}_n$

Since $X_1, \ldots, X_n$ are independent, the variance of their sum is the sum of their variances. Since they are identically distributed, $\mathrm{Var}(X_i) = \sigma^2$ for all i. Using $\mathrm{Var}(aZ) = a^2\,\mathrm{Var}(Z)$, we get

$$\mathrm{Var}\left(\bar{X}_n\right) = \mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n^2}\,\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right)$$

$$= \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}\left(X_i\right) = \frac{1}{n^2}\,n\,\sigma^2 = \frac{\sigma^2}{n}.$$

Thus, by the definition of the asymptotic variance,

$$\mathrm{AVar}\left(\bar{X}_n\right) = \lim_{n\to\infty} n\,\mathrm{Var}\left(\bar{X}_n\right) = \lim_{n\to\infty} n\,\frac{\sigma^2}{n} = \sigma^2.$$

Hence, $\sqrt{\mathrm{AVar}(\bar{X}_n)/n} = \sigma/\sqrt{n}$, which $\mathrm{CI}_n^{1-\alpha}$ estimates by $s_n/\sqrt{n}$.

What Does Theory Already Tell Us?

For *any* F_X with finite variance and *any* n,

$$\mathrm{Bias}\left(\bar{X}_n\right) = 0$$

and

$$\mathrm{Var}\left(\bar{X}_n\right) = \mathrm{MSE}\left(\bar{X}_n\right) = \frac{\sigma^2}{n}.$$

The coverage probability, however, is known only in the limit:

$$P(\mu \in \mathrm{CI}_n^{1-\alpha}) \to 1-\alpha \quad \text{as} \quad n \to \infty.$$

Nerd's Box: Derivations of Bias, Variance, MSE, and Coverage

Bias. By the linearity of the expectation and $E(X_i) = \mu$ for all i,

$$E\left(\bar{X}_n\right) = \frac{1}{n}\sum_{i=1}^{n} E\left(X_i\right) = \frac{1}{n}\, n\, \mu = \mu \quad \Rightarrow \quad \text{Bias}\left(\bar{X}_n\right) = E\left(\bar{X}_n\right) - \mu = 0.$$

Variance. As shown in the previous box (using independence),

$$\text{Var}\left(\bar{X}_n\right) = \frac{\sigma^2}{n}.$$

MSE. By the bias–variance decomposition,

$$\text{MSE}\left(\bar{X}_n\right) = \left(\text{Bias}\left(\bar{X}_n\right)\right)^2 + \text{Var}\left(\bar{X}_n\right) = 0 + \frac{\sigma^2}{n} = \frac{\sigma^2}{n}.$$

Coverage probability. The interval contains μ if and only if the t-statistic is small:

$$\mu \in \text{CI}_n^{1-\alpha} \quad \Leftrightarrow \quad |T_n| \leq z_{1-\alpha/2}, \quad \text{where} \quad T_n = \frac{\sqrt{n}\left(\bar{X}_n - \mu\right)}{s_n}.$$

Thus, $P(\mu \in \text{CI}_n^{1-\alpha}) = P(|T_n| \leq z_{1-\alpha/2})$. As $n \to \infty$,

- $\sqrt{n}(\bar{X}_n - \mu)/\sigma \to_D \mathcal{N}(0,1)$ by the CLT, and
- $s_n \to_P \sigma$ by the law of large numbers.

Together (Slutsky's theorem), $T_n \to_D \mathcal{N}(0,1)$. Therefore, as $n \to \infty$,

$$P\left(|T_n| \leq z_{1-\alpha/2}\right) \to \Phi\left(z_{1-\alpha/2}\right) - \Phi\left(-z_{1-\alpha/2}\right) = \left(1 - \frac{\alpha}{2}\right) - \frac{\alpha}{2} = 1 - \alpha.$$

Why only in the limit? For fixed n, the distribution of T_n depends on the shape of F_X and is typically unknown.
One exception is normal data: if $F_X = \mathcal{N}(\mu, \sigma^2)$, then T_n follows a t-distribution with $n-1$ degrees of freedom. Hence, the exact coverage is

$$P\left(\mu \in \text{CI}_n^{1-\alpha}\right) = 2\, F_{t_{n-1}}\left(z_{1-\alpha/2}\right) - 1 < 1 - \alpha,$$

where $F_{t_{n-1}}$ denotes the distribution function of the t_{n-1}-distribution. For $\alpha = 0.05$, this gives 0.878 for $n = 5$, 0.935 for $n = 20$, and 0.947 for $n = 100$. These values serve as a benchmark for the simulation results below. For skewed data such as χ^2_1, no such formula is available, so we need a simulation.

Questions for the Simulation

1. Do the Monte Carlo approximations of bias, variance, and MSE match the theory? (This is mainly a **sanity check** of our code.)
2. How close is the actual coverage probability to $1 - \alpha$ for small n, and does it depend on the shape of F_X? (This is the **open question**.)

14.2.2 Step 2: Choose the Simulation Design

We compare two distributions with the **same mean and variance** but different shapes:

- **Symmetric:** $F_X = \mathcal{N}(1, 2)$.
- **Right-skewed:** $F_X = \chi^2_1$, which also has $E(X) = 1$ and $\text{Var}(X) = 2$.

Thus, $\mu = 1$ and $\sigma^2 = 2$ in both cases. Any difference between the two cases is due to the **shape** of F_X alone. The remaining design parameters are:

- sample sizes $n \in \{5, 10, 20, 50, 100, 500\}$,
- significance level $\alpha = 0.05$, i.e., a nominal coverage of 95%,
- $B = 10{,}000$ Monte Carlo replications per design, and
- a fixed seed for reproducibility.

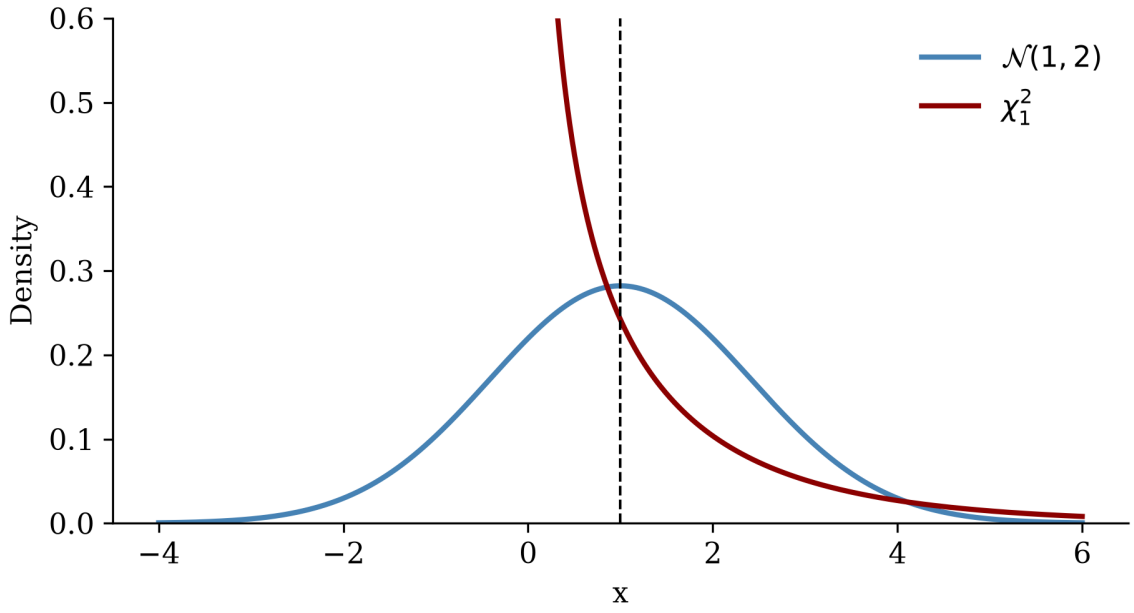


Figure 14.1: The two data distributions F_X. Both have mean $\mu = 1$ (dashed line) and variance $\sigma^2 = 2$.

14.2.3 Step 3: Write the Simulation Code

We implement the algorithm from above in three parts.

(a) Set the design parameters.

```
rng = np.random.default_rng(seed = 123)

mu    = 1                               # true mean E(X)
sigma = np.sqrt(2)                      # true standard deviation sd(X)
n_seq = [5, 10, 20, 50, 100, 500]       # sample sizes
B     = 10000                           # Monte Carlo replications
alpha = 0.05                            # significance level
z     = norm.ppf(1 - alpha / 2)         # standard normal quantile

## Random number generators for the two distributions F_X
r_FX = {
    "Normal":     lambda n: rng.normal(loc = mu, scale = sigma, size = n),
    "Chi-square": lambda n: rng.chisquare(df = 1, size = n),
}
```

(b) Write a function for one replication. It performs Steps 1 and 2 of the algorithm: it simulates one data set and returns the estimate and whether the confidence interval contains μ.

```
def one_replication(n, r_X):
    X_sim    = r_X(n)                                # 1. simulate data
    mu_hat   = X_sim.mean()                          # 2. compute estimate
    se_hat   = X_sim.std(ddof = 1) / np.sqrt(n)      #    and the CI (ddof =
 ↪  1: divide by n - 1)
    CI_lower = mu_hat - z * se_hat
    CI_upper = mu_hat + z * se_hat
    covers   = (CI_lower <= mu) and (mu <= CI_upper)
    return mu_hat, covers
```

(c) Repeat B times for each design and compute the Monte Carlo approximations.

```
rows      = []
estimates = {}                                       # keep realizations for
 ↪  plots

for dist, r_X in r_FX.items():
    for n in n_seq:
        sim = np.array([one_replication(n = n, r_X = r_X) for b in
 ↪  range(B)])
```

```
        mu_hat_B = sim[:, 0]                          # B realizations of the
↪  estimator
        covers_B = sim[:, 1]                          # B coverage indicators
↪  (0/1)

        estimates[(dist, n)] = mu_hat_B
        rows.append({
            "dist":     dist,
            "n":        n,
            "Bias":     mu_hat_B.mean() - mu,
            "Var":      np.mean((mu_hat_B - mu_hat_B.mean())**2),
            "Var_theo": sigma**2 / n,
            "MSE":      np.mean((mu_hat_B - mu)**2),
            "Coverage": covers_B.mean(),
        })

results = pd.DataFrame(rows)
```

14.2.4 Step 4: Present the Results

Table 14.1 reports the Monte Carlo approximations for all designs. Figure 14.2 shows the simulated distributions of $\bar{X}_n$, and Figure 14.3 shows the MSE and the coverage probability as functions of n.

Table 14.1: Monte Carlo approximations ($B = 10{,}000$) for the sample mean. `Var (theory)` is the exact variance σ^2/n. The nominal coverage is 0.95.

F_X	n	Bias	Var	Var (theory)	MSE	Coverage
Normal	5	0.0039	0.3958	0.4000	0.3958	0.880
Normal	10	-0.0003	0.1968	0.2000	0.1968	0.922
Normal	20	-0.0010	0.0982	0.1000	0.0982	0.937
Normal	50	0.0007	0.0402	0.0400	0.0402	0.942
Normal	100	0.0027	0.0199	0.0200	0.0199	0.949
Normal	500	-0.0000	0.0039	0.0040	0.0039	0.953
Chi-square	5	0.0139	0.4060	0.4000	0.4062	0.763
Chi-square	10	0.0089	0.2017	0.2000	0.2018	0.837
Chi-square	20	0.0011	0.1017	0.1000	0.1017	0.879
Chi-square	50	-0.0017	0.0388	0.0400	0.0388	0.917
Chi-square	100	-0.0017	0.0202	0.0200	0.0202	0.928
Chi-square	500	0.0006	0.0040	0.0040	0.0040	0.944

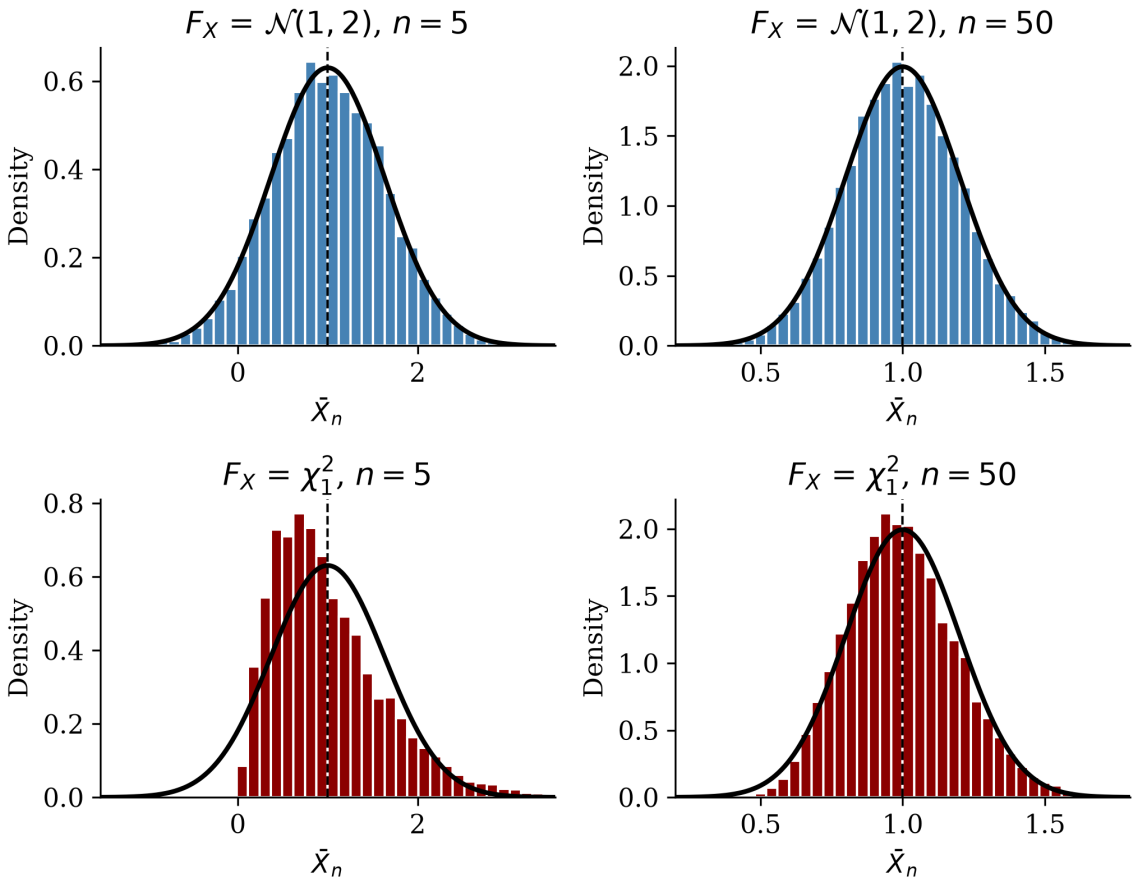


Figure 14.2: Histograms of the $B = 10{,}000$ realizations of $\bar{X}_n$. The black curve is the normal approximation $\mathcal{N}(\mu, \sigma^2/n)$ implied by the CLT; the dashed line marks $\mu = 1$.

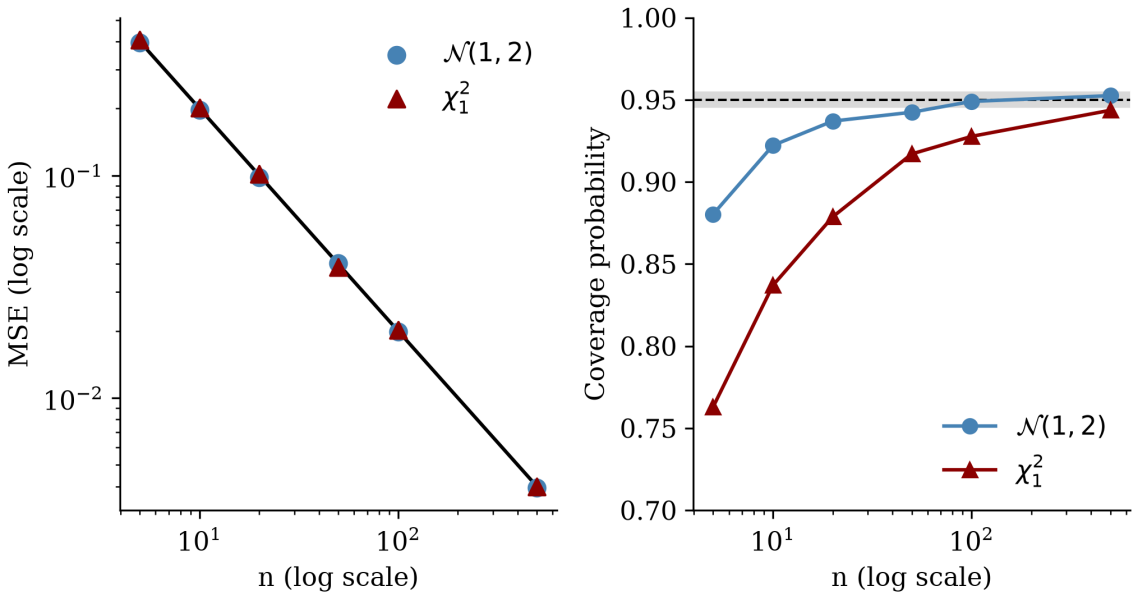


Figure 14.3: Left: MSE versus n on log-log scales; the black line is the theoretical MSE σ^2/n. Right: coverage probability versus n; the gray band shows the range $0.95 \pm 1.96 \cdot \sqrt{0.95 \cdot 0.05/B}$ expected from Monte Carlo error alone.

14.2.5 Step 5: Interpret the Results

Question 1: Bias, variance, and MSE. The simulation confirms the theory (Table 14.1 and Figure 14.3, left):

- The bias is close to zero for all designs. The remaining deviations are of the order of the Monte Carlo standard error of the bias, $\sqrt{\mathrm{Var}(\bar{X}_n)/B}$, e.g., $\sqrt{0.4/10{,}000} \approx 0.006$ for $n = 5$.
- The variance and the MSE match σ^2/n and decrease at rate $1/n$, i.e., with slope -1 on log-log scales.
- Both distributions yield the same results, since these properties depend on F_X only through σ^2.

Question 2: Coverage. Here the simulation reveals what asymptotic theory cannot (Table 14.1 and Figure 14.3, right):

- **Under-coverage for small n:** For $n = 5$, the coverage is only 0.880 (normal) and 0.763 (χ^2_1) instead of 0.95. Even for normal data, the interval is too short because it ignores the estimation uncertainty in s_n. (Using the t_{n-1}-quantile instead of $z_{1-\alpha/2}$ would fix this for normal data, but not for skewed data.)
- **The shape of F_X matters:** Under the skewed χ^2_1 distribution, the convergence to 0.95 is much slower. Even for $n = 100$, the coverage is 0.928. The reason is visible

in Figure 14.2: for small n, the distribution of $\bar{X}_n$ is still skewed, so the normal approximation behind the interval is poor.

- **Large n:** For $n = 500$, the coverage is close to 0.95 in both cases, as the CLT predicts.

How Precise Are the Monte Carlo Results?

Monte Carlo approximations are themselves estimates based on B replications. For a coverage probability p, the **Monte Carlo standard error** is

$$\sqrt{\frac{p(1-p)}{B}} \approx \sqrt{\frac{0.95 \cdot 0.05}{10{,}000}} \approx 0.002.$$

Thus, deviations from 0.95 within about ± 0.004 (the gray band in Figure 14.3) can be pure Monte Carlo error, while the under-coverage found above is real. To halve the Monte Carlo error, quadruple B.

14.2.6 Blueprint: Designing Your Own Simulation

1. **Define the estimator and the questions.** Which estimator (or test, or interval)? Which properties (bias, variance, MSE, coverage, rejection rates)? What does theory already tell you?
2. **Choose the design.** Which distributions F_X (include cases where the assumptions are stressed, e.g., skewness or heavy tails)? Which sample sizes n? Which number of replications B? Fix a seed for the (pseudo) random number generator(s).
3. **Write the code.** Write a function for *one* replication (simulate data, compute estimate), repeat it B times for each design, and store the results in a data frame.
4. **Present the results.** Use tables for exact numbers and figures for patterns across n and F_X. Always show the target value (e.g., μ, σ^2/n, or $1-\alpha$) for comparison.
5. **Interpret the results.** Answer each question from Step 1, and account for Monte Carlo error before drawing conclusions.

14.3 Simulation Example: Hypothesis Testing

We now apply the blueprint to a hypothesis test about μ, using the same design as above: $F_X \in \{\mathcal{N}(1,2), \chi^2_1\}$, $\sigma^2 = 2$, and $B = 10{,}000$.

14.3.1 Step 1: Define the Test and the Questions

Consider the test of

$$H_0 : \mu = \mu_0 \quad \text{against} \quad H_1 : \mu \neq \mu_0$$

based on the t-statistic

$$T_n = \frac{\sqrt{n}\,(\bar{X}_n - \mu_0)}{s_n}.$$

We reject H_0 if $|T_n| > z_{1-\alpha/2}$. By the CLT, as $n \to \infty$,

- the **size**, i.e., the rejection probability if H_0 is true, converges to α, and
- the **power**, i.e., the rejection probability if H_0 is false, converges to 1.

How large are size and power for small n?

> Tests and Confidence Intervals
>
> We reject $H_0 : \mu = \mu_0$ if and only if $\mu_0 \notin \mathrm{CI}_n^{1-\alpha}$. Thus, under H_0,
>
> $$P\,(\text{reject } H_0) = 1 - P\left(\mu_0 \in \mathrm{CI}_n^{1-\alpha}\right),$$
>
> i.e., the size is one minus the coverage probability. The under-coverage found above therefore implies that the test rejects a true H_0 too often.

14.3.2 Step 2: Choose the Simulation Design

We test $\mu_0 = 1$ and vary the **true** mean μ over a grid from 0 to 2. To shift the mean without changing the shape of F_X, we generate

$$X_i = \mu + U_i,$$

where U_i has mean zero and variance 2: either $U_i \sim \mathcal{N}(0, 2)$ or $U_i \sim \chi^2_1 - 1$. We consider $n \in \{20, 100\}$.

14.3.3 Step 3: Write the Simulation Code

As before, we write a function for one replication and repeat it B times for each design.

```
mu_0    = 1                                          # hypothesized mean
mu_grid = np.round(np.arange(0, 2.05, 0.1), 1)  # true means
n_test  = [20, 100]                                  # sample sizes

## Random number generators for the mean-zero errors U
r_U = {
    "Normal":     lambda n: rng.normal(loc = 0, scale = sigma, size = n),
```

```
    "Chi-square": lambda n: rng.chisquare(df = 1, size = n) - 1,
}

def one_test(n, mu_true, r_U_dist):
    X_sim  = mu_true + r_U_dist(n)                                  # 1.
 ↪  simulate data
    T_stat = np.sqrt(n) * (X_sim.mean() - mu_0) / X_sim.std(ddof = 1)  # 2.
 ↪  test statistic
    return abs(T_stat) > z                                            #
     ↪  reject H0?

rows = []
for dist, r_U_dist in r_U.items():
    for n in n_test:
        for mu_true in mu_grid:
            reject_B = [one_test(n, mu_true, r_U_dist) for b in range(B)]
            rows.append({"dist": dist, "n": n, "mu": mu_true,
                         "Reject": np.mean(reject_B)})   # MC rejection
                          ↪  probability

test_results = pd.DataFrame(rows)
```

14.3.4 Steps 4 and 5: Present and Interpret the Results

Figure 14.4 shows the rejection probabilities as functions of the true mean μ.

- **Size:** At $\mu = \mu_0$, the rejection probability for $n = 20$ is 0.064 (normal) and 0.124 (χ^2_1) instead of $\alpha = 0.05$. As expected, this is approximately one minus the coverage probabilities in Table 14.1; the small differences are Monte Carlo error.
- **Power:** The further μ is from μ_0, the higher the power. For $n = 100$, the power is close to 1 already for $|\mu - \mu_0| \geq 0.6$; for $n = 20$, a much larger distance is needed.
- **Skewness:** Under χ^2_1, the power curve is **asymmetric**. For $n = 20$ and $\mu = 1.3$, the power (0.091) is even smaller than the size. Again, the normal approximation is poor for skewed data and small n.

14.4 Exercises VI

Use the blueprint above for both exercises.

Exercise 1:

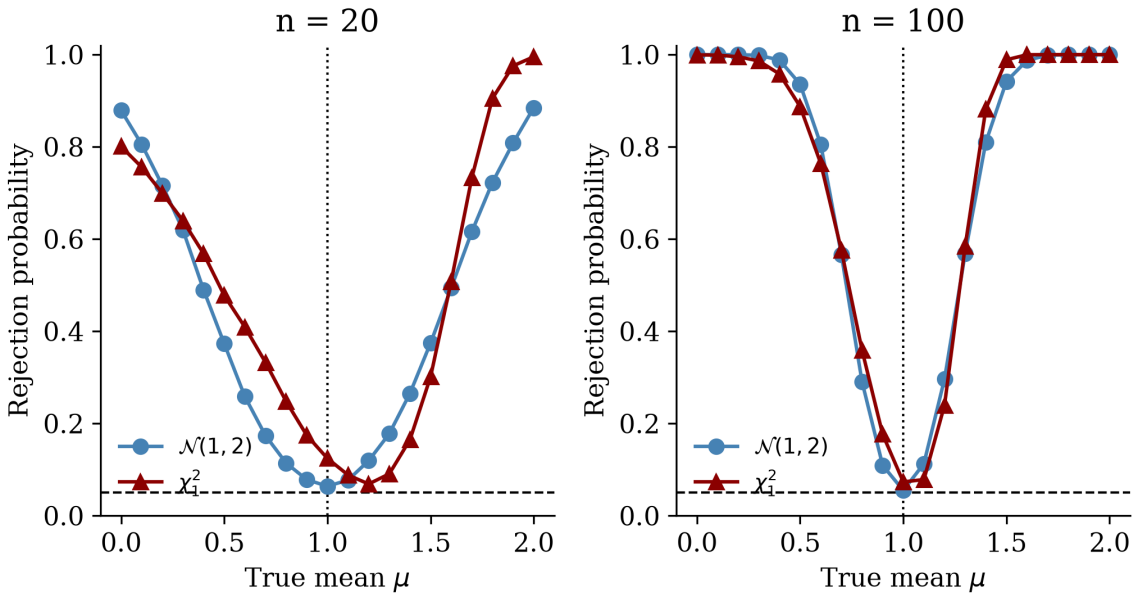


Figure 14.4: Monte Carlo rejection probabilities of the t-test of $H_0 \colon \mu = 1$ as functions of the true mean μ. At $\mu = 1$ (dotted line), the curves show the size; elsewhere, they show the power. The dashed line marks $\alpha = 0.05$.

Suppose $X \sim \mathcal{U}[0, 2]$, where $\mathcal{U}[0, 2]$ denotes the uniform distribution on the interval $[0, 2]$. It is easy to show (make sure to verify this!) that $E(X) = 1$. Now repeat the following two steps at least 1,000 times:

1. Draw a random sample of size $n = 20$ from the distribution of X.
2. Compute the sample mean and a 95% confidence interval for $E(X)$ from this sample.

You now have 1,000 sample means and 1,000 confidence intervals. Use them to answer the following questions:

a. What is the average of your 1,000 sample means?

b. What is the standard deviation of your 1,000 sample means?

c. How often does the 95% confidence interval contain $E(X)$?

Exercise 2:

In this exercise, we use the Monte Carlo method to approximate the number π. Randomly draw n points from a square and count how many of them lie inside the square's incircle. Based on this count and the formula for the area of a circle, compute an approximation of π.

15 Regression Analysis

15.1 Basic Linear Model

You have probably all learnt about the linear regression model in a previous econometrics class where

$$Y = \beta_1 X_1 + \beta_2 X_2 + ... + \beta_k X_k + \beta_{k+1} + e$$

and

$$E(e \mid X_1, X_2, ..., X_k) = 0.$$

That is, the conditional mean of the outcome variable Y is a linear function of the regressors $X_1, X_2, ..., X_k$.

When we have a random sample of size n, we can also write

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_k X_{ik} + \beta_{k+1} + e_i, \qquad i = 1, 2, ..., n.$$

Our goal is to estimate the coefficients $\beta_1, ..., \beta_{k+1}$ using the data $\{Y_i, X_{i1}, X_{i2}, ..., X_{ik}\}_{i=1}^n$.

15.2 Empirical Example

We will now see how to run regressions in Python using the movie data set and the package **statsmodels**. Again, you can load the data using

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

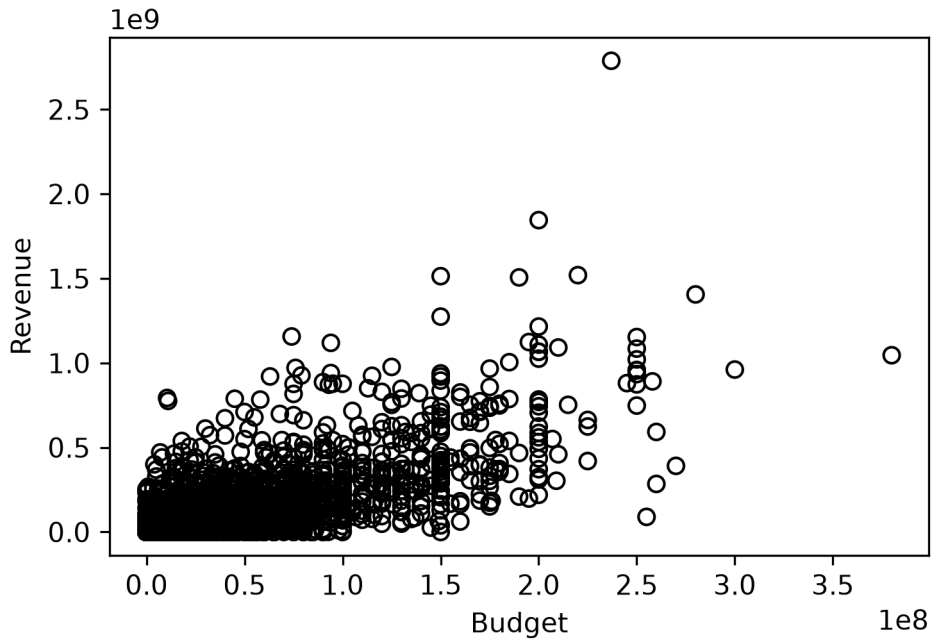
data_set = pd.read_csv("tmdb_5000_movies.csv")
```

Suppose we are interested in studying how the budget of a movie is related to its revenue and we model the relationship as:

$$revenue_i = \beta_1 budget_i + \beta_2 + e_i$$

Before we run a regression, let's look at a scatter plot of the relevant variables:

```
plt.scatter(data_set["budget"], data_set["revenue"], facecolors = "none",
 ↪  edgecolors = "black")
plt.xlabel("Budget")
plt.ylabel("Revenue")
plt.show()
```



Clearly, movies with higher budgets tend to have higher revenues. To estimate the regression coefficients, you can use the `smf.ols()` command, where the model is specified by a formula as in R, and the method `fit()` estimates the model:

```
lm_result = smf.ols(formula = "revenue ~ budget", data = data_set).fit()
print(lm_result.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                revenue   R-squared:                       0.534
Model:                            OLS   Adj. R-squared:                  0.534
Method:                 Least Squares   F-statistic:                     5504.
Date:                Sun, 08 Nov 2026   Prob (F-statistic):               0.00
Time:                        23:01:05   Log-Likelihood:                -95797.
No. Observations:                4803   AIC:                         1.916e+05
Df Residuals:                    4801   BIC:                         1.916e+05
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
```

```
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   -2.63e+06   1.97e+06     -1.335      0.182   -6.49e+06    1.23e+06
budget         2.9227      0.039     74.188      0.000       2.845       3.000
==============================================================================
Omnibus:                     4260.045   Durbin-Watson:                   1.513
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           381126.754
Skew:                           3.828   Prob(JB):                         0.00
Kurtosis:                      45.963   Cond. No.                     6.14e+07
==============================================================================

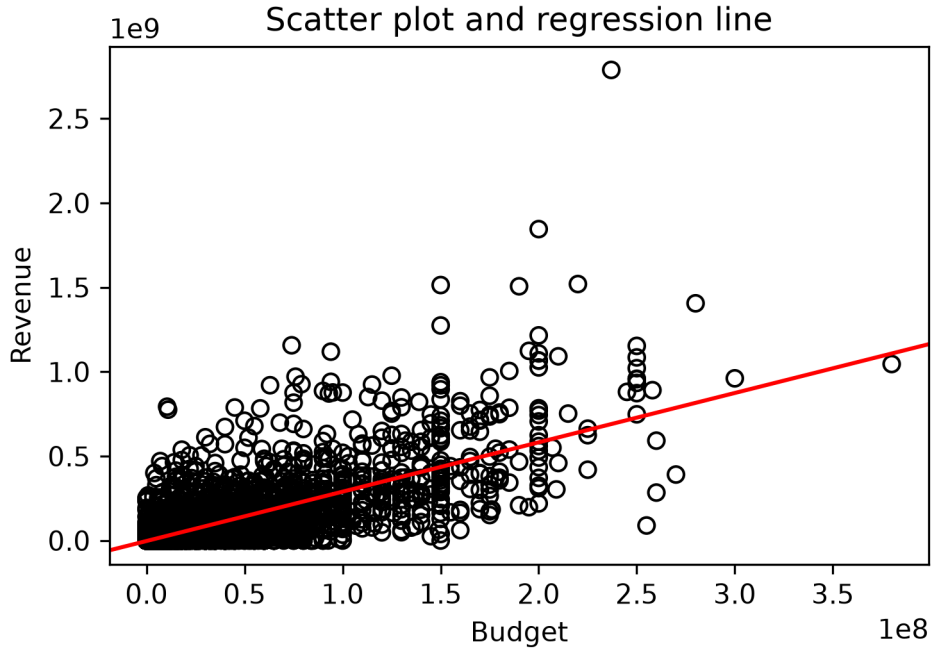
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified
[2] The condition number is large, 6.14e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Think about how you can interpret the estimated slope coefficient.

We can also plot the regression line in the scatter plot:

```
b0 = lm_result.params["Intercept"]
b1 = lm_result.params["budget"]

## Plot all:
plt.scatter(data_set["budget"], data_set["revenue"], facecolors = "none",
 ↪  edgecolors = "black")
plt.axline((0, b0), slope = b1, color = "red")
plt.xlabel("Budget")
plt.ylabel("Revenue")
plt.title("Scatter plot and regression line")
plt.show()
```



Let's now see if a higher budget also implies a higher rating:

```
lm_result_rating = smf.ols(formula = "vote_average ~ budget", data =
↪  data_set).fit()
print(lm_result_rating.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:           vote_average   R-squared:                       0.009
Model:                            OLS   Adj. R-squared:                  0.008
Method:                 Least Squares   F-statistic:                     42.02
Date:                Sun, 08 Nov 2026   Prob (F-statistic):           9.95e-11
Time:                        23:01:05   Log-Likelihood:                -7647.8
No. Observations:                4803   AIC:                         1.530e+04
Df Residuals:                    4801   BIC:                         1.531e+04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      6.0128      0.021    285.190      0.000       5.971       6.054
budget      2.732e-09   4.22e-10      6.482      0.000    1.91e-09    3.56e-09
==============================================================================
Omnibus:                     2034.362   Durbin-Watson:                   1.890
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            13977.313
```

```
Skew:                          -1.882   Prob(JB):                         0.00
Kurtosis:                      10.462   Cond. No.                     6.14e+07
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified
[2] The condition number is large, 6.14e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Notice that the estimated slope coefficient is tiny. Does that mean that the rating is unrelated to the budget?

The budget is often huge. For a better interpretation of the coefficients, define the budget in millions as

```
data_set["budget_mil"] = data_set["budget"]/1000000
```

look at summary statistics of the variables

```
data_set[["vote_average", "budget_mil"]].describe()
```

	vote_average	budget_mil
count	4803.000000	4803.000000
mean	6.092172	29.045040
std	1.194612	40.722391
min	0.000000	0.000000
25%	5.600000	0.790000
50%	6.200000	15.000000
75%	6.800000	40.000000
max	10.000000	380.000000

and redo the analysis:

```
lm_result_rating = smf.ols(formula = "vote_average ~ budget_mil", data =
 ↪  data_set).fit()
print(lm_result_rating.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:           vote_average   R-squared:                       0.009
Model:                            OLS   Adj. R-squared:                  0.008
Method:                 Least Squares   F-statistic:                     42.02
```

```
Date:                Sun, 08 Nov 2026   Prob (F-statistic):           9.95e-11
Time:                        23:01:06   Log-Likelihood:                -7647.8
No. Observations:                4803   AIC:                         1.530e+04
Df Residuals:                    4801   BIC:                         1.531e+04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      6.0128      0.021    285.190      0.000       5.971       6.054
budget_mil     0.0027      0.000      6.482      0.000       0.002       0.004
==============================================================================
Omnibus:                     2034.362   Durbin-Watson:                   1.890
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            13977.313
Skew:                          -1.882   Prob(JB):                         0.00
Kurtosis:                      10.462   Cond. No.                         61.4
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified
```

Do think that the budget has a large impact on the rating?

Of course, we are not limited to one regressors. To include three regressors, can use `smf.ols(formula = "Y ~ X1 + X2 + X3", data = ...)`. We will return to regression analysis later in the semester.

15.3 Exercises VII

Exercise 1:

In the previous section, we looked at Monte Carlo simulations in a very simple setting. We can analyze analogous issues in other settings, such as the linear regression model. Also in this case, we have parameters (the regression coefficients) and estimators (the OLS estimators). The parameters are random and have certain known small and large sample properties that you discussed in the theoretical part of the class. Many other small sample properties, such as actual coverage rates of confidence intervals are unknown, but can be studies in Monte Carlo simulations.

Let

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i2} + \beta_3 + e_i, \qquad i = 1, 2, \ldots, n.$$

where

$$\begin{pmatrix} X_{i1} \\ X_{i2} \end{pmatrix} \sim N\left(\begin{pmatrix} 2 \\ -3 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \right)$$

and $(\beta_1, \beta_2, \beta_3) = (2, 1, -1)$. Let $e_i \sim U[-2, 2]$ and assume that e_i is independent of (X_{i1}, X_{i2}). Assume that the data is a random sample. Let $(\hat{\beta}_1, \hat{\beta}_2, \hat{\beta}_3)$ be the OLS estimator of $(\beta_1, \beta_2, \beta_3)$.

For different values of n and ρ use simulations to calculate

- $E(\hat{\beta}_1)$,
- $Var(\hat{\beta}_1)$,
- the probability that β_1 is in a 95% confidence interval,
- the expected length of the confidence interval,
- the probability that we reject $H_0 : \beta_1 = 2$ using a two-sided t-test and a 1% significance level, and
- the probability that we reject $H_0 : \beta_1 = 1.8$ using a two-sided t-test and a 1% significance level.

Discuss the results.

How do you think the results will change if X_{i1} and e_i are correlated?

Exercise 2:

Start with the help files `help(smf.ols)` and `help(lm_result)` of the linear model - one of the models most often used in statistics - and try to answer these questions:

- How do you estimate a model without a constant? (Hint: the formula syntax is the same as in R.)
- In what setting could `missing = "raise"` be more useful to you than the function's default value `missing = "drop"`?
- For a fitted model, what does the method `summary()` do? Which other methods and attributes (e.g., `params`, `bse`, `conf_int()`, `resid`) are available?

Exercise 3:

Assume that $X \sim U[0, 1]$ and $Y \sim U[0, 1]$ are independent random variables. For different sample sizes ($n \in \{2, 5, 10, 100, 1000\}$) draw two i.i.d. samples of X and Y and regress Y on X for both samples. Using a Monte-Carlo simulation, approximate the probability that the two corresponding regression lines intersect on $[0, 1] \times [0, 1]$. Do the results depend on the sample size?

16 Some Notes on Useful Packages

16.1 The PyData Family

NumPy, pandas, matplotlib, and seaborn, which we used throughout this tutorial, form the core of the so-called PyData ecosystem. Together, they provide a comprehensive framework for many useful tools we need: from reading in data with pandas [csv, Stata, SPSS, SAS, Excel etc.], wrangling it into the shape one desires with pandas to visualization with matplotlib and seaborn. A comprehensive guide is provided in Wes McKinney's (the creator of pandas) book Python for Data Analysis. A fast, modern alternative to pandas is the package polars.

16.2 scikit-learn

`"Simple and efficient tools for predictive data analysis."` The standard package for machine learning in Python that can be used for a multitude of tasks, such as regression, classification, clustering, and model selection. https://scikit-learn.org/

16.3 statsmodels

`"statsmodels is a Python module that provides classes and functions for the estimation of many different statistical models, as well as for conducting statistical tests, and statistical data exploration."`

Besides the linear regression model, statsmodels provides, e.g., generalized linear models, time series models, and diagnostic tests. It can also calculate many different robust covariance matrix estimators. A simple application for heteroskedasticity consistent covariance estimation in an OLS setting is given below.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

rng = np.random.default_rng()

x = rng.uniform(low = 0, high = 10, size = 1000)
eps = rng.normal(loc = 0, scale = np.sqrt(x), size = 1000)
```

```
y = 1.2*x + eps
my_data = pd.DataFrame({"x": x, "y": y})

my_model = smf.ols("y ~ x", data = my_data).fit()
my_model_hc = smf.ols("y ~ x", data = my_data).fit(cov_type = "HC1")
```

The first table describes our estimates when we assume homoskedastic error terms, while the second one correctly assumes heteroskedasticity.

```
print(my_model.summary().tables[1])
print(my_model_hc.summary().tables[1])
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -0.0350      0.144     -0.243      0.808      -0.318       0.248
x              1.2322      0.025     48.764      0.000       1.183       1.282
==============================================================================
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -0.0350      0.104     -0.337      0.736      -0.239       0.169
x              1.2322      0.026     48.044      0.000       1.182       1.282
==============================================================================
```

16.4 SciPy

Collection of many useful functions for scientific computing, such as optimization, integration, linear algebra, and statistics. Its module `scipy.stats` provides densities, distribution functions, quantile functions, and random number generators for a large number of distributions. For drawing from a multivariate normal with a given mean vector and covariance matrix, one can also directly use NumPy's random number generator.

```
my_sample = rng.multivariate_normal(mean = [0,0], cov = [[1,0.5],[0.5,1]],
 ↪  size = 100)
my_sample[0:6,:]
```

```
array([[ 0.70969674, -0.33657496],
       [ 0.32844052, -0.82551647],
       [ 0.01187946, -1.53156329],
       [-0.15803953, -1.41188271],
       [ 0.76239808, -0.9029202 ],
       [ 1.13674724,  1.30162234]])
```

16.5 linearmodels

Provides many useful functions in the context of applied Econometrics, such as panel data models and system estimators. One prominent example is the class `IV2SLS` used for IV-estimation. https://bashtage.github.io/linearmodels/

16.6 Jupyter and Quarto

Jupyter notebooks allow you to use Python interactively, intermingling code and text freely. Quarto can be especially useful in your studies, if you want to use Python (or R) in conjunction with markdown and output to different files. For example, this introduction is created from a bunch of qmd files [Quarto Markdown].

16.7 Streamlit and Shiny for Python

Used to build interactive web-apps with Python. Somewhat more advanced, but very cool. https://streamlit.io/ and https://shiny.posit.co/py/

16.8 Faster Code

You might want to look at these packages:

- `numba`
- `multiprocessing` & `joblib`
- `polars`
- `jax`

These can get complicated very quickly, so beware.